

Phenix Overview and General Reference (Part 3 of 5)

20 documentation pages

Generated from local Phenix documentation

Contents

1. Finding a program or test using `find_program`
2. Fitting loops to fill in gaps in models with `fit_loops`
3. Fixing register errors in a model with `fix_insertions_deletions`
4. `<string>`
5. Find matching fragments in the PDB (`fragment_search`)
6. Correlation of two maps (deprecated, use `MapCorrelations`)
7. Correlation of map and model (deprecated, use `MapCorrelations`)
8. Guess location of molecular centers in a map with `guess_molecular_centers`
9. `phenix.reduce`: tool for adding hydrogens to a PDB model
10. Phenix Documentation - version dev-svn
11. Optimizing a map with `local_aniso_sharpen` (deprecated, use `MapSharpening`)
12. `phenix.model_idealization`
13. FFT-ing map coefficients
14. Density modification with multi-crystal averaging with `phenix.multi_crystal_average`
15. Building biological assembly and crystallographic asymmetric unit from a PDB file
16. Calculating a simple NCS averaged map
17. Model building with Pathwalker

18. Rapid phase improvement and model-building using phase_and_build

19. <string>

20. Planning a SAD experiment with plan_sad_experiment

Finding a program or test using find_program

find_program.html

Author(s)

- find_program: Tom Terwilliger

Purpose

The routine find_program will show all the programs that match supplied text (either in program name or in program descriptions). The locations of the programs (which tab in the GUI or command-line name) are shown.

Alternatively, find_program can search for a regression test based on words in the name of the test or based on the name of a module that is tested by that regression test.

Lastly, find_program can find all the Python programs that have changed since last checking in programs to github and list all the regression tests that test those programs.

Usage

How find_program works:

Searches for programs:

Find_program uses the list of programs and short descriptions in the file \$PHENIX/modules/phenix/phenix/utilities/phenix_list_help.py as a list of program names. Additionally it compiles program names and descriptions from \$PHENIX/modules/phenix/wxGUI2/Programs/__init__.params and \$PHENIX/modules/phenix/wxGUI2/Home/Tools.py.

Find_program then makes a list of all programs, descriptions, and locations.

This list is then searched with the search_text values provided.

Searches for regression tests:

Find_program can also look for and display regression tests. A database (test_and_module_info.pkl) of regression tests and all the modules called by each test is created using the tool phenix_regression.get_test_and_module_info. This database is read by find_program and can be searched either by words from the name of a regression test (providing the full path of the regression test), or by words from the name of a module (providing the full path of regression tests that use that module).

The search for regression tests by module is a way to find a test that will run a module that you are interested in.

Search for regression tests affecting all changed programs:

Find_program can find all the Python programs that have changed since the last check-in to github (it uses git to do this). Then for each of these programs, it finds all the regression tests that are affected. Then it lists all

the regression tests it finds.

This search is a way to find all the tests that could possibly fail due to any changed Python module. Note that this search will not find C++ code that is affected. It will also not find code that is accessed in tests using the `easy_run` command which uses a system call that does not pass back traceback information.

Running regression tests

The standard framework for running regression tests is described in [Running Phenix regression tests](#) . This approach uses `find_program` if you use the `git` or `sort` options.

Standard run of `find_program`:

You can use `find_program` to find a program like this:

```
phenix.find_program search_text=autosol
```

You can also use it to try and find a regression test that exercises a routine with the word `anomalous` in it (anywhere) and also the word `signal` (anywhere):

```
phenix.find_program search_type=tests search_tests_by=function_called search_text=anomalous search_text=signal
```

Setting up the database for `find_program`:

You can update the database for `find_program` by running the tool

```
phenix_regression.get_test_and_module_info nproc=64
```

in an empty directory. This will take a while (perhaps an hour or so with `nproc=64`) and produce a file called:

```
test_and_module_info.pkl
```

You can copy this file to its official location at:

```
$PHENIX/modules/phenix_regression/misc/test_and_module_info.pkl
```

and then `find_program` will use this new version. You can also specify where this `.pkl` file is located with the keyword:

```
phenix_test_database_file=xxx.pkl
```

Possible Problems

Specific limitations and problems:

The `search_tests` and `git_affected_tests` features of `find_program` do not find C++ code that is affected by a test (only Python code).

Literature

Additional information

List of all available keywords

`all_tests = False` Find all regression tests
`git_affected_tests = False` Find regression tests to cover all changed Python files in
`githubdirectories_to_search = phenix cctbx_project solve_resolve` Directories to search to find

changed filessearch_type = *programs tests You can search for programs (real_space_refine) or tests (tst_ncs.py). For programs, your search text must match part of the program name or its description. For tests you can search based on the name of the test or based on the name of any function that is accessed by the test.search_text = None Text to search for. If multiple, any match is displayed. If search_text=ANY (must be capitalized) or None (empty) then searches will match any text. Each entry of search_text counts as one search term. So if there are two entries of search text like < density modification > and <autosol> , then if all_search_terms_required=True, the phrase < density modification > and the word < autosol > must both appear somewhere in the name or description to be selected.case_sensitive_search = False Normally searches are case-insensitive. You can force a case-sensitive search with this keyword.all_search_terms_required = True All search terms required (a AND b AND c). Alternative is to match any search terms (a OR b OR c). Each entry of search_text counts as one search term. So if there are two entries of search text like < density modification > and <autosol> , then if all_search_terms_required=True, the phrase < density modification > and the word < autosol > must both appear somewhere in the name or description to be selected.try_splitting_text = True If no programs are found, try splitting search text entries into individual words and search for all the words in any order.ignore_hyphens = True Ignore hyphens in searchesnumber_to_display = 20 Number of programs or tests to display. Up to this number of entries will be shown.show_tests_by_module = True Show tests by modulefind_programs_without_tests = None Find programs without testsjob_title = None Job title in PHENIX GUI, not used on command lineinput_filesprogram_data_file_name = None Input data file name with program informationread_program_data_if_available = True Read program dataphenix_test_database_file = None Input database file (.pkl) with information about all Phenix tests and modules that are called by those tests. This is normally found automaticallyoutput_filesprogram_data_file_name = None Output data file name with program informationfiles_with_tests_file_name = files_with_tests.txt Output data file name with list of files with testsfiles_with_no_tests_file_name = files_with_no_tests.txt Output data file name with list of files with no testsmodules_with_tests_file_name = modules_with_tests.txt Output data file name with list of modules with testsmodules_with_no_tests_file_name = modules_with_no_tests.txt Output data file name with list of modules with no testsprogramsinclude_program_description = True Search in documentationstringtestssearch_tests_by = *test_name function_called For tests you can search based on the name of any function that is called by the test. This is a way to find a test that will run a function that you are interested in. The alternative is to search by the name of the test.guiGUI-specific parameter required for output directoryoutput_dir = None

- all_tests = False Find all regression tests
- git_affected_tests = False Find regression tests to cover all changed Python files in github
- directories_to_search = phenix cctbx_project solve_resolve Directories to search to find changed files
- search_type = *programs tests You can search for programs (real_space_refine) or tests (tst_ncs.py). For programs, your search text must match part of the program name or its description. For tests you can search based on the name of the test or based on the name of any function that is accessed by the test.
- search_text = None Text to search for. If multiple, any match is displayed. If search_text=ANY (must be capitalized) or None (empty) then searches will match any text. Each entry of search_text counts as one search term. So if there are two entries of search text like < density modification > and <autosol> , then if all_search_terms_required=True, the phrase < density modification > and the word < autosol > must both appear somewhere in the name or description to be selected.
- case_sensitive_search = False Normally searches are case-insensitive. You can force a case-sensitive search with this keyword.
- all_search_terms_required = True All search terms required (a AND b AND c). Alternative is to match any search terms (a OR b OR c). Each entry of search_text counts as one search term. So if there are

two entries of search text like < density modification > and <autosol> , then if all_search_terms_required=True, the phrase < density modification > and the word < autosol > must both appear somewhere in the name or description to be selected.

- try_splitting_text = True If no programs are found, try splitting search text entries into individual words and search for all the words in any order.
- ignore_hyphens = True Ignore hyphens in searches
- number_to_display = 20 Number of programs or tests to display. Up to this number of entries will be shown.
- show_tests_by_module = True Show tests by module
- find_programs_without_tests = None Find programs without tests
- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesprogram_data_file_name = None Input data file name with program information
- read_program_data_if_available = True Read program data
- phenix_test_database_file = None Input database file (.pkl) with information about all Phenix tests and modules that are called by those tests. This is normally found automatically
- program_data_file_name = None Input data file name with program information
- read_program_data_if_available = True Read program data
- phenix_test_database_file = None Input database file (.pkl) with information about all Phenix tests and modules that are called by those tests. This is normally found automatically
- output_filesprogram_data_file_name = None Output data file name with program information
- files_with_tests_file_name = files_with_tests.txt Output data file name with list of files with tests
- files_with_no_tests_file_name = files_with_no_tests.txt Output data file name with list of files with no tests
- modules_with_tests_file_name = modules_with_tests.txt Output data file name with list of modules with tests
- modules_with_no_tests_file_name = modules_with_no_tests.txt Output data file name with list of modules with no tests
- programsinclude_program_description = True Search in documentation string
- include_program_description = True Search in documentation string
- testssearch_tests_by = *test_name function_called For tests you can search based on the name of any function that is called by the test. This is a way to find a test that will run a function that you are interested in. The alternative is to search by the name of the test.
- search_tests_by = *test_name function_called For tests you can search based on the name of any function that is called by the test. This is a way to find a test that will run a function that you are interested in. The alternative is to search by the name of the test.
- guiGUI-specific parameter required for output directoryoutput_dir = None

- output_dir = None

Fitting loops to fill in gaps in models with fit_loops

fit_loops.html

Contents

- Author(s)
- Purpose
- Usage
- How fit_loops works:
- Output files from fit_loops
- Examples
- Standard run of fit_loops:
- Specifying column names
- Fitting a loop with poor density
- Using a loop library
- Using a loop library to connect all segments
- Specifying a gap to fill in
- Possible Problems
- Specific limitations and problems:
- Literature
- Additional information
- List of all available keywords

Author(s)

- fit_loops: Tom Terwilliger

Purpose

fit_loops is a tool for building a loop into density to connect existing chain ends. You supply a model with a gap and a sequence file and coefficients for an electron density map, and you specify the first and last residues to be built. Then fit_loops will attempt to build the loop that you specify. One loop can be done at a time with fit_loops (but if you have multiple identical chains, you can fit them all at once).

You can use either of two methods to fit the loops. By default fit_loops uses resolve chain extension to try to trace residues from the ends of segments in your input PDB file. If it can connect the segments, it writes out the connecting loops. Alternatively you can use a loop library supplied with PHENIX to connect ends of segments from your input PDB file.

If you want a more complete model-building process, then you will want to use phenix.autobuild .

fit_loops can be run from the command line or from the PHENIX GUI.

Usage

How fit_loops works:

fit_loops calculates a map based on the supplied map coefficients, then tries to extend the ends of the supplied model into the gap region, following the electron density in the map.

Output files from fit_loops

model_with_loops.pdb: The output from fit_loops is a new PDB file containing your input model with the newly-built loop inserted into it (if a loop could be found).

Examples

Standard run of fit_loops:

A typical command-line input would be:

```
phenix.fit_loops pdb_in=nsf_gap.pdb mtz_in=map_coeffs.mtz \  
seq_file=nsf.seq start=37 end=43 chain_id=None
```

This will fit a loop starting with residue 37 and ending with residue 43 in nsf_gap.pdb. phenix.fit_loops will expect that your existing nsf_gap.pdb model has a chain ending at residue 36 and another starting at residue 44. As chain_id=None in this example, if there are multiple chains A,B,C in nsf_gap.pdb then all 3 will be filled in.

Specifying column names

If you want (or need) to specify the column names from your mtz file, you will need to tell fit_loops what FP and PHIB (and optionally FOM) are, in this format:

```
phenix.fit_loops pdb_in=nsf_gap.pdb mtz_in=map_coeffs.mtz \  
seq_file=nsf.seq start=37 end=43 chain_id=None \  
labin="FP=2F0FCWT PHIB=PH2F0FCWT"
```

Fitting a loop with poor density

If you want to try and fit a loop with poor density, you might want to lower the threshold for the correlation of density in the loop (default minimum correlation is 0.2):

```
phenix.fit_loops pdb_in=nsf_gap.pdb mtz_in=map_coeffs.mtz \  
seq_file=nsf.seq start=37 end=43 chain_id=None \  
loop_cc_min=0.1
```

Using a loop library

To use the loop library in PHENIX, use the keyword loop_lib:

```
phenix.fit_loops pdb_in=nsf_gap.pdb mtz_in=map_coeffs.mtz \  
seq_file=nsf.seq start=37 end=39 chain_id=None loop_lib=True
```

This will fit a loop starting with residue 37 and ending with residue 39. The maximum current length in the loop library is 3 residues.

Using a loop library to connect all segments

To use the loop library in PHENIX and try to connect any pair of segments that have an geometrical relationship, use the keyword connect_all:

```
phenix.fit_loops pdb_in=nsf_gap.pdb mtz_in=map_coeffs.mtz \
seq_file=nsf.seq connect_all=True
```

This will go through all pairs of segments, trying to connect them with a loop from the PHENIX loop library. Note that this is a last-resort approach, normally instead use the default and let fit_loops connect segments that are close in sequence.

Specifying a gap to fill in

To specify a particular gap for loop fitting, you can say

```
phenix.fit_loops pdb_in=nsf_gap.pdb mtz_in=map_coeffs.mtz \
seq_file=nsf.seq start=37 end=39 chain_id=A
```

Note the residue number for start and end are the first and last in the gap, not the residues flanking the gap that are already present. This command fits a gap present at residues 37-39. Note: if any of the residues from 37-39 are still present in your model, specify remove_loop=True and fit_loops will remove them (otherwise it will just fit the existing gap).

Possible Problems

One of the most common problems is that the residues just before and after a gap are themselves poorly fitted. A solution is to remove the last couple residues before a loop and the first couple residues after the loop and to try and fit the resulting larger gap. To do this you can say something like:

```
phenix.fit_loops pdb_in=nsf_gap.pdb mtz_in=map_coeffs.mtz \
seq_file=nsf.seq start=35 end=41 chain_id=A replace_residues=True
```

Here the replace_residues=True command is required to tell fit_loops to remove residues 35 and 36 and residues 40 and 41 before trying to fit residues 35-41.

You can also accomplish this by leaving the parameter trim_ends_if_needed=True, which will do this automatically (trying several possible trimmings of the ends).

Specific limitations and problems:

Literature

Additional information

List of all available keywords

input_files_pdb_in = None Model with gap to fill. mtz_in = None MTZ file with coefficients for a map
mtz_in_virtual = None Used internally map_coeff_labels = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: map_coeff_labels (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM) labin = "" Labin line for MTZ file with map coefficients. Normally use instead map_coeff_labels. This is available for backward compatibility. You can specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP map_in = None CCP4 or MRC-style map file seq_file = None sequence file (1-letter code, one copy of each chain) seq_prob_file = None File seq_prob.dat from resolve sequence alignment output_files_pdb_out = connect.pdb Output PDB file (will be missing if no result). results_as_json = None Output result summary as JSON log = None Output logfile

params_out = fit_loops_params.eff Parameters file to rerun fit_loops
 fitting_min_acceptable_prob = None Minimum loop probability to consider
 refine_loops = True Refine fitted loops in loop_lib
 chain_id = None Chain ID for chain containing missing loops. If None any chain ID is allowed. All missing segments matching chain_id, start and end will be fit. If chain_id is specified without start and end, all gaps in that chain will be fit. If no chain_id and no start and end are specified, all gaps are fit.
 start = None Starting residue number of loop(s) to be fit. NOTE: This is the residue number of the first residue that is NOT present in your model already (one more than the residue number of the residue just before the gap). If None any start is allowed. All missing segments matching chain_id, start and end will be fit.
 end = None Ending residue number of loop(s) to be fit. NOTE: This is the residue number of the last residue that is NOT present in your model already (one fewer than the residue number of the residue just after the gap). If None any end is allowed. All missing segments matching chain_id, start and end will be fit.
 insert_or_delete_residues = None Try to insert or delete this many residues in the loop (ignore loop sequence if so and use any side chain).
 remove_loops = True Remove existing residues and replace with new loop. All segments matching chain_id, start and end will be fit.
 skip_trim = True If skip_trim=True (default) then model with added loops will be written out without checking for overlaps with non-sequence-aligned residues. If skip_trim=False then this check will be carried out. Note that skip_trim=False can cause some residues to be de-assigned from sequence if they cannot be successfully matched to density. In such cases you might try skip_trim=True.
 score_min = 1.0 Minimum connection score for connect_all_segments.
 min_dist = 3. Minimum distance between connections for connect_all_segments.
 min_log_prob = -5. Minimum log(P) for sequence to consider locations.
 skip_seq_prob = False Skip sequence probability calculations.
 save_acceptable_loops = False Just return a file with possible loops.
 sn_random_loop = 200 Number of tries at building loops.
 connect_all_segments = False Try to connect all segments to each other, regardless of sequence and residue numbers. Note: this is a last-resort approach. Normally just use the default and let fit_loops connect segments that are close in sequence.
 sequential_only = False Only connect adjacent segments in connect_all_segments.
 dist_max = 15 Maximum CA-CA distance in connect_all_segments.
 ignore_sequence_register = False Ignore the input sequence register.
 all_assigned = True Assume all residues in model can be assigned to sequence. If you set all_assigned=False then any residues with residue number greater than the number of residues in the longest sequence in the sequence file are...

- input_files
 pdb_in = None Model with gap to fill.
 mtz_in = None MTZ file with coefficients for a map.
 mtz_in_virtual = None Used internally.
 map_coeff_labels = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: map_coeff_labels (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM).
 labin = "" Labin line for MTZ file with map coefficients. Normally use instead map_coeff_labels. This is available for backward compatibility. You can specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP.
 map_in = None CCP4 or MRC-style map file.
 seq_file = None sequence file (1-letter code, one copy of each chain).
 seq_prob_file = None File seq_prob.dat from resolve sequence alignment.
- pdb_in = None Model with gap to fill.
- mtz_in = None MTZ file with coefficients for a map
- mtz_in_virtual = None Used internally
- map_coeff_labels = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: map_coeff_labels (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM)

- `labin = ""` Labin line for MTZ file with map coefficients. Normally use instead `map_coeff_labels`. This is available for backward compatibility. You can specify: `LABIN FP=myFP PHIB=myPHI FOM=myFOM` where `myFP` is your column label for FP
- `map_in = None` CCP4 or MRC-style map file
- `seq_file = None` sequence file (1-letter code, one copy of each chain)
- `seq_prob_file = None` File `seq_prob.dat` from resolve sequence alignment
- `output_files`
 - `pdb_out = connect.pdb` Output PDB file (will be missing if no result).
 - `results_as_json = None` Output result summary as JSON
 - `log = None` Output logfile
 - `params_out = fit_loops_params.eff` Parameters file to rerun `fit_loops`
- `fittingmin_acceptable_prob = None` Minimum loop probability to consider
 - `refine_loops = True` Refine fitted loops in `loop_libchain_id = None` Chain ID for chain containing missing loops. If `None` any chain ID is allowed. All missing segments matching `chain_id`, start and end will be fit. If `chain_id` is specified without start and end, all gaps in that chain will be fit. If no `chain_id` and no start and end are specified, all gaps are fit.
 - `start = None` Starting residue number of loop(s) to be fit. NOTE: This is the residue number of the first residue that is NOT present in your model already (one more than the residue number of the residue just before the gap). If `None` any start is allowed. All missing segments matching `chain_id`, start and end will be fit.
 - `end = None` Ending residue number of loop(s) to be fit. NOTE: This is the residue number of the last residue that is NOT present in your model already (one fewer than the residue number of the residue just after the gap). If `None` any end is allowed. All missing segments matching `chain_id`, start and end will be fit.
 - `insert_or_delete_residues = None` Try to insert or delete this many residues in the loop (ignore loop sequence if so and use any side chain)
 - `remove_loops = True` Remove existing residues and replace with new loop. All segments matching `chain_id`, start and end will be fit.
 - `skip_trim = True` If `skip_trim=True` (default) then model with added loops will be written out without checking for overlaps with non-sequence-aligned residues. If `skip_trim=False` then this check will be carried out. Note that `skip_trim=False` can cause some residues to be de-assigned from sequence if they cannot be successfully matched to density. In such cases you might try `skip_trim=True`.
 - `score_min = 1.0` Minimum connection score for `connect_all_segments`
 - `min_dist = 3` Minimum distance between connections for `connect_all_segments`
 - `min_log_prob = -5` Minimum log(P) for sequence to consider
 - `locationskip_seq_prob = False` Skip sequence probability calculations
 - `save_acceptable_loops = False` Just return a file with possible loops
 - `sn_random_loop = 200` Number of tries at building loops
 - `connect_all_segments = False` Try to connect all segments to each other, regardless of sequence and residue numbers. Note: this is a last-resort approach. Normally just use the default and let `fit_loops` connect segments that are close in sequence.
 - `sequential_only = False` Only connect adjacent segments in `connect_all_segments`
 - `dist_max = 15` Maximum CA-CA distance in `connect_all_segments`
 - `ignore_sequence_register = False` Ignore the input sequence register
 - `all_assigned = True` Assume all residues in model can be assigned to sequence. If you set `all_assigned=False` then any residues with residue number greater than the number of residues in the longest sequence in the sequence file are assumed not to be assigned to sequence. This is useful if you are using resolve model-building as resolve will identify non-sequence-aligned residues with high residue numbers.
 - `sequence_offset = None` You can specify that `fit_loops` should offset input sequence file residue numbers by `sequence_offset` before use (same effect as adding `sequence_offset*X` to the beginning of

the sequence file.) Note: number of of sequence_offset values must match number of chains in sequence file
 loop_cc_min = 0.2 Minimum loop map-model correlation
 aggressive = False Aggressive loop building (risky)
 target_insert = None You can try to force the number of residues to insert with trace_loops. If none, try to fill in the gap based on the number of missing residues. If set and greater than 0, only accept the number of residues given. If zero, take any length. Not supported.
 time_per_residue = None You can specify how long to try in fitting (sec/residue)
 trim_ends_if_needed = True If a single loop is specified and no loop is found, try trimming back the ends.
 loop_backup_residues = None Number of tries removing one residue at a time from each end of existing ends of loop if no loop is found with initial ...

- min_acceptable_prob = None Minimum loop probability to consider
- refine_loops = True Refine fitted loops in loop_lib
- chain_id = None Chain ID for chain containing missing loops. If None any chain ID is allowed. All missing segments matching chain_id, start and end will be fit. If chain_id is specified without start and end, all gaps in that chain will be fit. If no chain_id and no start and end are specified, all gaps are fit.
- start = None Starting residue number of loop(s) to be fit. NOTE: This is the residue number of the first residue that is NOT present in your model already (one more than the residue number of the residue just before the gap). if None any start is allowed. All missing segments matching chain_id, start and end will be fit
- end = None Ending residue number of loop(s) to be fit. NOTE: This is the residue number of the last residue that is NOT present in your model already (one fewer than the residue number of the residue just after the gap). If None any end is allowed. All missing segments matching chain_id, start and end will be fit
- insert_or_delete_residues = None Try to insert or delete this many residues in the loop (ignore loop sequence if so and use any side chain)
- remove_loops = True Remove existing residues and replace with new loop All segments matching chain_id, start and end will be fit
- skip_trim = True If skip_trim=True (default) then model with added loops will be written out without checking for overlaps with non-sequence-aligned residues. If skip_trim=False then this check will be carried out. Note that skip_trim=False can cause some residues to be de-assigned from sequence if they cannot be successfully matched to density. In such cases you might try skip_trim=True."
- score_min = 1.0 Minimum connection score for connect_all_segments
- min_dist = 3. Minimum distance between connections for connect_all_segments"
- min_log_prob = -5. Minimum log(P) for sequence to consider location
- skip_seq_prob = False Skip sequence probability calculation
- save_acceptable_loops = False Just return a file with possible loops
- n_random_loop = 200 Number of tries at building loops
- connect_all_segments = False Try to connect all segments to each other, regardless of sequence and residue numbers. Note: this is a last-resort approach. Normally just use the default and let fit_loops connect segments that are close in sequence.
- sequential_only = False Only connect adjacent segments in connect_all_segments
- dist_max = 15 Maximum CA-CA distance in connect_all_segments
- ignore_sequence_register = False Ignore the input sequence register
- all_assigned = True Assume all residues in model can be assigned to sequence If you set all_assigned=False then any residues with residue number greater than the number of residues in the

longest sequence in the sequence file are assumed not to be assigned to sequence. This is useful if you are using resolve model-building as resolve will identify non-sequence-aligned residues with high residue numbers.

- `sequence_offset = None` You can specify that `fit_loops` should offset input sequence file residue numbers by `sequence_offset` before use (same effect as adding `sequence_offset*X` to the beginning of the sequence file.) Note: number of `sequence_offset` values must match number of chains in sequence file
- `loop_cc_min = 0.2` Minimum loop map-model correlation
- `aggressive = False` Aggressive loop building (risky)
- `target_insert = None` You can try to force the number of residues to insert with `trace_loops`. If none, try to fill in the gap based on the number of missing residues. If set and greater than 0, only accept the number of residues given. If zero, take any length. Not supported.
- `time_per_residue = None` You can specify how long to try in fitting (sec/residue)
- `trim_ends_if_needed = True` If a single loop is specified and no loop is found, try trimming back the ends.
- `loop_backup_residues = None` Number of tries removing one residue at a time from each end of existing ends of loop if no loop is found with initial gap Default is 4 (1 if quick)
- `split_by_gap = True` Split up work by gaps and run individually (normally True)
- `loop_lib = False` Use loop library to fit loops Only applicable for `chain_type=PROTEIN`
- `standard_loops = True` Use standard loop fitting
- `trace_loops = False` Use loop tracing to fit loops Only applicable for `chain_type=PROTEIN`
- `refine_trace_loops = True` Refine loops (real-space) after `trace_loops`
- `density_of_points = None` Packing density of points to consider as as possible CA atoms in `trace_loops`. Try 1.0 for a quick run, up to 5 for much more thorough run If None, try value depending on value of quick.
- `a_cut_min = None` Minimum density (relative to SD of map, normalized for solvent content) for `trace_loop` points
- `max_density_of_points = None` Maximum packing density of points to consider as as possible CA atoms in `trace_loops`.
- `cutout_model_radius = None` Radius to cut out density for `trace_loops` If None, guess based on length of loop
- `max_cutout_model_radius = 20`. Maximum value of `cutout_model_radius` to try
- `padding = 1`. Padding for cut out density in `trace_loops`
- `cut_out_density = True` Cut out density for `trace_loops`
- `max_span = 30` Maximum length of a gap to try to fill
- `max_c_ca_dist = None` Maximum C-CA distance for a connection
- `max_overlap_rmsd = 2`. Maximum rmsd for 3 residues on each end of loop-lib fit
- `max_overlap = None` Maximum number of residues from ends to start with. (1=use existing ends, 2=one in from ends etc) If None, set based on value of quick.
- `min_overlap = None` Minimum number of residues from ends to start with. (1=use existing ends, 2=one in from ends etc)

- `crystal_inforesolution = 0` high-resolution limit for map calculation
- `chain_type = *PROTEIN DNA RNA` Chain type (for identifying main-chain and side-chain atoms)
- `final_solvent_content = None` Solvent content (after cutout if `trace_loops` used) . Normally determined automatically.
- `resolution = 0` high-resolution limit for map calculation
- `chain_type = *PROTEIN DNA RNA` Chain type (for identifying main-chain and side-chain atoms)
- `final_solvent_content = None` Solvent content (after cutout if `trace_loops` used) . Normally determined automatically.
- `directorytemp_dir = "temp_dir"` Temporary work directory
- `create_temp_dir_if_missing = True` Create `temp_dir` if not present
- `output_dir = None` Output directory where files are to be written
- `gui_output_dir = None` GUI use only - does not apply to command line version
- `temp_dir = "temp_dir"` Temporary work directory
- `create_temp_dir_if_missing = True` Create `temp_dir` if not present
- `output_dir = None` Output directory where files are to be written
- `gui_output_dir = None` GUI use only - does not apply to command line version
- `controlverbose = False` Verbose output
- `quick = False` Try to run quickly
- `raise_sorry = False` Raise sorry if problems
- `debug = False` Debugging output
- `dry_run = False` Just read in and check parameter names
- `coarse_grid = False` Use coarse grid (saves on memory)
- `i_ran_seed = None` random seed
- `resolve_command_list = None` Commands for resolve. One per line in the form: keyword value value can be optional Examples: `coarse_grid resolution 200 2.0 hkl in test.mtz` NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: `resolve_command_list="'no_build' 'b_overall 23' "`
- `write_run_directory_to_file = None` The working directory name is written to this file
- `pickled_arg_dict = None` Pickled keywords to `__init__` are in this file
- `nproc = 1` You can specify the number of processors to use
- `max_wait_time = 1.0` You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'
- `wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_time is File exists:....
- `background = None` run jobs in background or not (if `nproc` is greater than 1) Usually set automatically. If `run_command` is `sh` or `csh`, `Truerun_command = "sh "` Command for running jobs (e.g., `sh` or `qsub`)
- `verbose = False` Verbose output
- `quick = False` Try to run quickly
- `raise_sorry = False` Raise sorry if problems
- `debug = False` Debugging output
- `dry_run = False` Just read in and check parameter names
- `coarse_grid = False` Use coarse grid (saves on memory)
- `i_ran_seed = None` random seed
- `resolve_command_list = None` Commands for resolve. One per line in the form: keyword value value can be optional Examples: `coarse_grid resolution 200 2.0 hkl in test.mtz` NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: `resolve_command_list="'no_build' 'b_overall 23' "`
- `write_run_directory_to_file = None` The working directory name is written to this file

- `pickled_arg_dict = None` Pickled keywords to `__init__` are in this file
- `nproc = 1` You can specify the number of processors to use
- `max_wait_time = 1.0` You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'
- `wait_between_submit_time = 1.0` You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a `wait_between_submit_time` is File exists:....
- `background = None` run jobs in background or not (if `nproc` is greater than 1) Usually set automatically. If `run_command` is `sh` or `csch`, `True`
- `run_command = "sh "` Command for running jobs (e.g., `sh` or `qsub`)
- `non_user_paramsprint_citations = True` Print citation information at end of run
- `print_citations = True` Print citation information at end of run
- `guiGUI-specific parameters, not used on command line`
`result_file = None` Job title in PHENIX GUI, not used on command line
- `result_file = None`
- `job_title = None` Job title in PHENIX GUI, not used on command line
- `outputDuplicate of output scope, needed because fit_loops constructs its own parameters in __init__ with params=self.process_paramstarget_output_format = *None mmCIF pdb`
- `target_output_format = *None mmCIF pdb`

Fixing register errors in a model with `fix_insertions_deletions`

`fix_insertions_deletions.html`

Author(s)

- `fix_insertions_deletions`: Tom Terwilliger

Purpose

The routine `fix_insertions_deletions` is a tool for fixing register errors in a model by comparing the density at side-chain positions to a sequence file.

Usage

Normally you will access the functionality of `fix_insertions_deletions` by running the Phenix `map_to_model` tool in the Phenix GUI. However you can run it directly as well (there is no GUI for `fix_insertions_deletions`).

How `fix_insertions_deletions` works:

The `fix_insertions_deletions` tool examines the density in the supplied map at the position of each side chain in the supplied model and creates a table of side-chain probabilities corresponding to each segment in the model.

These side-chain probabilities are used to generate a map-based sequence for the model and map. This map-based sequence is then compared to the actual sequence to identify positions where the sequence register is likely to be incorrect, and what changes in register are needed to fix it.

Each place a register shift is needed is used as a target for main-chain rebuilding. During rebuilding the specified insertion or deletion is enforced so that only models with the desired changes are obtained (if possible).

Additional rebuilding of the worst-fitting regions is also carried out.

Using `fix_insertions_deletions`:

The tool `fix_insertions_deletions` is usually run automatically as part of `trace_and_build`. However you can run it yourself to try and fix up a model.

Input map file: The map file should cover the model you supply.

Resolution: Specify the resolution of your map (usually the resolution defined by your half-dataset Fourier shell correlation)

Model: Supply a model that you want to fix. Only the main-chain will matter.

Sequence: Supply a sequence file that covers at least the part of the model that is supplied

Examples

Standard run of `fix_insertions_deletions`:

You can use `fix_insertions_deletions` to fix register shifts in a model based on a cryo-EM map:

```
phenix.fix_insertions_deletions my_map.mrc resolution=3 my_model.pdb seq.dat
```

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
input_filesmap_file = None File with CCP4-style map. May have origin in any location.
model_file = None Input PDB file with chains to be adjusted.
seq_file = None Optional sequence file
placement_pickle_file = None Read placements from this file
output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
pdb_out = fix_insertions_deletions.pdb Output rebuilt model
output_placement_pickle_file = None Write placements to this file
temp_dir = None Temporary directory. Default is fix_insertions_deletions_xx where xx creates new directory
crystal_inforesolution = None High-resolution limit for map analysis.
scattering_table = n_gaussian wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.
chain_type = *PROTEIN Chain type. Must be PROTEIN
solvent_content = None Solvent fraction of the cell. If this is density cut out from a bigger cell, you can specify the fraction of the volume of this cell that is taken up by the macromolecule. Normally set automatically. Values go from 0 to 1.
solvent_content_iterations = 3 Iterations of solvent fraction estimation
use_mask_if_present = True If map is masked, use the mask as solvent content
sequence = None Sequences
origin_cart = None Origin (cartesian coordinates, overrides value based on map)
strategyfirst_cycle_for_fixing = 1 First macro-cycle where insertions/deletions should be tested
loop_method = *trace_chain *extend_only *split_loop *rebuild Method for loop building. None means try everything. trace_chain is finding CA positions to trace chain. extend-only is using resolve model-building to build loop. split_loop is cut loop in the middle and refine. rebuild is rebuild
loopsmask_secondary_structure_in_split_loop = True Mask out mask secondary structure in split
loopresidues_to_skip_on_ends = -2 Skip residues_to_skip_on_ends at ends of secondary structure in masking. Negative means keep that many residues off ends of secondary
structurebuildingfix_insertions_deletions = False Fix insertions and deletions using sequence_from_map to adjust alignment to match sequence that is supplied. NOTE: Use instead split_with_sequence which works better.
restrain_ends_to_original = None Restrain this many residues at each end to original
fix_insertions_deletions_only = None Only fix clear insertions and deletion
stake_all_insertions_deletions = None Accept insertion/deletion fixes without scoring (default quick is True)
ratio_for_sequence_register = 1.0 Accept fixes for insertions/deletions if score is above ratio_for_sequence_register times previous score
try_as_is = None Try rebuilding without insertions/deletion
try_insertions = None Try insertion
try_deletions = None Try deletions
refine = None Refine models at start of procedure. Default is True unless quick is set.
refine_cycles = 1 Refinement cycles (except final cycles)
refine_b = None Refine B-values
good_enough_cc = None If all residues have this CC, don't bother rebuilding them. Default is 0.7 (0.6 if quick=True)
rebuild_length_worst = 15 Longest rebuild length to try
max_insert_or_delete = 1 Maximum residues to insert or delete in on rebuild
stageaverage_rebuild_length = True Choose rebuild length as average of minimum and optimal if optimal is

longer than rebuild_length_worst. Alternative is use minimum if optimal is too long.time_per_residue = 1 How long to try in fitting loops (sec/residue)max_rebuild_cycles = None Maximum rebuilding cycles per macro cycle. Default is 20 (1 if quick is set and sequence is present)macro_cycles = None Macro cycles of rebuilding and refinement. Default is 4 (1 if quick is set)start_rebuild = None Starting residue to rebuild. If specified, this is all that is doneend_rebuild = None Ending residue to rebuild. If s...

- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesmap_file = None File with CCP4-style map. May have origin in any location.model_file = None Input PDB file with chains to be adjusted.seq_file = None Optional sequence fileplacement_pickle_file = None Read placements from this file
- map_file = None File with CCP4-style map. May have origin in any location.
- model_file = None Input PDB file with chains to be adjusted.
- seq_file = None Optional sequence file
- placement_pickle_file = None Read placements from this file
- output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.pdb_out = fix_insertions_deletions.pdb Output rebuilt modeloutput_placement_pickle_file = None Write placements to this filetemp_dir = None Temporary directory. Default is fix_insertions_deletions_xx where xx creates new directory
- target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- pdb_out = fix_insertions_deletions.pdb Output rebuilt model
- output_placement_pickle_file = None Write placements to this file
- temp_dir = None Temporary directory. Default is fix_insertions_deletions_xx where xx creates new directory
- crystal_inforesolution = None High-resolution limit for map analysis.scattering_table = n_gaussian wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.chain_type = *PROTEIN Chain type. Must be PROTEINsolvent_content = None Solvent fraction of the cell. If this is density cut out from a bigger cell, you can specify the fraction of the volume of this cell that is taken up by the macromolecule. Normally set automatically. Values go from 0 to 1.solvent_content_iterations = 3 Iterations of solvent fraction estimationuse_mask_if_present = True If map is masked, use the mask as solvent contentsequence = None Sequencesorigin_cart = None Origin (cartesian coordinates, overrides value based on map)
- resolution = None High-resolution limit for map analysis.
- scattering_table = n_gaussian wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.
- chain_type = *PROTEIN Chain type. Must be PROTEIN
- solvent_content = None Solvent fraction of the cell. If this is density cut out from a bigger cell, you can specify the fraction of the volume of this cell that is taken up by the macromolecule. Normally set automatically. Values go from 0 to 1.
- solvent_content_iterations = 3 Iterations of solvent fraction estimation
- use_mask_if_present = True If map is masked, use the mask as solvent content

- sequence = None Sequences
- origin_cart = None Origin (cartesian coordinates, overrides value based on map)
- strategyfirst_cycle_for_fixing = 1 First macro-cycle where insertions/deletions should be tested
loop_method = *trace_chain *extend_only *split_loop *rebuild Method for loop building. None means try everything. trace_chain is finding CA positions to trace chain. extend-only is using resolve model-building to build loop. split_loop is cut loop in the middle and refine. rebuild is rebuild loops
mask_secondary_structure_in_split_loop = True Mask out mask secondary structure in split
loopresidues_to_skip_on_ends = -2 Skip residues_to_skip_on_ends at ends of secondary structure in masking. Negative means keep that many residues off ends of secondary structure
- first_cycle_for_fixing = 1 First macro-cycle where insertions/deletions should be tested
- loop_method = *trace_chain *extend_only *split_loop *rebuild Method for loop building. None means try everything. trace_chain is finding CA positions to trace chain. extend-only is using resolve model-building to build loop. split_loop is cut loop in the middle and refine. rebuild is rebuild loops
- mask_secondary_structure_in_split_loop = True Mask out mask secondary structure in split loop
- residues_to_skip_on_ends = -2 Skip residues_to_skip_on_ends at ends of secondary structure in masking. Negative means keep that many residues off ends of secondary structure
- rebuildingfix_insertions_deletions = False Fix insertions and deletions using sequence_from_map to adjust alignment to match sequence that is supplied. NOTE: Use instead split_with_sequence which works better.
restrain_ends_to_original = None Restrain this many residues at each end to original
originalfix_insertions_deletions_only = None Only fix clear insertions and deletion
stake_all_insertions_deletions = None Accept insertion/deletion fixes without scoring (default quick is True)
ratio_for_sequence_register = 1.0 Accept fixes for insertions/deletions if score is above ratio_for_sequence_register times previous score
try_as_is = None Try rebuilding without insertions/deletion
try_insertions = None Try insertion
try_deletions = None Try deletions
refine = None Refine models at start of procedure. Default is True unless quick is set.
refine_cycles = 1 Refinement cycles (except final cycles)
refine_b = None Refine B-values
good_enough_cc = None If all residues have this CC, don't bother rebuilding them. Default is 0.7 (0.6 if quick=True)
rebuild_length_worst = 15 Longest rebuild length to try
max_insert_or_delete = 1 Maximum residues to insert or delete in on rebuild
stageaverage_rebuild_length = True Choose rebuild length as average of minimum and optimal if optimal is longer than rebuild_length_worst. Alternative is use minimum if optimal is too long.
time_per_residue = 1 How long to try in fitting loops (sec/residue)
max_rebuild_cycles = None Maximum rebuilding cycles per macro cycle. Default is 20 (1 if quick is set and sequence is present)
macro_cycles = None Macro cycles of rebuilding and refinement. Default is 4 (1 if quick is set)
start_rebuild = None Starting residue to rebuild. If specified, this is all that is done
end_rebuild = None Ending residue to rebuild. If specified, this is all that is done
rebuild_segment = 1 Segment to rebuild from start_rebuild to end_rebuild. None means rebuild all segments from start_rebuild to end_rebuild.
minimum_contact_distance = 3 Minimum distance between CA atoms not immediately connected
split_with_sequence = False Use sequence assignment to identify sequence register errors. Runs sequence_from_map to split and fix assignment and then fit_all_loops to fill in gaps.
keep_connectivity_in_split_with_sequence = True Keep connectivity in split_with_sequence
first_cycle_for_split = 2 First macro-cycle where splitting should be tested
split_input_model = True Input model will be split into segments
- fix_insertions_deletions = False Fix insertions and deletions using sequence_from_map to adjust alignment to match sequence that is supplied. NOTE: Use instead split_with_sequence which works better.
- restrain_ends_to_original = None Restrain this many residues at each end to original

- `fix_insertions_deletions_only` = None Only fix clear insertions and deletions
- `take_all_insertions_deletions` = None Accept insertion/deletion fixes without scoring (default quick is True)
- `ratio_for_sequence_register` = 1.0 Accept fixes for insertions/deletions if score is above `ratio_for_sequence_register` times previous score
- `try_as_is` = None Try rebuilding without insertions/deletions
- `try_insertions` = None Try insertions
- `try_deletions` = None Try deletions
- `refine` = None Refine models at start of procedure. Default is True unless quick is set.
- `refine_cycles` = 1 Refinement cycles (except final cycles)
- `refine_b` = None Refine B-values
- `good_enough_cc` = None If all residues have this CC, don't bother rebuilding them. Default is 0.7 (0.6 if quick=True)
- `rebuild_length_worst` = 15 Longest rebuild length to try
- `max_insert_or_delete` = 1 Maximum residues to insert or delete in on rebuild stage
- `average_rebuild_length` = True Choose rebuild length as average of minimum and optimal if optimal is longer than `rebuild_length_worst`. Alternative is use minimum if optimal is too long.
- `time_per_residue` = 1 How long to try in fitting loops (sec/residue)
- `max_rebuild_cycles` = None Maximum rebuilding cycles per macro cycle. Default is 20 (1 if quick is set and sequence is present)
- `macro_cycles` = None Macro cycles of rebuilding and refinement. Default is 4 (1 if quick is set)
- `start_rebuild` = None Starting residue to rebuild. If specified, this is all that is done
- `end_rebuild` = None Ending residue to rebuild. If specified, this is all that is done
- `rebuild_segment` = 1 Segment to rebuild from `start_rebuild` to `end_rebuild`. None means rebuild all segments from `start_rebuild` to `end_rebuild`.
- `minimum_contact_distance` = 3 Minimum distance between CA atoms not immediately connected
- `split_with_sequence` = False Use sequence assignment to identify sequence register errors. Runs `sequence_from_map` to split and fix assignment and then `fit_all_loops` to fill in gaps.
- `keep_connectivity_in_split_with_sequence` = True Keep connectivity in `split_with_sequence`
- `first_cycle_for_split` = 2 First macro-cycle where splitting should be tested
- `split_input_model` = True Input model will be split into segments
- `weight_vdw` = 10 weight on very close CA-CA contacts
- `weight_ca_ca_dist` = 1. weight on CA-CA distance
- `weight_proximity_to_known_position` = 0.5 weight on proximity to well-identified CA position
- `weight_density` = 1.0 weight on density at mid-points between CA atoms
- `weight_cc_mask_score` = 1.0 Weight on map correlation in evaluating chain direction
- `weight_seq_score` = 1.0 Weight on sequence-map match in evaluating chain direction
- `weight_x_gly_score` = 1.0 Weight on excess Gly and X residues in evaluating chain direction

- `weight_density = 1.0` weight on density at mid-points between CA atoms
- `weight_cc_mask_score = 1.0` Weight on map correlation in evaluating chain direction
- `weight_seq_score = 1.0` Weight on sequence-map matchin evaluating chain direction
- `weight_x_gly_score = 1.0` Weight on excess Gly and X residues in evaluating chain direction
- `sequencingrandom_sequences = 100` Number of random sequences of each length to use as baseline
- `positive_gap_penalty = 1` Penalty for missing residues is $\text{positive_gap_penalty} * \text{gap}^{**2}$
- `negative_gap_penalty = 2` Gap penalty for extra residues is $\text{negative_gap_penalty} * \text{gap}^{**2}$
- `max_gap_length = 2` Maximum gap length for adjacent alignments
- `minimum_alignment_length = 5` Minimum length of an alignment
- `score_by_residue_groups = None` Use residue groups in sequence alignment and listing of optimal sequences. Default is True unless no sequence is supplied.
- `random_sequences = 100` Number of random sequences of each length to use as baseline
- `positive_gap_penalty = 1` Penalty for missing residues is $\text{positive_gap_penalty} * \text{gap}^{**2}$
- `negative_gap_penalty = 2` Gap penalty for extra residues is $\text{negative_gap_penalty} * \text{gap}^{**2}$
- `max_gap_length = 2` Maximum gap length for adjacent alignments
- `minimum_alignment_length = 5` Minimum length of an alignment
- `score_by_residue_groups = None` Use residue groups in sequence alignment and listing of optimal sequences. Default is True unless no sequence is supplied.
- `trace_chainhelices_strands_cc_min = 0.5` Minimum map CC for helices/strands
- `helices_strands_cc_min = 0.5` Minimum map CC for helices/strands
- `n_random_frag = 100`
- `controlmultiprocessing = *` multiprocessing sge lsf pbs condor pbspro slurm Choices are multiprocessing (single machine) or queuing systems
- `queue_run_command = None` run command for queue jobs. For example qsub.
- `nproc = 1` Number of processors to use
- `random_seed = 171731` Random seed
- `verbose = False` Verbose output
- `skip_temp_dir = True` Skip temp_dir when scoring
- `quick = True` Quick run
- `superquick = False` Very quick run
- `ignore_symmetry_conflicts = False` You can ignore the symmetry information (CRYST1) from coordinate files. This may be necessary if your model has been placed in a box with box_map for

example.

- `max_dirs = 1000` Maximum number of directories (`fix_insertions_deletions_xxxx`)
- `resolve_size = None` Size of resolve to use.
- `coarse_grid = None` Use a coarse grid in RESOLVE (saves on memory)
- `em_side_density = False` Use EM side chain density. Alternative is to use standard x-ray side chain density in sequence templates.
- `gui` GUI-specific parameter required for output directory `output_dir = None`
- `output_dir = None`

<string>

fmodel.html

phenix.fmodel: a tool to compute structure factors, Fmodel:

$$F_{\text{model}} = \text{scale} * \exp(\text{AnisoScale}) * (F_{\text{calc}} + k_{\text{sol}} * \exp(-b_{\text{sol}} * s^2/4) * F_{\text{mask}})$$
 where: Fmodel - total model structure factor (complex value) AnisoScale = $-\text{ht} * A(-1) * b_{\text{cart}} * A(-1)_{\text{th}}/4$ h - column vector with Miller indices A - orthogonalization matrix b_cart - anisotropic scale matrix t and (-1) denotes transposition and inversion operations scale - overall scale factor Fcalc - structure factors calculated from atomic model k_sol and b_sol - Flat Bulk solvent model parameters Fmask - structure factors calculated from bulk solvent mask

$$F_{\text{model}} = \text{scale} * \exp(\text{AnisoScale}) * (F_{\text{calc}} + k_{\text{sol}} * \exp(-b_{\text{sol}} * s^2/4) * F_{\text{mask}})$$

where:

- Fmodel - total model structure factor (complex value)
- AnisoScale = $-\text{ht} * A(-1) * b_{\text{cart}} * A(-1)_{\text{th}}/4$
- h - column vector with Miller indices
- A - orthogonalization matrix
- b_cart - anisotropic scale matrix
- t and (-1) denotes transposition and inversion operations
- scale - overall scale factor
- Fcalc - structure factors calculated from atomic model
- k_sol and b_sol - Flat Bulk solvent model parameters
- Fmask - structure factors calculated from bulk solvent mask

Usage examples:

phenix.fmodel model.pdb high_resolution=1.5 will result in a file containing complete set of Fmodel = Fcalc computed from atomic model up to 1.5Å resolution. phenix.fmodel model.pdb scale=2 k_sol=0.35 b_sol=50 b_cart="1 2 3 0 4 7" high_res=1.5 low_res=10 will result in a file containing complete set of Fmodel computed using the above formula in resolution range 1.5-20.0Å. phenix.fmodel model.pdb high_resolution=1.5 algorithm=direct is similar to "1)" but the Fcalc are computed using direct summation algorithm. phenix.fmodel model.pdb high_res=1.5 format=cns label=FOBS type=real r_free=0.1 will result in CNS formatted file containing complete set of amplitudes of Fmodel = Fcalc computed up to 1.5Å resolution, labelled as FOBS, and free-R flags with 10% of test reflections. This is a typical command to simulate Fobs. phenix.fmodel model.pdb high_res=1.5 scattering_table=neutron will result in a file containing complete set of Fmodel = Fcalc computed from atomic model up to 1.5Å resolution using neutron scattering table. phenix.fmodel model.pdb parameters.txt will result in a structure factor file, where Fmodel were computed using parameters defined in parameters.txt file. The parameters.txt file can contain all or any subset of parameters listed below. Note, that each { must have a matching one }. phenix.fmodel model.pdb reflection_data.mtz will result in a file containing a set of Fmodel = Fcalc that will match the set of Miller indices of the data in reflection_data.mtz file. phenix.fmodel model.pdb reflection_data.mtz data_column_label="FOBS,SIGMA" similar to "7)", where the specific data array is selected. phenix.fmodel model.pdb reflection_data.mtz twin_law="l,-k,h" twin_fraction=0.3 generates twin data set (real type) with given twin law and fraction.

- phenix.fmodel model.pdb high_resolution=1.5 will result in a file containing complete set of Fmodel = Fcalc computed from atomic model up to 1.5Å resolution.

phenix.fmodel model.pdb high_resolution=1.5

will result in a file containing complete set of Fmodel = Fcalc computed from atomic model up to 1.5A resolution.

- phenix.fmodel model.pdb scale=2 k_sol=0.35 b_sol=50 b_cart="1 2 3 0 4 7" high_res=1.5 low_res=10 will result in a file containing complete set of Fmodel computed using the above formula in resolution range 1.5-20.0A.

phenix.fmodel model.pdb scale=2 k_sol=0.35 b_sol=50 b_cart="1 2 3 0 4 7" high_res=1.5 low_res=10

will result in a file containing complete set of Fmodel computed using the above formula in resolution range 1.5-20.0A.

- phenix.fmodel model.pdb high_resolution=1.5 algorithm=direct is similar to "1)" but the Fcalc are computed using direct summation algorithm.

phenix.fmodel model.pdb high_resolution=1.5 algorithm=direct

is similar to "1)" but the Fcalc are computed using direct summation algorithm.

- phenix.fmodel model.pdb high_res=1.5 format=cns label=FOBS type=real r_free=0.1 will result in CNS formatted file containing complete set of amplitudes of Fmodel = Fcalc computed up to 1.5A resolution, labelled as FOBS, and free-R flags with 10% of test reflections. This is a typical command to simulate Fobs.

phenix.fmodel model.pdb high_res=1.5 format=cns label=FOBS type=real r_free=0.1

will result in CNS formatted file containing complete set of amplitudes of Fmodel = Fcalc computed up to 1.5A resolution, labelled as FOBS, and free-R flags with 10% of test reflections. This is a typical command to simulate Fobs.

- phenix.fmodel model.pdb high_res=1.5 scattering_table=neutron will result in a file containing complete set of Fmodel = Fcalc computed from atomic model up to 1.5A resolution using neutron scattering table.

phenix.fmodel model.pdb high_res=1.5 scattering_table=neutron

will result in a file containing complete set of Fmodel = Fcalc computed from atomic model up to 1.5A resolution using neutron scattering table.

- phenix.fmodel model.pdb parameters.txt will result in a structure factor file, where Fmodel were computed using parameters defined in parameters.txt file. The parameters.txt file can contain all or any subset of parameters listed below. Note, that each { must have a matching one }.

phenix.fmodel model.pdb parameters.txt

will result in a structure factor file, where Fmodel were computed using parameters defined in parameters.txt file. The parameters.txt file can contain all or any subset of parameters listed below. Note, that each { must have a matching one }.

- phenix.fmodel model.pdb reflection_data.mtz will result in a file containing a set of Fmodel = Fcalc that will match the set of Miller indices of the data in reflection_data.mtz file.

phenix.fmodel model.pdb reflection_data.mtz

will result in a file containing a set of Fmodel = Fcalc that will match the set of Miller indices of the data in reflection_data.mtz file.

- phenix.fmodel model.pdb reflection_data.mtz data_column_label="FOBS,SIGMA" similar to "7)", where the specific data array is selected.

```
phenix.fmodel model.pdb reflection_data.mtz data_column_label="FOBS,SIGMA"
```

similar to "7)", where the specific data array is selected.

- `phenix.fmodel model.pdb reflection_data.mtz twin_law="l,-k,h" twin_fraction=0.3` generates twin data set (real type) with given twin law and fraction.

```
phenix.fmodel model.pdb reflection_data.mtz twin_law="l,-k,h" twin_fraction=0.3
```

generates twin data set (real type) with given twin law and fraction.

See below for complete list of available parameters.

Find matching fragments in the PDB (fragment_search)

fragment_search.html

Author(s)

- fragment_search: Tom Terwilliger

Purpose

Extract a segment from the PDB that is similar to a segment in a supplied model.

This tool can be used in early stages of model-building to try and improve a segment in a preliminary model by replacing it with a similar fragment from a deposited structure.

Note that the fragment from the PDB may not actually have good geometry because it is morphed (distorted) to match the supplied model.

Fragment search differs from `replace_with_fragments_from_pdb` in that `fragment_search` is looking for a single fragment from the PDB that matches a target segment in your model, while `replace_with_fragments_from_pdb` is trying to completely replace your model with a set of fragments.

Usage

`fragment_search` can find a fragment in the PDB similar to a supplied fragment (protein chains only)

You can adjust the maximum number of insertions/deletions in the replacement fragment and whether to morph the fragment.

You can also choose to exclude or include specific PDB entries or exclude all PDB entries with sequences similar to a particular PDB entry.

You can choose to create a replacement for just a selected part of your model (by supplying a selection string that specifies what to select).

How fragment_search works:

Fragment searching is done using a quality-filtered set of chains from the PDB, filtered at 90% sequence identity (`pdb2018_90`). The curated subset of the PDB consist of non-redundant (<90% identity), high-resolution structures (< 2 Å) that have very good geometry. The main-chain coordinates of these structures are supplied in a database as part of Phenix.

Potential matching fragments are found by comparing end-to-end distances for the target with all fragments of the desired length in structures in the database. Fragments with the desired end-to-end distance are then examined for matching of distances from end to several internal positions in the target fragment. Finally those that match all these are superimposed on the target fragment, morphed to match the target fragment, and the rmsd of CA positions are used to evaluate the match.

Morphing includes two steps. In the first, the fragment is distorted with an offset that changes linearly for each residue along the chain and that allows the first and last CA positions of the matching fragment to superimpose on those of the target fragment.

In the second step, a distortion field is applied to the matching fragment to make it more similar to the target fragment. This morphing consists of calculation of a real-space distortion field that changes over a typical

distance of about 10 Å (set by the parameter `distortion_field_length`), and applying this distortion field to all the atoms in the matching fragment.

Examples

Standard run of `fragment_search`:

Running `fragment_search` is easy. From the command-line you can type:

```
phenix.fragment_search model.pdb nproc=8 selection="chain A and resseq 1:20"
```

This will find fragments in the PDB similar to residues 1-20 of `model.pdb`.

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

`job_title` = None Job title in PHENIX GUI, not used on command line
`input_filesmodel` = None Input PDB file with segments to try and match
`selection` = None If specified, match selected part of model
`output_files`superposed_model_prefix = None Output files with superposed models will begin with this prefix
`fragment_searchtrim` = True Trim models down to match the target fragment. Otherwise return full
`chainn_res` = None Number of residues to get. If None, use number of residues in input fragment
`max_indel` = 0 Maximum insertions/deletions relative to `n_resn_half_window` = 5 Number of residues over which to average morphing shifts
`max_gap` = 20 Maximum gap in match to keep. Interpolate morphing values over gaps up to `max_gap`. Trim gaps longer than `max_gap`
`max_segment_matching_ca_ca_deviation` = 0.8 Maximum rmsd between segments to allow a match
`max_models` = 1 Number of models to return
`morph` = True Morph output models after superposition and before trimming
`pdb_include_list` = None List of PDB ID or pdb ID and chain_id to include (2DY1 or 2DY1_A)
`pdb_exclude_list` = None List of PDB ID or pdb ID and chain_id to exclude (2DY1 or 2DY1_A)
`exclude_similar_to_this_pdb_id` = None Exclude all PDB chains with sequence identity > `maximum_percent_identity_to_this_pdb_id` to
`exclude_similar_to_this_pdb_idmaximum_percent_identity_to_this_pdb_id` = 30 Maximum percent identity to supplied PDB chain to include. Exclude all PDB chains with sequence identity > `maximum_percent_identity_to_this_pdb_id` to
`exclude_similar_to_this_pdb_idcontrolnproc` = 1 Number of processors to use. If None, uses all available
`write_files` = True Write output files
`gui`GUI-specific parameter required for output directory
`output_dir` = None

- `job_title` = None Job title in PHENIX GUI, not used on command line
- `input_filesmodel` = None Input PDB file with segments to try and match
`selection` = None If specified, match selected part of model
- `model` = None Input PDB file with segments to try and match
- `selection` = None If specified, match selected part of model

- `output_filesuperposed_model_prefix = None` Output files with superposed models will begin with this prefix
- `superposed_model_prefix = None` Output files with superposed models will begin with this prefix
- `fragment_searchtrim = True` Trim models down to match the target fragment. Otherwise return full chain
- `n_res = None` Number of residues to get. If None, use number of residues in input fragment
- `max_indel = 0` Maximum insertions/deletions relative to `n_res`
- `n_half_window = 5` Number of residues over which to average morphing shifts
- `max_gap = 20` Maximum gap in match to keep. Interpolate morphing values over gaps up to `max_gap`. Trim gaps longer than `max_gap`
- `max_segment_matching_ca_ca_deviation = 0.8` Maximum rmsd between segments to allow a match
- `max_models = 1` Number of models to return
- `morph = True` Morph output models after superposition and before trimming
- `pdb_include_list = None` List of PDB ID or pdb ID and chain_id to include (2DY1 or 2DY1_A)
- `pdb_exclude_list = None` List of PDB ID or pdb ID and chain_id to exclude (2DY1 or 2DY1_A)
- `exclude_similar_to_this_pdb_id = None` Exclude all PDB chains with sequence identity > `maximum_percent_identity_to_this_pdb_id` to `exclude_similar_to_this_pdb_id`
- `maximum_percent_identity_to_this_pdb_id = 30` Maximum percent identity to supplied PDB chain to include. Exclude all PDB chains with sequence identity > `maximum_percent_identity_to_this_pdb_id` to `exclude_similar_to_this_pdb_id`
- `controlnproc = 1` Number of processors to use. If None, uses all available
- `write_files = True` Write output files
- `nproc = 1` Number of processors to use. If None, uses all available
- `write_files = True` Write output files
- `gui` GUI-specific parameter required for output directory
- `output_dir = None`

Correlation of two maps (deprecated, use MapCorrelations)

`get_cc_mtz_mtz.html`

Author(s)

- `get_cc_mtz_mtz`: Tom Terwilliger

Purpose

`get_cc_mtz_mtz` is a command line tool for adjusting the origin of a map so that the map superimposes on another map, and obtaining the correlation of the two maps. The maps are calculated from map coefficients supplied by the user in two mtz files.

Usage

How `get_cc_mtz_mtz` works:

`get_cc_mtz_mtz` calculates maps based on the supplied mtz files, then uses RESOLVE to find the origin shift compatible with space-group symmetry that maximizes the correlation of the two maps. This shift is applied to the second map and the correlation of the maps is calculated.

Several parameters can be set by the user to define how the correlations are calculated. By default, maps are calculated using all the reflections present (to the specified high-resolution limit, if any) in each mtz file.

If you specify

```
use_only_refl_present_in_mtz_1=True
```

Then the map calculated using your second mtz file will only include reflections that were present in your first mtz file. This removes the effects of missing reflections on the correlation.

If you specify

```
scale=True
```

then `get_cc_mtz_mtz` scales the amplitudes from the second input mtz file to those in the first input mtz, including an overall B factor and a scale factor. This reduces effects of differences in overall B factors between the two mtz files on the correlation.

If you specify

```
keep_f_mag=False
```

then `get_cc_mtz_mtz` uses amplitudes from the first input mtz file and phases and figure of merit from both to do the correlation. This has the effect of removing effects due to differences in amplitudes on the correlation, and focusing on differences in phases and figures of merit.

Output files from `get_cc_mtz_mtz`

`offset.log`: Log file for correlation calculation.

Examples

Standard run of get_cc_mtz_mtz:

Running the get_cc_mtz_mtz is easy. From the command-line you can type:

```
phenix.get_cc_mtz_mtz map_coeffs_1.mtz map_coeffs_2.mtz
```

If you want (or need) to specify the column names from your mtz file, you will need to tell get_cc_mtz_mtz what FP and PHIB (and optionally FOM) are, in this format:

```
phenix.get_cc_mtz_mtz map_coeffs_1.mtz map_coeffs_2.mtz \  
labin_1="FP=2FOFCWT PHIB=PH2FOFCWT" labin_2="FP=2FOFCWT PHIB=PH2FOFCWT"
```

Possible Problems

Specific limitations and problems:

- Versions of phenix.get_cc_mtz_mtz up to 1.3-final used a different set of defaults, with the values,

```
scale=True  
keep_f_mag=False  
use_only_refl_present_in_mtz_1=True
```

These defaults were changed after version 1.3-final in order to make the results independent of the order of the mtz files and to make the default be to get the correlation of maps without manipulation.

List of all available keywords

get_cc_mtz_mtzmtz_1 = None MTZ file 1 with coefficients for a map mtz_2 = None MTZ file 2 with coefficients for a map labin_1 = "" Labin line for MTZ file 1 with map coefficients. This is optional if get_cc_mtz_mtz can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP labin_2 = "" Labin line for MTZ file 2 with map coefficients. This is optional if get_cc_mtz_mtz can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP map_in_1 = None CCP4 or MRC-style map file for file 1 (instead of mtz_1)map_in_2 = None CCP4 or MRC-style map file for file 2 (instead of mtz_2)resolution = None high-resolution limit for map calculationlow_resolution = None low-resolution limit for map calculationoffset_suffix = offset Suffix for output version of mtz file 2 or map_in_2, offset to maximize correlation with mtz file 1temp_dir = "temp_dir" Temporary work directoryoutput_dir = "" Output directory where files are to be writtengui_output_dir = None keep_f_mag = True If you set keep_f_mag=False then get_cc_mtz_mtz uses amplitudes from the first input mtz file and phases and fom from both to do the correlation. If you specify keep_f_mag=True then the amplitudes from both files are included.scale = False If you set scale=True then get_cc_mtz_mtz scales the amplitudes from the second input mtz file to those in the first input mtz, including an overall B factor and a scale factor. use_only_refl_present_in_mtz_1 = False You can specify that only reflections present in your first mtz file are used in the comparison. Note that this means that the order of the files will have an effect on the correlation coefficientany_offset = False You can search for a match with any offset even though this is not allowed by space-group symmetryfix_xyz = False You can get map-map correlation with no translations Does not apply to mtz-mtz comparisonsverbose = True Verbose outputraise_sorry = False Raise sorry if problemsdebug = False Debugging outputdry_run = False Just read in and check parameter namesresolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' "; job_title = None Job title in PHENIX GUI, not used on command line

• `get_cc_mtz_mtz` `mtz_1` = None MTZ file 1 with coefficients for a map `mtz_2` = None MTZ file 2 with coefficients for a map `labin_1` = "" Labin line for MTZ file 1 with map coefficients. This is optional if `get_cc_mtz_mtz` can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=`myFP` PHIB=`myPHI` FOM=`myFOM` where `myFP` is your column label for FP `labin_2` = "" Labin line for MTZ file 2 with map coefficients. This is optional if `get_cc_mtz_mtz` can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=`myFP` PHIB=`myPHI` FOM=`myFOM` where `myFP` is your column label for FP `map_in_1` = None CCP4 or MRC-style map file for file 1 (instead of `mtz_1`) `map_in_2` = None CCP4 or MRC-style map file for file 2 (instead of `mtz_2`) `resolution` = None high-resolution limit for map calculation `low_resolution` = None low-resolution limit for map calculation `offset_suffix` = offset Suffix for output version of `mtz` file 2 or `map_in_2`, offset to maximize correlation with `mtz` file 1 `temp_dir` = "temp_dir" Temporary work directory `output_dir` = "" Output directory where files are to be written `gui_output_dir` = None `keep_f_mag` = True If you set `keep_f_mag`=False then `get_cc_mtz_mtz` uses amplitudes from the first input `mtz` file and phases and fom from both to do the correlation. If you specify `keep_f_mag`=True then the amplitudes from both files are included. `scale` = False If you set `scale`=True then `get_cc_mtz_mtz` scales the amplitudes from the second input `mtz` file to those in the first input `mtz`, including an overall B factor and a scale factor. `use_only_refl_present_in_mtz_1` = False You can specify that only reflections present in your first `mtz` file are used in the comparison. Note that this means that the order of the files will have an effect on the correlation coefficient `any_offset` = False You can search for a match with any offset even though this is not allowed by space-group symmetry `fix_xyz` = False You can get map-map correlation with no translations Does not apply to `mtz-mtz` comparisons `verbose` = True Verbose output `raise_sorry` = False Raise sorry if problems `debug` = False Debugging output `dry_run` = False Just read in and check parameter names `resolve_command_list` = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: `resolve_command_list`="'no_build' 'b_overall 23' "; `job_title` = None Job title in PHENIX GUI, not used on command line

- `mtz_1` = None MTZ file 1 with coefficients for a map
- `mtz_2` = None MTZ file 2 with coefficients for a map
- `labin_1` = "" Labin line for MTZ file 1 with map coefficients. This is optional if `get_cc_mtz_mtz` can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=`myFP` PHIB=`myPHI` FOM=`myFOM` where `myFP` is your column label for FP
- `labin_2` = "" Labin line for MTZ file 2 with map coefficients. This is optional if `get_cc_mtz_mtz` can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=`myFP` PHIB=`myPHI` FOM=`myFOM` where `myFP` is your column label for FP
- `map_in_1` = None CCP4 or MRC-style map file for file 1 (instead of `mtz_1`)
- `map_in_2` = None CCP4 or MRC-style map file for file 2 (instead of `mtz_2`)
- `resolution` = None high-resolution limit for map calculation
- `low_resolution` = None low-resolution limit for map calculation
- `offset_suffix` = offset Suffix for output version of `mtz` file 2 or `map_in_2`, offset to maximize correlation with `mtz` file 1
- `temp_dir` = "temp_dir" Temporary work directory
- `output_dir` = "" Output directory where files are to be written
- `gui_output_dir` = None

- `keep_f_mag = True` If you set `keep_f_mag=False` then `get_cc_mtz_mtz` uses amplitudes from the first input mtz file and phases and fom from both to do the correlation. If you specify `keep_f_mag=True` then the amplitudes from both files are included.
- `scale = False` If you set `scale=True` then `get_cc_mtz_mtz` scales the amplitudes from the second input mtz file to those in the first input mtz, including an overall B factor and a scale factor.
- `use_only_refl_present_in_mtz_1 = False` You can specify that only reflections present in your first mtz file are used in the comparison. Note that this means that the order of the files will have an effect on the correlation coefficient
- `any_offset = False` You can search for a match with any offset even though this is not allowed by space-group symmetry
- `fix_xyz = False` You can get map-map correlation with no translations Does not apply to mtz-mtz comparisons
- `verbose = True` Verbose output
- `raise_sorry = False` Raise sorry if problems
- `debug = False` Debugging output
- `dry_run = False` Just read in and check parameter names
- `resolve_command_list = None` You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: `resolve_command_list="'no_build' 'b_overall 23' "`
- `job_title = None` Job title in PHENIX GUI, not used on command line

Correlation of map and model (deprecated, use MapCorrelations)

[get_cc_mtz_pdb.html](#)

Author(s)

- `get_cc_mtz_pdb`: Tom Terwilliger

Purpose

`get_cc_mtz_pdb` is a command line tool for adjusting the origin of a PDB file using space-group symmetry so that the PDB file superimposes on a map, obtaining the correlation of model and map, and analyzing the correlation for each residue.

Usage

How `get_cc_mtz_pdb` works:

`get_cc_mtz_pdb` calculates a model map based on the supplied PDB file, then uses RESOLVE to find the origin shift (using space-group symmetry) that maximizes the correlation of this model map with a map calculated with the supplied map coefficients in an mtz file. This shift is applied to the atoms in the PDB file to create `offset.pdb` and then the correlation, residue-by-residue of `offset.pdb` with the map is analyzed. Atoms and residues that are out of density or are in weak density are flagged.

You can set several parameters to define how the correlations are calculated.

By default model density is calculated using the atom types, occupancies and isotropic thermal factors (B-values) supplied in the PDB file. If you specify

```
scale=True
```

then an overall B as well as an increment in B-values for each atom beyond CB (for proteins) will be added to the values in the PDB file, after adjusting these parameters to maximize the map correlation.

If you specify

```
use_only_refl_present_in_mtz=True
```

then the model-based map will be calculated using the same set of reflections as the map calculated from your input mtz file. This reduces the contribution of missing reflections on the calculation (but the correlation is no longer the actual map-model correlation).

In the calculation of the map correlation in the region of the model, the region where the model is located is defined as all points within a distance `rad_max` of an atom in the model. The value of `rad_max` is adjusted in each case to maximize this correlation. Its value is typically similar to the high-resolution limit of the map.

Output files from `get_cc_mtz_pdb`

`offset.pdb`: A PDB file offset to match the origin in the mtz file.

Examples

Standard run of get_cc_mtz_pdb:

Running the get_cc_mtz_pdb is easy. From the command-line you can type:

```
phenix.get_cc_mtz_pdb map_coeffs.mtz coords.pdb
```

If you want (or need) to specify the column names from your mtz file, you will need to tell get_cc_mtz_pdb what FP and PHIB (and optionally FOM) are, in this format:

```
phenix.get_cc_mtz_pdb map_coeffs.mtz coords.pdb \
  labin="FP=2FOFCWT PHIB=PH2FOFCWT"
```

Possible Problems

Specific limitations and problems:

The option to use atom selections in get_cc_mtz_pdb can cause some confusion, because the CC values obtained can depend on the atom selections. The way this works is that the PDB file (after atom selections) is used to calculate model density. This model density is compared to the map from your input map coefficients in the region surrounding the atoms selected. This model density can depend on the presence of nearby atoms (where the density extends a ways away from those atoms).

In versions of PHENIX up to 1.3-final, defaults were set to maximize the correlation coefficient rather than to give the correlation using the existing thermal parameters and including only the reflections present in the mtz file. These previous defaults were equivalent to using the values:

```
scale=True
use_only_refl_present_in_mtz=True
```

These defaults were changed so that the correlation values obtained by default in a case where no origin shifts are needed would correspond to those obtained by simply calculating (1) a map using the input map coefficients and (2) a map from the PBB file and then determining the correlation between these maps.

List of all available keywords

get_cc_mtz_pdbpdb_in = None PDB file with coordinates to evaluateatom_selection = None Any selection specified with atom_selection is applied to input model (pdb_in) before using the model. NOTE: this option can result in confusing output because the model density near an atom and therefore the local CC value depends not just on the atoms being considered but also nearby atoms. Therefore if you remove some atoms next to a residue with atom_selection then the CC for that residue may change.mtz_in = None MTZ file with coefficients for a map map_in = None CCP4 or MRC-style map filelabin = "" Labin line for MTZ file with map coefficients. This is optional if get_cc_mtz_pdb can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP offset_pdb_suffix = offset Suffix for output version of pdb file, (offset to maximize correlation with mtz file if fix_xyz is not set)resolution = None high-resolution limit for map calculationuse_only_refl_present_in_mtz = False You can specify that only reflections present in your mtz file are used in the comparison. scale = False If you set scale=True then get_cc_mtz_pdb applies an overall B factor and a delta_b for each atom beyond CB. split_conformers = False If you want to have A and B conformers analyzed separately you can say split_conformers=True. Note that this requires that the entire A conformer for a residue must be before (or after) the entire B conformer for that residue.use_cc_mask = None Use cc_mask calculation to match validation methods Default is True for map_in and False for mtz_infix_xyz = False If you want your PDB file compared to the map from your mtz file with no offsets at all (fixed position) then specify fix_xyz=Truefix_rad_max = False If you want to use a fixed radius around all atoms for calculation of the

correlation use fix_rad=True. To set the value, use rad_max=xxx, otherwise it is set automatically. If rad_max is set, fix_rad_max is set to True rad_max = None If you want to use a fixed radius around all atoms for calculation of the correlation use fix_rad=True. To set the value, use rad_max=2.5, otherwise it is set automatically any_offset = False You can search for a match with any offset even though this is not allowed by space-group symmetry chain_type = *PROTEIN DNA RNA Chain type (for identifying main-chain and side-chain atoms) temp_dir = "temp_dir" Temporary work directory output_dir = "" Output directory where files are to be written gui_output_dir = None verbose = True Verbose output quick = False Skip the residue-by-residue correlations for a quick run raise_sorry = False Raise sorry if problems debug = False Debugging output dry_run = False Just read in and check parameter names resolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' "; job_title = None Job title in PHENIX GUI, not used on command line output_file target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.

- get_cc_mtz_pdb pdb_in = None PDB file with coordinates to evaluate atom_selection = None Any selection specified with atom_selection is applied to input model (pdb_in) before using the model. NOTE: this option can result in confusing output because the model density near an atom and therefore the local CC value depends not just on the atoms being considered but also nearby atoms. Therefore if you remove some atoms next to a residue with atom_selection then the CC for that residue may change. mtz_in = None MTZ file with coefficients for a map map_in = None CCP4 or MRC-style map file labin = "" Labin line for MTZ file with map coefficients. This is optional if get_cc_mtz_pdb can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP offset_pdb_suffix = offset Suffix for output version of pdb file, (offset to maximize correlation with mtz file if fix_xyz is not set) resolution = None high-resolution limit for map calculation use_only_refl_present_in_mtz = False You can specify that only reflections present in your mtz file are used in the comparison. scale = False If you set scale=True then get_cc_mtz_pdb applies an overall B factor and a delta_b for each atom beyond CB. split_conformers = False If you want to have A and B conformers analyzed separately you can say split_conformers=True. Note that this requires that the entire A conformer for a residue must be before (or after) the entire B conformer for that residue. use_cc_mask = None Use cc_mask calculation to match validation methods Default is True for map_in and False for mtz_in fix_xyz = False If you want your PDB file compared to the map from your mtz file with no offsets at all (fixed position) then specify fix_xyz=True fix_rad_max = False If you want to use a fixed radius around all atoms for calculation of the correlation use fix_rad=True. To set the value, use rad_max=xxx, otherwise it is set automatically. If rad_max is set, fix_rad_max is set to True rad_max = None If you want to use a fixed radius around all atoms for calculation of the correlation use fix_rad=True. To set the value, use rad_max=2.5, otherwise it is set automatically any_offset = False You can search for a match with any offset even though this is not allowed by space-group symmetry chain_type = *PROTEIN DNA RNA Chain type (for identifying main-chain and side-chain atoms) temp_dir = "temp_dir" Temporary work directory output_dir = "" Output directory where files are to be written gui_output_dir = None verbose = True Verbose output quick = False Skip the residue-by-residue correlations for a quick run raise_sorry = False Raise sorry if problems debug = False Debugging output dry_run = False Just read in and check parameter names resolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' "; job_title = None Job title in PHENIX GUI, not used on command line output_file target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does

not fit in pdb format, mmCIF will be used. Default is pdb.

- `pdb_in` = None PDB file with coordinates to evaluate
- `atom_selection` = None Any selection specified with `atom_selection` is applied to input model (`pdb_in`) before using the model. NOTE: this option can result in confusing output because the model density near an atom and therefore the local CC value depends not just on the atoms being considered but also nearby atoms. Therefore if you remove some atoms next to a residue with `atom_selection` then the CC for that residue may change.
- `mtz_in` = None MTZ file with coefficients for a map
- `map_in` = None CCP4 or MRC-style map file
- `labin` = "" Labin line for MTZ file with map coefficients. This is optional if `get_cc_mtz_pdb` can guess the correct coefficients for FP PHI and FOM. Otherwise specify: `LABIN FP=myFP PHIB=myPHI FOM=myFOM` where `myFP` is your column label for FP
- `offset_pdb_suffix` = offset Suffix for output version of pdb file, (offset to maximize correlation with mtz file if `fix_xyz` is not set)
- `resolution` = None high-resolution limit for map calculation
- `use_only_refl_present_in_mtz` = False You can specify that only reflections present in your mtz file are used in the comparison.
- `scale` = False If you set `scale=True` then `get_cc_mtz_pdb` applies an overall B factor and a `delta_b` for each atom beyond CB.
- `split_conformers` = False If you want to have A and B conformers analyzed separately you can say `split_conformers=True`. Note that this requires that the entire A conformer for a residue must be before (or after) the entire B conformer for that residue.
- `use_cc_mask` = None Use `cc_mask` calculation to match validation methods Default is True for `map_in` and False for `mtz_in`
- `fix_xyz` = False If you want your PDB file compared to the map from your mtz file with no offsets at all (fixed position) then specify `fix_xyz=True`
- `fix_rad_max` = False If you want to use a fixed radius around all atoms for calculation of the correlation use `fix_rad=True`. To set the value, use `rad_max=xxx`, otherwise it is set automatically. If `rad_max` is set, `fix_rad_max` is set to True
- `rad_max` = None If you want to use a fixed radius around all atoms for calculation of the correlation use `fix_rad=True`. To set the value, use `rad_max=2.5`, otherwise it is set automatically
- `any_offset` = False You can search for a match with any offset even though this is not allowed by space-group symmetry
- `chain_type` = *PROTEIN DNA RNA Chain type (for identifying main-chain and side-chain atoms)
- `temp_dir` = "temp_dir" Temporary work directory
- `output_dir` = "" Output directory where files are to be written
- `gui_output_dir` = None
- `verbose` = True Verbose output
- `quick` = False Skip the residue-by-residue correlations for a quick run
- `raise_sorry` = False Raise sorry if problems
- `debug` = False Debugging output

- `dry_run = False` Just read in and check parameter names
- `resolve_command_list = None` You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: `resolve_command_list="'no_build' 'b_overall 23' "`
- `job_title = None` Job title in PHENIX GUI, not used on command line
- `output_filetarget_output_format = *None` pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- `target_output_format = *None` pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.

Guess location of molecular centers in a map with `guess_molecular_centers`

`guess_molecular_centers.html`

Author(s)

- `guess_molecular_centers`: Tom Terwilliger

Purpose

`guess_molecular_centers` is a tool to identify potential locations of molecules in an electron density map.

Usage

How `guess_molecular_centers` works:

`guess_molecular_centers` identifies centers of macromolecules in the density map by finding local maxima of the local RMS density. You can specify the radius of the sphere used to estimate local RMS density.

Output files from `guess_molecular_centers`

`guess_molecular_centers.pdb`: A PDB file with pseudo-atoms marking the locations of centers of your molecule(s)

Examples

Standard run of `guess_molecular_centers`:

Running `guess_molecular_centers` is easy. From the command-line you can type:

```
phenix.guess_molecular_centers map_coeffs.mtz
```

Possible Problems

If your molecule is large, you may wish to increase the radius used for smoothing (`smoothing_radius`). If you cannot find the centers you want, you can try looking for more by increasing `n_centers_find`.

Literature

Finding non-crystallographic symmetry in density maps of macromolecular structures. T.C. Terwilliger. J Struct Funct Genomics 14, 91-5 (2013).

- Finding non-crystallographic symmetry in density maps of macromolecular structures. T.C. Terwilliger. J Struct Funct Genomics 14, 91-5 (2013).

List of all available keywords

`guess_molecular_centersinput_filesmtz_in` = None MTZ file with coefficients for a map `labin` = "" Labin line for MTZ file with map coefficients. This is optional if `guess_molecular_centers` can guess the correct coefficients

for FP PHI and FOM. Otherwise specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.pdb_out = guess_molecular_centers.pdb Output PDB filelog = guess_molecular_centers.log Output log fileparams_out = guess_molecular_centers_params.eff Parameters file to rerun guess_molecular_centersdirectoriestemp_dir = "temp_dir" Optional temporary work directory output_dir = "" Output directory where files are to be writtenfind_centersssmoothing_radius = 10. Radius for smoothing squared density to find centers Choose a smaller value to get more center guessesn_centers = 1 Report up to n_centers centersn_center_find = None Target number of centers to find Smoothing radius will be varied from 2 to 2*smoothing_radius and the value giving the number of peaks closest to n_center_find will be usedcrystal_inforesolution = 0. high-resolution limit for map calculationsolvent_fraction = 0.5 solvent fractioncontrolverbose = True Verbose outputraise_sorry = False Raise sorry if problemsdebug = False Debugging outputdry_run = False Just read in and check parameter namesresolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' ";

- guess_molecular_centersinput_filesmtz_in = None MTZ file with coefficients for a map labin = "" Labin line for MTZ file with map coefficients. This is optional if guess_molecular_centers can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.pdb_out = guess_molecular_centers.pdb Output PDB filelog = guess_molecular_centers.log Output log fileparams_out = guess_molecular_centers_params.eff Parameters file to rerun guess_molecular_centersdirectoriestemp_dir = "temp_dir" Optional temporary work directory output_dir = "" Output directory where files are to be writtenfind_centersssmoothing_radius = 10. Radius for smoothing squared density to find centers Choose a smaller value to get more center guessesn_centers = 1 Report up to n_centers centersn_center_find = None Target number of centers to find Smoothing radius will be varied from 2 to 2*smoothing_radius and the value giving the number of peaks closest to n_center_find will be usedcrystal_inforesolution = 0. high-resolution limit for map calculationsolvent_fraction = 0.5 solvent fractioncontrolverbose = True Verbose outputraise_sorry = False Raise sorry if problemsdebug = False Debugging outputdry_run = False Just read in and check parameter namesresolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' ";
- input_filesmtz_in = None MTZ file with coefficients for a map labin = "" Labin line for MTZ file with map coefficients. This is optional if guess_molecular_centers can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP
- mtz_in = None MTZ file with coefficients for a map
- labin = "" Labin line for MTZ file with map coefficients. This is optional if guess_molecular_centers can guess the correct coefficients for FP PHI and FOM. Otherwise specify: LABIN FP=myFP PHIB=myPHI FOM=myFOM where myFP is your column label for FP
- output_filetarget_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.pdb_out = guess_molecular_centers.pdb Output PDB filelog =

guess_molecular_centers.log Output log file
params_out = guess_molecular_centers_params.eff
Parameters file to rerun guess_molecular_centers

- target_output_format = *None pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- pdb_out = guess_molecular_centers.pdb Output PDB file
- log = guess_molecular_centers.log Output log file
- params_out = guess_molecular_centers_params.eff Parameters file to rerun guess_molecular_centers
- directorytemp_dir = "temp_dir" Optional temporary work directory output_dir = "" Output directory where files are to be written
- temp_dir = "temp_dir" Optional temporary work directory
- output_dir = "" Output directory where files are to be written
- find_centerssmoothing_radius = 10. Radius for smoothing squared density to find centers Choose a smaller value to get more center guessesn_centers = 1 Report up to n_centers centersn_center_find = None Target number of centers to find Smoothing radius will be varied from 2 to 2*smoothing_radius and the value giving the number of peaks closest to n_center_find will be used
- smoothing_radius = 10. Radius for smoothing squared density to find centers Choose a smaller value to get more center guesses
- n_centers = 1 Report up to n_centers centers
- n_center_find = None Target number of centers to find Smoothing radius will be varied from 2 to 2*smoothing_radius and the value giving the number of peaks closest to n_center_find will be used
- crystal_inforesolution = 0. high-resolution limit for map calculationsolvent_fraction = 0.5 solvent fraction
- resolution = 0. high-resolution limit for map calculation
- solvent_fraction = 0.5 solvent fraction
- controlverbose = True Verbose outputraise_sorry = False Raise sorry if problemsdebug = False Debugging outputdry_run = False Just read in and check parameter namesresolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' ";
- verbose = True Verbose output
- raise_sorry = False Raise sorry if problems
- debug = False Debugging output
- dry_run = False Just read in and check parameter names
- resolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' ";

phenix.reduce: tool for adding hydrogens to a PDB model

hydrogens.html

Purpose

phenix.reduce is a command line tool for adding hydrogens to a PDB structure file. Hydrogens are added in standardized geometry with optimization of the orientations of OH, SH, NH₃⁺, Met methyls, Asn and Gln sidechain amides, and His rings. Both proteins and nucleic acids can be processed. HET groups can also be processed as long as the atom connectivity is provided. The program is described in Word, et al.(1999). J. Mol. Biol. 285, 1735-1747. For more information visit: <http://kinemage.biochem.duke.edu/software/reduce.php>

How to run

phenix.reduce is run from the command line:

```
% phenix.reduce [pdb_file] [options]
```

To get information about command line options type:

```
% phenix.reduce
```

or for a longer list:

```
% phenix.reduce -h
```

Hydrogens in refinement

Please refer to phenix.refine documentation to see how hydrogen atoms are used in structure refinement.

Phenix Documentation - version dev-svn

index.html

Phenix help chatbot, AI agent, and AI analysis tool

You can use the Phenix chatbot to get help on using any Phenix program. Just ask it any question about using Phenix.

You can use the Phenix AI Agent to help you run Phenix tools with your data in the GUI or from the command line.

You can also use the Phenix AI analysis tool to summarize the results of a run in the GUI and suggest next steps.

- Instructions for using the chatbot
- Phenix AI tool overview
- Phenix AI analysis tool
- Phenix AI Agent

Introduction to Phenix

- What is Phenix
- How to install and run Phenix
- The Phenix graphical interface
- The main modules in Phenix
- Phenix atom selection syntax
- FAQS: Frequently asked questions
- Dictionary of common terms
- Overview of file formats
- Index for Phenix documentation
- Tutorials and Examples
- Finding a program or regression test in Phenix

Tools for analysing and manipulating experimental data in Phenix

- Data quality assessment with phenix.xtriage (GUI version)
- Planning a SAD experiment with phenix.plan_sad_experiment
- Scaling unmerged anomalous data or multiple datasets with phenix.scale_and_merge
- Analyzing anomalous signal in a SAD experiment with phenix.anomalous_signal
- Data quality assessment with phenix.reflection_statistics
- Structure factor file inspection and conversions
- Graphical editor for MTZ files
- Manipulating reflection data with phenix.xmanip

- Exploring the symmetry of your crystal with `phenix.explore_metric_symmetry`
- Importing data from the PDB with `phenix.cif_as_mtz`
- Visualizing reciprocal-space data
- Using unmerged data in Phenix
- Showing information about a map file
- Calculate local resolution from two cryo-EM half-maps

Experimental phasing

- Automated Structure Solution using AutoSol (GUI version)
- SAD phasing in the Phaser GUI
- Substructure determination with `phenix.hyss`
- Comparison of substructure sites with `phenix.emma`
- Setting up SAD data from the PDB with `phenix.sad_data_from_pdb`
- Frequently asked questions about experimental phasing

Molecular replacement

- Overview of molecular replacement in Phenix
- Molecular replacement in the Phaser GUI
- MRage: automated molecular replacement
- MR and model building with AutoMR, AutoBuild and Rosetta
- Morphing a poor model into electron density
- Documentation for the Phaser program
- Ensembler: create ensemble of models for molecular replacement
- Sculptor: modify input model for molecular replacement
- Frequently asked questions about molecular replacement

Model-building

- Solve an X-ray structure or interpret a cryo-EM map with `PredictAndBuild`
- Automated Model Building and Rebuilding using AutoBuild (GUI version)
- Rapid cryo-EM model-building with `trace_and_build`
- Dock chains in Cryo-EM map with `dock_in_map`
- Likelihood-based single model docking in Cryo-EM map with `em_placement`
- Dock AlphaFold model with `dock_predicted_model`
- Dock and rebuild AlphaFold model with `dock_and_rebuild`
- Cryo-EM model rebuilding with `rebuild_model`
- Cryo-EM model-building with Pathwalker
- Low-resolution/cryo-EM model-building with `map_to_model`
- Rapid model-building with `find_helices_strands`

- Fitting loops with `fit_loops`
- Building starting with a very poor map with `parallel_autobuild`
- Rapid iterative map improvement and model-building with `phase_and_build`
- Rapid model-building with `build_one_model`
- Assigning sequence to a model with `assign_sequence`
- Replace side chains with rotamers using `replace_side_chains`
- Combine the best parts of two models with `combine_models`
- Remove incorrect protein from complex with `remove_poor_fragments`
- Fixing register errors with `fix_insertions_deletions`
- Adjusting a CA/CB model with `refine_ca_model`
- Frequently asked questions about model-building
- Build water into cryo-EM map
- Create a model with alternative side-chain conformations with `phenix.create_alt_conf`
- Analyze a model with alternative side-chain conformations with `phenix.analyze_alt_conf`

Structure refinement and restraint generation

- AQuaRef: AI-enabled Quantum Refinement of macromolecular models <AQuaRef.html>
- Structure refinement with `phenix.refine` (GUI version)
- Determining non-crystallographic symmetry (NCS) from a PDB file with `phenix.simple_ncs_from_pdb`
- Finding and analyzing NCS from heavy-atom sites or a model with `phenix.find_ncs`
- Generating ligand coordinates and restraints using eLBOW (GUI version)
- Editing ligand restraints from eLBOW using REEL
- Add hydrogens; generate metal coordination files; introduce neutron exchange sites using ReadySet!
- Adding secondary structure restraints
- Mode idealization before refinement
- Generating hydrogen atoms for refinement using `phenix.reduce`
- Refinement in real space with `phenix.real_space_refine`
- Ensemble refinement
- Variability refinement
- DEN refinement with `phenix.den_refine`
- Hybrid Rosetta/Phenix refinement of protein X-ray crystal structures
- Hybrid Phenix/Amber refinement for protein and RNA/DNA X-ray crystal structure
- Hybrid Phenix/AFITT refinement for ligands in X-ray crystal structure
- Frequently asked questions (FAQs) about `phenix.refine`

Validation

- Validation in the Phenix GUI
- `phenix.model_vs_data`: compute model, data and model-to-data fit statistics

- Low-resolution secondary structure validation with CaBLAM
- Map statistics (resolution, etc) with Mtriage
- `phenix.get_pdb_validation_report`: get validation report from RCSB via OneDep API
- Model geometry evaluation with `phenix.holton_geometry_validation`

Ligand fitting and restraints

- Quantum Interface (QI)
- Automated Ligand Fitting using LigandFit (GUI version)
- Automated Ligand Fitting using Guided Ligand Replacement (GLR)
- Automated ligand identification in Phenix
- Finding all the ligands in a map with `phenix.find_all_ligands`
- Generating ligand coordinates and restraints using eLBOW (GUI version)
- Editing ligand restraints from eLBOW using REEL
- Automated protein-ligand structure determination with `phenix.ligand_pipeline`
- Frequently asked questions about ligand fitting and restraints

Carbohydrates

- Carbohydrate Primer
- Simple Carbohydrate Line-Entry System (SCaLES)

Map utilities

- `phenix.map_symmetry`: Map symmetry analysis
- Generating electron density maps in the GUI
- `phenix.maps`: tool to compute various maps and save them in most of known formats
- Generating composite omit maps
- `phenix.map_value_at_point`: compute map value at a given point
- `phenix.mtz2map`: FFT-ing map coefficients
- `phenix.ncs_average`: Simple NCS averaging
- `phenix.polder`: compute omit map excluding bulk solvent
- Correlation of maps or map and model with `map_correlations`
- Cut out density from a crystallographic map with `cut_out_density`
- Cut out density from a cryo-EM or crystallographic map with `map_box`
- Segment a cryo-EM map with `segment_and_split_map`
- Compare CA positions in two models with `chain_comparison`
- Optimize an Xray or cryo-EM map with `map_sharpening`
- Combine cryo-EM maps with `combine_focused_maps`
- Density modify a cryo-EM map with `resolve_cryo_em`
- Remove test set reflections from a map using `remove_free_from_map`

- Guessing molecular centers in a map with `guess_molecular_centers`
- Density modification with `phenix.density_modification`
- Density modification with multi-crystal averaging with `phenix.multi_crystal_average`
- Find NCS in a map with `find_ncs_from_density`
- Map superposition with `phenix.superpose_maps`
- Modify map coefficients to remove density near a model
- `phenix.fem`: compute Feature Enhanced Map (FEM)
- Residual difference map analysis with `phenix.find_peaks_holes`
- Convert map into structure factors `phenix.map_to_structure_factors`

Model utilities

- Predict AlphaFold2 models with `phenix.predict_model`
- Process AlphaFold2 and other predicted models with `phenix.process_predicted_model`
- Idealize model geometry with `phenix.geometry_minimization`
- Superimpose models with `phenix.superpose_models`
- Generating NCS copies with `apply_ncs`
- PDB model: statistics, manipulations and more with `phenix.pdbtools`
- `phenix.fmodel`: structure factors from a PDB file
- `phenix.f000`: given PDB file and mean bulk-solvent density estimate $F(0,0,0)$
- Map one PDB file close to another using SG symmetry with `phenix.map_to_object`
- SCEDS - identify domains with normal mode analysis
- `structural_domains_search` - identify domains using a graph search
- Find symmetry equivalent molecular replacement solution on alternative origin with `phenix.find_alt_orig_sym_mate`
- Building biological assembly and crystallographic asymmetric unit
- Calculating Fragment Antigen-Binding (Fab) elbow angle
- Find Similar Structures in PDB using phi-psi angles and sequence
- Find Structures in PDB using SSM matching and morphing
- Find fragments in PDB matching a target fragment
- Replace model with fragments from the PDB
- Shaking a structure with simple dynamics
- Sorting ligands and waters to group with polymer chains
- Trim one model to avoid atoms in another with `trim_overlapping`

Other tools in Phenix

- Setting up GUI tests in Phenix
- Finding a program or regression test in Phenix
- Running regression tests in Phenix

- Characterizing raw data and model with LABELIT
- Running SOLVE/RESOLVE in Phenix
- Parallel validation and analysis of related structures
- phenix.reciprocal_space_arrays: compute various arrays such as Fcalc, Fmask, Fbulk, HL ...
- Generating Table 1 for publication

Useful tools outside of Phenix

- Manual model inspection and building with Coot
- Molecular graphics with PyMOL
- MolProbity - An **Active** Validation Tool

Appendix

- Hints on writing scripts with AI tools
- Generating code documentation strings with with AI tools
- Phenix html documentation generation procedures
- Phenix Windows build procedures
- Bundling base components as zip files for the Windows build
- Creating a Windows installer for PHENIX

Optimizing a map with local_aniso_sharpen (deprecated, use MapSharpening)

[local_aniso_sharpen.html](#)

Optimizing a map with local_aniso_sharpen (deprecated, use MapSharpening)

Author(s)

- local_aniso_sharpen: Tom Terwilliger

Purpose

The routine local_aniso_sharpen is a tool for optimizing a map by applying a local, anisotropic, resolution-dependent scaling factor.

How local_aniso_sharpen works:

Local anisotropic sharpening can be carried out locally, as the name suggests, or on a map as a whole.

The basis for sharpening is an analysis of the resolution-dependent fall-off of amplitudes of Fourier coefficients for the map, along with an analysis of the resolution-dependent fall-off of correlation between two half-maps, or between a map and a model-based map.

When carried out locally, a full map is divided into small boxes. The density near the edges of each box is masked so that it gradually diminishes to zero at the edges. Each small box is treated as a full map to identify its optimal sharpening. Then the optimal sharpening parameters from the small boxes are applied to the full map in a way that has no edge effects and smoothly varies from one place to another in the map.

To identify the optimal anisotropic sharpening of a map based on the information in two half-maps, two analyses are done. The first is an analysis of the resolution-dependent fall-off of rms amplitudes of Fourier coefficients representing the map. This is examined as a function of direction in reciprocal space, and is similar to the calculation normally done to apply an anisotropy correction to a map. This analysis shows the anisotropy of the map itself.

The second is an analysis of the correlation between Fourier coefficients for the two half-maps. This is also done as a function of resolution and direction in reciprocal space. This analysis shows the anisotropy of the errors in the map.

For purposes of this analysis, the optimal map is the one that has the maximal expected correlation to an idealized version of the true map. This idealized map is a map that would be obtained from a model where all the atoms are point atoms (B values of about zero).

For a map with zero error (all correlations at all resolutions and directions equal to 1), the optimal map will be one that has no anisotropy and the same resolution dependence as the idealized map. For such a map, first the anisotropy in the map is removed, then an overall resolution-dependence matching that of the idealized map is imposed by simple multiplication with a resolution_dependent scale factor.

For a map with errors, the map coefficients obtained from the previous step are modified by a local scale factor that reflects the expected signal-to-noise in that map coefficient. The scale factor for a particular map coefficient is given by $1/(1 + E^2)$, where E is the normalized expected error in that map coefficient. This

scale factor will ordinarily be anisotropic and resolution-dependent.

Examples

You can use `local_aniso_sharpen` with either two half-maps or a map and a model.

Standard run of `local_aniso_sharpen` with two half-maps:

To run `local_aniso_sharpen` with two half maps, you can say:

```
phenix.local_aniso_sharpen half_map_A.mrc half_map_B.mrc seq_file=seq.fa
```

If you wish, you can specify a nominal resolution. The sequence file is to let the program figure out the volume of the molecule that is present.

To run `local_aniso_sharpen` with a map and model, you can say:

```
phenix.local_aniso_sharpen map.mrc model.pdb resolution=3 seq_file=seq.fa
```

The resolution is again optional.

You can specify whether anisotropy or local sharpening are to be applied:

```
local_sharpen=True  
anisotropic_sharpen=True
```

Output

`Local_aniso_sharpen` will provide a summary of the anisotropy of your map if you leave `anisotropic_sharpen=True`. Here are some of the values that are calculated for a test case, with annotations:

```
NOTE: All values apply after removal of overall U of:  
(0.81, 0.84, 0.78, -0.03, -0.01, 0.04)  
  
>>> This overall U value has large and similar values for the first 3  
>>> numbers, and small values for the last 3. This indicates that this  
>>> map is relatively isotropic.  
>>> In this output, the overall U listed above is removed from all other  
>>> U values before printing them out.  
  
>>> All values are reported in normalized units (U values). See  
>>> https://www.iucr.org/\_\_data/iucr/cifdic\_html/1/cif\_core.dic/Iatom\_site\_U\_iso\_or\_equiv.html  
  
Estimated anisotropic fall-off of the data relative to ideal  
(Positive means amplitudes fall off more in this direction)  
( X, Y, Z, XY, XZ, YZ)  
(0.002, 0.089, 0.037, 0.031, 0.042, -0.002)  
  
>>> This is the fall-off of the data, after removal of the overall U values,  
>>> relative to a standard of values calculated from a beta-galactosidase  
>>> model without a bulk solvent correction (available from the phenix  
>>> python module cctbx.development.approx_amplitude_vs_resolution)  
  
Anisotropy of the data  
(Positive means amplitudes fall off more in this direction)  
( X, Y, Z, XY, XZ, YZ)  
(-0.119, 0.167, -0.133, -0.013, 0.031, 0.064)  
  
>>> This is the anisotropy of the data, after removal of overall U values  
  
Anisotropy of the uncertainties  
(Positive means uncertainties decrease more in this direction)  
( X, Y, Z, XY, XZ, YZ)  
(0.160, -0.032, -0.035, 0.031, 0.054, -0.010)
```

>> This is the anisotropy of the uncertainties in scale factors,
> after removal of overall U values

Anisotropy of the scale factors
(Positive means scale factors increase more in this direction)
(X, Y, Z, XY, XZ, YZ)
(-0.022, -0.121, -0.088, -0.050, -0.040, -0.019)

>> This is the anisotropy of the scale factors,
> after removal of overall U values

Possible Problems

If the half-maps are not actually independent the procedure will not work well

If the model is very poor the procedure will not work well

For model-based sharpening, if local sharpening is used, the sharpening is only applied in the region of the model

Specific limitations and problems:

Literature

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
input_filesseq_file = None Sequence file. Include all copies of each sequence so that the full contents of the map can be estimated. (Fasta format or sequences separated by blank lines)
map_modelfull_map = None Input full map file
half_map = None Input half map files
model = None Input model file
outputsharpened_map_file = default Sharpened map file
namesharpened_map_file_1 = default Sharpened half map 1 file
namesharpened_map_file_2 = default Sharpened half map 2 file
namelocal_resolution_map_file = default Local resolution map file
nameoverwrite = True Overwrite files with same names
file_name = None Not used
filename = None Not used
serial = None Not used
output_scale_factor = None Scale factor to be applied to output map just before writing. Normally the output map will have a mean of zero and SD of 1. This may lead to the maximum in the map being much greater than 1. You can adjust the output SD with this scale factor.
sharpeninglocal_sharpen = False Sharpen locally (alternative is global sharpening). Note: can take a long time
anisotropic_sharpen = True Use anisotropic sharpening. Can be combined with local sharpening
model_sharpen = None Model sharpening (default if model is supplied and only one map is supplied)
n_bins = None Number of bins for sharpening (default 200 overall and 20 local)
n_boxes = None Number of boxes
box_size_grid_units = None Size of core region of boxes (not including region where mask is applied, in grid units)
sharpen_all_maps = True Sharpen and write out half-maps in addition to the full map
crystal_inforesolution = None Nominal resolution of map
wrapping = None You can specify whether the map is wrapped (can map values outside bounds to inside with cell translations).
controlmultiprocessing = *multiprocessing sge lsf pbs condor pbspro slurm Choices are multiprocessing (single machine) or queuing systems Not implemented
queue_run_command = None run command for queue jobs. For example qsub. Not implemented
nproc = 1 Number of processors to use. NOTE: by default multiple processors will only be used in the map-to-model step (this is because multiprocessing requires writing out nproc sets of huge files and it can be very slow with distributed queues.). You can override this with force_nproc = True.
ignore_symmetry_conflicts = False You can ignore the symmetry information (CRYST1) from coordinate files. This may be necessary if your model has been placed in a box with box_map

for example.verbose = False Verbose outputguiGUI-specific parameter required for output directoryoutput_dir = None

- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesseq_file = None Sequence file. Include all copies of each sequence so that the full contents of the map can be estimated. (Fasta format or sequences separated by blank lines)map_modelfull_map = None Input full map filehalf_map = None Input half map filesmodel = None Input model file
- seq_file = None Sequence file. Include all copies of each sequence so that the full contents of the map can be estimated. (Fasta format or sequences separated by blank lines)
- map_modelfull_map = None Input full map filehalf_map = None Input half map filesmodel = None Input model file
- full_map = None Input full map file
- half_map = None Input half map files
- model = None Input model file
- outputsharpened_map_file = default Sharpened map file namesharpened_map_file_1 = default Sharpened half map 1 file namesharpened_map_file_2 = default Sharpened half map 2 file name
local_resolution_map_file = default Local resolution map file name
overwrite = True Overwrite files with same names
file_name = None Not used
filename = None Not used
serial = None Not used
output_scale_factor = None Scale factor to be applied to output map just before writing. Normally the output map will have a mean of zero and SD of 1. This may lead to the maximum in the map being much greater than 1. You can adjust the output SD with this scale factor.
- sharpened_map_file = default Sharpened map file name
- sharpened_map_file_1 = default Sharpened half map 1 file name
- sharpened_map_file_2 = default Sharpened half map 2 file name
- local_resolution_map_file = default Local resolution map file name
- overwrite = True Overwrite files with same names
- file_name = None Not used
- filename = None Not used
- serial = None Not used
- output_scale_factor = None Scale factor to be applied to output map just before writing. Normally the output map will have a mean of zero and SD of 1. This may lead to the maximum in the map being much greater than 1. You can adjust the output SD with this scale factor.
- sharpeninglocal_sharpen = False Sharpen locally (alternative is global sharpening). Note: can take a long time
anisotropic_sharpen = True Use anisotropic sharpening. Can be combined with local sharpening
model_sharpen = None Model sharpening (default if model is supplied and only one map is supplied)
n_bins = None Number of bins for sharpening (default 200 overall and 20 local)
n_boxes = None Number of boxes
box_size_grid_units = None Size of core region of boxes (not including region where mask is applied, in grid units)
sharpen_all_maps = True Sharpen and write out half-maps in addition to the full map
- local_sharpen = False Sharpen locally (alternative is global sharpening). Note: can take a long time
- anisotropic_sharpen = True Use anisotropic sharpening. Can be combined with local sharpening
- model_sharpen = None Model sharpening (default if model is supplied and only one map is supplied)
- n_bins = None Number of bins for sharpening (default 200 overall and 20 local)

- `n_boxes` = None Number of boxes
- `box_size_grid_units` = None Size of core region of boxes (not including region where mask is applied, in grid units)
- `sharpen_all_maps` = True Sharpen and write out half-maps in addition to the full map
- `crystal_inforesolution` = None Nominal resolution of mapwrapping = None You can specify whether the map is wrapped (can map values outside bounds to inside with cell translations).
- `resolution` = None Nominal resolution of map
- `wrapping` = None You can specify whether the map is wrapped (can map values outside bounds to inside with cell translations).
- `controlmultiprocessing` = *multiprocessing sge lsf pbs condor pbspro slurm Choices are multiprocessing (single machine) or queuing systems Not implemented
- `queue_run_command` = None run command for queue jobs. For example qsub. Not implemented
- `nproc` = 1 Number of processors to use. NOTE: by default multiple processors will only be used in the map-to-model step (this is because multiprocessing requires writing out nproc sets of huge files and it can be very slow with distributed queues.). You can override this with `force_nproc` = True.
- `ignore_symmetry_conflicts` = False You can ignore the symmetry information (CRYST1) from coordinate files. This may be necessary if your model has been placed in a box with `box_map` for example.
- `verbose` = False Verbose output
- `gui` GUI-specific parameter required for output directory
- `output_dir` = None

phenix.model_idealization

model_idealization.html

Description

The program for geometry idealization of protein secondary structure.

Program reads model from .pdb file, extracts information from HELIX/SHEET records if available and replaces these helices and sheets with geometrically ideal ones while preserving initial atom positions as much as possible.

If HELIX/SHEET records are not available, it will try to find them automatically. Correct secondary structure annotation is extremely important for proper functioning of this program. Incorrect annotations most likely will lead to distorted model. Please refer to SS documentation for more details.

If input .pdb file contains unknown ligands, their description should be supplied. This description can be obtained using eLBOW (GUI version)

The model should not contain non-standard protein residues inside secondary structure elements. The presence of unknown (UNK) residues in model (outside SS elements) is not advised because appropriate geometry restraints will not be generated for them and therefore such residues may be significantly distorted.

Command line usage examples

```
% phenix.model_idealization model.pdb
```

GUI

Not available

List of all available keywords

model_file_name = None map_file_name = None User-provided map that will be used as
reference hkl_file_name = None User-provided X-ray data to generate 2mFo-DFc map that will be used as
reference data_labels = None Labels for experimental data. r_free_flags_labels = None Labels for free
reflections. ligands_file_name = None User-provided ligand restraints mask_and_he_map = False Mask and
histogram equalization of the input map trim_alternative_conformations = False Leave only atoms with empty
alt loc additionally_fix_rotamer_outliers = True At the late stage if rotamer is still outlier choose another one
with minimal clash with surrounding atoms use_ss_restraints = True Use Secondary Structure
restraints use_starting_model_for_final_gm = False Use supplied model for final geometry minimization.
Otherwise just use self. output_prefix = None output_pkl = False output_model_h = True
use_map_for_reference = True run_minimization_first = True run_minimization_last = True
use_hydrogens_in_minimization = False reference_map_resolution = 5 number_of_refinement_cycles = 5
cycles_to_converge = 2 Number of cycles of geometry optimization without considerable stat change before
stopping ignore_ncs = False Don't use NCS even if it is present in model. filter_input_ss = True Filter input
annotation debug = False Output all intermediate files verbose = False More output to log nonbonded_weight =
10000 apply_all_trans = True rama_weight = 1.0 scale_allowed = 1.0 rama_potential = *oldfield emsley
rama_selection = None Selection of part of the model for which Ramachandran restraints will be set
up. restrain_rama_outliers = True Apply restraints to Ramachandran outliers restrain_rama_allowed = True
Apply restraints to residues in allowed region on Ramachandran plot restrain_allowed_outliers_with_emsley =

False In case of restrain_rama_outliers=True and/or restrain_rama_allowed=True still restrain these residues with emsley. Make sense only in case of using oldfield potential.ss_idealizationenabled = True Enable secondary structure idealizationfile_name_before_regularization = None skip_empty_ss_elements = True skip SS element if there is no atoms in the structure skip_good_ss_elements = False Skip SS elements without wrong/outlier Ramachandran angles and with reasonable hydrogen bonds lengthrestrain_torsion_angles = False Restraine torsion anglesfix_rotamer_outliers = True Fix rotamer outliers after replacing SS elementssigma_on_reference_non_ss = 1 Weight on original model coordinates restraints where no ss present. Keeps loops close to initial model. (bigger number gives lighter restraints)sigma_on_reference_helix = 1 Weight on original model coordinates restraints where helix present. Bends helices a bit according to initial model. (bigger number gives lighter restraints)sigma_on_reference_sheet = 0.5 Weight on original model coordinates restraints where sheet present. Bends helices a bit according to initial model. (bigger number gives lighter restraints)sigma_on_torsion_ss = 5 Weight on torsion angles restraints where ss present. Keeps helices torsion angles close to ideal. (bigger number gives lighter restraints)sigma_on_torsion_nonss = 5 sigma_on_ramachandran = 1 sigma_on_cbeta = 2.5 n_macro = Auto n_iter = 300 nproc = 1 verbose = False oldfieldesd = 10.0 weight_scale = 1.0 dist_weight_max = 10.0 weight = None plot_cutoff = 0.027 secondary_structuress_by_chain = True Only applies if search_method = from_ca. Find secondary structure only within individual chains. Alternative is to allow H-bonds between chains. Can be much slower with ss_by_chain=False. If your model is complete use ss_by_chain=True. If your model is many fragments, use ss_by_chain=False.from_ca_conservative = False various parameters changed to make from_ca method more conservative, hopefully to closer resemble ksdssp.max_rmsd = 1 Only applies if search_method = from_ca. Maximum rmsd to consider two chains with identical sequences as the same for ss identificationuse_representative_chains = True...

- model_file_name = None
- map_file_name = None User-provided map that will be used as reference
- hkl_file_name = None User-provided X-ray data to generate 2mFo-DFc map that will be used as reference
- data_labels = None Labels for experimental data.
- r_free_flags_labels = None Labels for free reflections.
- ligands_file_name = None User-provided ligand restraints
- mask_and_he_map = False Mask and histogram equalization of the input map
- trim_alternative_conformations = False Leave only atoms with empty altloc
- additionally_fix_rotamer_outliers = True At the late stage if rotamer is still outlier choose another one with minimal clash with surrounding atoms
- use_ss_restraints = True Use Secondary Structure restraints
- use_starting_model_for_final_gm = False Use supplied model for final geometry minimization. Otherwise just use self.
- output_prefix = None
- output_pkl = False
- output_model_h = True
- use_map_for_reference = True
- run_minimization_first = True
- run_minimization_last = True

- use_hydrogens_in_minimization = False
- reference_map_resolution = 5
- number_of_refinement_cycles = 5
- cycles_to_converge = 2 Number of cycles of geometry optimization without considerable stat change before stopping
- ignore_ncs = False Don't use NCS even if it is present in model.
- filter_input_ss = True Filter input annotation
- debug = False Output all intermediate files
- verbose = False More output to log
- nonbonded_weight = 10000
- apply_all_trans = True
- rama_weight = 1.0
- scale_allowed = 1.0
- rama_potential = *oldfield emsley
- rama_selection = None Selection of part of the model for which Ramachandran restraints will be set up.
- restrain_rama_outliers = True Apply restraints to Ramachandran outliers
- restrain_rama_allowed = True Apply restraints to residues in allowed region on Ramachandran plot
- restrain_allowed_outliers_with_emsley = False In case of restrain_rama_outliers=True and/or restrain_rama_allowed=True still restrain these residues with emsley. Make sense only in case of using oldfield potential.
- ss_idealizationenabled = True Enable secondary structure idealizationfile_name_before_regularization = None skip_empty_ss_elements = True skip SS element if there is no atoms in the structureskip_good_ss_elements = False Skip SS elements without wrong/outlier Ramachandran angles and with reasonable hydrogen bonds lengthrestrain_torsion_angles = False Restraining torsion anglesfix_rotamer_outliers = True Fix rotamer outliers after replacing SS elementssigma_on_reference_non_ss = 1 Weight on original model coordinates restraints where no ss present. Keeps loops close to initial model. (bigger number gives lighter restraints)sigma_on_reference_helix = 1 Weight on original model coordinates restraints where helix present. Bends helices a bit according to initial model. (bigger number gives lighter restraints)sigma_on_reference_sheet = 0.5 Weight on original model coordinates restraints where sheet present. Bends helices a bit according to initial model. (bigger number gives lighter restraints)sigma_on_torsion_ss = 5 Weight on torsion angles restraints where ss present. Keeps helices torsion angles close to ideal. (bigger number gives lighter restraints)sigma_on_torsion_nonss = 5 sigma_on_ramachandran = 1 sigma_on_cbeta = 2.5 n_macro = Auto n_iter = 300 nproc = 1 verbose = False
- enabled = True Enable secondary structure idealization
- file_name_before_regularization = None
- skip_empty_ss_elements = True skip SS element if there is no atoms in the structure
- skip_good_ss_elements = False Skip SS elements without wrong/outlier Ramachandran angles and with reasonable hydrogen bonds length
- restrain_torsion_angles = False Restraining torsion angles

- `fix_rotamer_outliers = True` Fix rotamer outliers after replacing SS elements
- `sigma_on_reference_non_ss = 1` Weight on original model coordinates restraints where no ss present. Keeps loops close to initial model. (bigger number gives lighter restraints)
- `sigma_on_reference_helix = 1` Weight on original model coordinates restraints where helix present. Bends helices a bit according to initial model. (bigger number gives lighter restraints)
- `sigma_on_reference_sheet = 0.5` Weight on original model coordinates restraints where sheet present. Bends helices a bit according to initial model. (bigger number gives lighter restraints)
- `sigma_on_torsion_ss = 5` Weight on torsion angles restraints where ss present. Keeps helices torsion angles close to ideal. (bigger number gives lighter restraints)
- `sigma_on_torsion_nonss = 5`
- `sigma_on_ramachandran = 1`
- `sigma_on_cbeta = 2.5`
- `n_macro = Auto`
- `n_iter = 300`
- `nproc = 1`
- `verbose = False`
- `oldfieldesd = 10.0` `weight_scale = 1.0` `dist_weight_max = 10.0` `weight = None` `plot_cutoff = 0.027`
- `esd = 10.0`
- `weight_scale = 1.0`
- `dist_weight_max = 10.0`
- `weight = None`
- `plot_cutoff = 0.027`
- `secondary_structuress_by_chain = True` Only applies if `search_method = from_ca`. Find secondary structure only within individual chains. Alternative is to allow H-bonds between chains. Can be much slower with `ss_by_chain=False`. If your model is complete use `ss_by_chain=True`. If your model is many fragments, use `ss_by_chain=False`.
`from_ca_conservative = False` various parameters changed to make `from_ca` method more conservative, hopefully to closer resemble `ksdssp`.
`max_rmsd = 1` Only applies if `search_method = from_ca`. Maximum rmsd to consider two chains with identical sequences as the same for ss identification.
`use_representative_chains = True` Only applies if `search_method = from_ca`. Use a representative of all chains with the same sequence. Alternative is to examine each chain individually. Can be much slower with `use_representative_of_chain=False` if there are many symmetry copies. Ignored unless `ss_by_chain` is `True`.
`max_representative_chains = 100` Only applies if `search_method = from_ca`. Maximum number of representative chains.
`enabled = False` Turn on secondary structure restraints (main switch).
`proteinenabled = True` Turn on secondary structure restraints for protein.
`search_method = *ksdssp from_ca cablam` Particular method to search protein secondary structure.
`distance_ideal_n_o = 2.9` Target length for N-O hydrogen bond.
`distance_cut_n_o = 3.5` Hydrogen bond with length exceeding this value will not be established.
`remove_outliers = True` If true, h-bonds exceeding `distance_cut_n_o` length will not be established.
`restrain_hbond_angles = True`
`helixserial_number = None` `helix_identifier = None` `enabled = True` Restraining this particular helix.
`selection = None` `helix_type = *alpha pi 3_10 unknown` Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints.
`sigma = 0.05` `slack = 0` `top_out = False` `angle_sigma_scale = 1` Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
`angle_sigma_set = None` Use this parameter to set sigmas for h-bond angles to a particular value.
`hbonddonor = None`

acceptor = None sheetenabled = True Restraining this particular sheetfirst_strand = None sheet_id = None
sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this
value. Original sigmas range from 5 to 10.angle_sigma_set = None Use this parameter to set sigmas for
h-bond angles to a particular valuestrandselection = None sense = parallel antiparallel *unknown
bond_start_current = None bond_start_previous = None hbonddonor = None acceptor = None
nucleic_acidenabled = True Turn on secondary structure restraints for nucleic
acidshbond_distance_cutoff = 3.4 Hydrogen bonds with length exceeding this limit will not be
establishedscale_bonds_sigma = 1. All sigmas for h-bond length will be multiplied by this number. The
smaller number is tighter restraints.base_pairenabled = True Restraining this particular base-pairbase1 =
None Selection string selecting at least one atom in the desired residuebase2 = None Selection string
selecting at least one atom in the desired residuesaenger_class = 0 Type of
base-pairingrestrain_planarity = False Apply planarity restraint to this base-pairplanarity_sigma = 0.176
restrain_hbonds = True Restraining hydrogen bonds length for this base-pairrestrain_hb_angles = True
Restraining angles around hydrogen bonds for this base-pairrestrain_parallelity = True Apply parallelity
restraint to this base-pairparallelity_target = 0 parallelity_sigma = 0.0335 stacking_pairenabled = True
Restraining this particular base-pairbase1 = None Selection string selecting at least one atom in the desired
residuebase2 = None Selection string selecting at least one atom in the desired residueangle = 0 sigma =
0.027

- ss_by_chain = True Only applies if search_method = from_ca. Find secondary structure only within individual chains. Alternative is to allow H-bonds between chains. Can be much slower with ss_by_chain=False. If your model is complete use ss_by_chain=True. If your model is many fragments, use ss_by_chain=False.
- from_ca_conservative = False various parameters changed to make from_ca method more conservative, hopefully to closer resemble ksdssp.
- max_rmsd = 1 Only applies if search_method = from_ca. Maximum rmsd to consider two chains with identical sequences as the same for ss identification
- use_representative_chains = True Only applies if search_method = from_ca. Use a representative of all chains with the same sequence. Alternative is to examine each chain individually. Can be much slower with use_representative_of_chain=False if there are many symmetry copies. Ignored unless ss_by_chain is True.
- max_representative_chains = 100 Only applies if search_method = from_ca. Maximum number of representative chains
- enabled = False Turn on secondary structure restraints (main switch)
- proteinenabled = True Turn on secondary structure restraints for proteinsearch_method = *ksdssp from_ca cablam Particular method to search protein secondary structure.distance_ideal_n_o = 2.9 Target length for N-O hydrogen bonddistance_cut_n_o = 3.5 Hydrogen bond with length exceeding this value will not be establishedremove_outliers = True If true, h-bonds exceeding distance_cut_n_o length will not be establishedrestrain_hbond_angles = True helixserial_number = None helix_identifier = None enabled = True Restraining this particular helixselection = None helix_type = *alpha pi 3_10 unknown Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints.sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular valuehbonddonor = None acceptor = None sheetenabled = True Restraining this particular sheetfirst_strand = None sheet_id = None sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular valuestrandselection = None

sense = parallel antiparallel *unknown bond_start_current = None bond_start_previous = None
hbonddonor = None acceptor = None

- enabled = True Turn on secondary structure restraints for protein
- search_method = *ksdssp from_ca cablam Particular method to search protein secondary structure.
- distance_ideal_n_o = 2.9 Target length for N-O hydrogen bond
- distance_cut_n_o = 3.5 Hydrogen bond with length exceeding this value will not be established
- remove_outliers = True If true, h-bonds exceeding distance_cut_n_o length will not be established
- restrain_hbond_angles = True
- helixserial_number = None helix_identifier = None enabled = True Restraining this particular helixselection = None helix_type = *alpha pi 3_10 unknown Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints.sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular valuehbonddonor = None acceptor = None
- serial_number = None
- helix_identifier = None
- enabled = True Restraining this particular helix
- selection = None
- helix_type = *alpha pi 3_10 unknown Type of helix, defaults to alpha. Only alpha, pi, and 3_10 helices are used for hydrogen-bond restraints.
- sigma = 0.05
- slack = 0
- top_out = False
- angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
- angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular value
- hbonddonor = None acceptor = None
- donor = None
- acceptor = None
- sheetenabled = True Restraining this particular sheetfirst_strand = None sheet_id = None sigma = 0.05 slack = 0 top_out = False angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular valuestrandselection = None sense = parallel antiparallel *unknown bond_start_current = None bond_start_previous = None hbonddonor = None acceptor = None
- enabled = True Restraining this particular sheet
- first_strand = None
- sheet_id = None
- sigma = 0.05
- slack = 0
- top_out = False

- angle_sigma_scale = 1 Multiply sigmas for h-bond angles by this value. Original sigmas range from 5 to 10.
- angle_sigma_set = None Use this parameter to set sigmas for h-bond angles to a particular value
- strandselection = None sense = parallel antiparallel *unknown bond_start_current = None bond_start_previous = None
- selection = None
- sense = parallel antiparallel *unknown
- bond_start_current = None
- bond_start_previous = None
- hbonddonor = None acceptor = None
- donor = None
- acceptor = None
- nucleic_acidenabled = True Turn on secondary structure restraints for nucleic acidshbond_distance_cutoff = 3.4 Hydrogen bonds with length exceeding this limit will not be establishedscale_bonds_sigma = 1. All sigmas for h-bond length will be multiplied by this number. The smaller number is tighter restraints.base_pairenable = True Restrict this particular base-pairbase1 = None Selection string selecting at least one atom in the desired residuebase2 = None Selection string selecting at least one atom in the desired residueaenger_class = 0 Type of base-pairingrestrain_planarity = False Apply planarity restraint to this base-pairplanarity_sigma = 0.176 restrain_hbonds = True Restrict hydrogen bonds length for this base-pairrestrain_hb_angles = True Restrict angles around hydrogen bonds for this base-pairrestrain_parallelity = True Apply parallelity restraint to this base-pairparallelity_target = 0 parallelity_sigma = 0.0335 stacking_pairenable = True Restrict this particular base-pairbase1 = None Selection string selecting at least one atom in the desired residuebase2 = None Selection string selecting at least one atom in the desired residueangle = 0 sigma = 0.027
- enabled = True Turn on secondary structure restraints for nucleic acids
- hbond_distance_cutoff = 3.4 Hydrogen bonds with length exceeding this limit will not be established
- scale_bonds_sigma = 1. All sigmas for h-bond length will be multiplied by this number. The smaller number is tighter restraints.
- base_pairenable = True Restrict this particular base-pairbase1 = None Selection string selecting at least one atom in the desired residuebase2 = None Selection string selecting at least one atom in the desired residueaenger_class = 0 Type of base-pairingrestrain_planarity = False Apply planarity restraint to this base-pairplanarity_sigma = 0.176 restrain_hbonds = True Restrict hydrogen bonds length for this base-pairrestrain_hb_angles = True Restrict angles around hydrogen bonds for this base-pairrestrain_parallelity = True Apply parallelity restraint to this base-pairparallelity_target = 0 parallelity_sigma = 0.0335
- enabled = True Restrict this particular base-pair
- base1 = None Selection string selecting at least one atom in the desired residue
- base2 = None Selection string selecting at least one atom in the desired residue
- saenger_class = 0 Type of base-pairing
- restrain_planarity = False Apply planarity restraint to this base-pair
- planarity_sigma = 0.176

- `restrain_hbonds = True` Restrain hydrogen bonds length for this base-pair
- `restrain_hb_angles = True` Restrain angles around hydrogen bonds for this base-pair
- `restrain_parallelity = True` Apply parallelity restraint to this base-pair
- `parallelity_target = 0`
- `parallelity_sigma = 0.0335`
- `stacking_pairenabled = True` Restraint this particular base-pair
`base1 = None` Selection string selecting at least one atom in the desired residue
`base2 = None` Selection string selecting at least one atom in the desired residue
`angle = 0`
`sigma = 0.027`
- `loop_idealizationenabled = True` `output_prefix = None` `minimize_whole = True` `fix_rotamers = True` Fix rotamers before minimization
`force_rama_fixes = False` If true, the procedure will pick and apply the best variant even if all of them are above thresholds to be picked straight away. Alternatively, when False, the procedure will accept failure and leave a ramachandran outlier intact.
`include_allowed = False` Try to fix residues in allowed Ramachandran region with CCD
`avoid_allowed_region = False` If true, CCD will not let residues go to allowed region on Ramachandran plot
`make_all_trans = True` If true, procedure will try to get rid of all twisted and cis-peptides making all of them trans.
`save_states = False` Save states of CCD. Generates a states file for every model. Warning! Significantly slower!
`number_of_ccd_trials = 3` How many times we are trying to fix outliers in the same chain
`variant_search_level = 2` how thoroughly variants will be explored (1-3)
`variant_number_cutoff = 50` how many first variants to take from generated
`variant_deviation_accept_level = low * med high very_high` what deviation from starting model is acceptable when screening variants. med is fine for real-space, low is for reciprocal space.
`expert_level = 2` `debug = False` Output of intermediate files
- `enabled = True`
- `output_prefix = None`
- `minimize_whole = True`
- `fix_rotamers = True` Fix rotamers before minimization
- `force_rama_fixes = False` If true, the procedure will pick and apply the best variant even if all of them are above thresholds to be picked straight away. Alternatively, when False, the procedure will accept failure and leave a ramachandran outlier intact.
- `include_allowed = False` Try to fix residues in allowed Ramachandran region with CCD
- `avoid_allowed_region = False` If true, CCD will not let residues go to allowed region on Ramachandran plot
- `make_all_trans = True` If true, procedure will try to get rid of all twisted and cis-peptides making all of them trans.
- `save_states = False` Save states of CCD. Generates a states file for every model. Warning! Significantly slower!
- `number_of_ccd_trials = 3` How many times we are trying to fix outliers in the same chain
- `variant_search_level = 2` how thoroughly variants will be explored (1-3)

- variant_number_cutoff = 50 how many first variants to take from generated
- variant_deviation_accept_level = low *med high very_high what deviation from starting model is acceptable when screening variants. med is fine for real-space, low is for reciprocal space.
- expert_level = 2
- debug = False Output of intermediate files
- cablam_idealizationenabled = True nproc = 1 Parallelization is not implementedrotation_angle = 30 angle to rotate the oxygen while searching for better conformationrequire_h_bond = True require presence of (new) h-bond after rotationdo_gm = False Run geometry minimization after rotationfind_ss_after_fixes = True re-evaluate SS after fixing Cablam outliers. May be helpful to identify new or extend previous SS elementssave_intermediates = False Save all cablam rotation for particular residue in separate file
- enabled = True
- nproc = 1 Parallelization is not implemented
- rotation_angle = 30 angle to rotate the oxygen while searching for better conformation
- require_h_bond = True require presence of (new) h-bond after rotation
- do_gm = False Run geometry minimization after rotation
- find_ss_after_fixes = True re-evaluate SS after fixing Cablam outliers. May be helpful to identify new or extend previous SS elements
- save_intermediates = False Save all cablam rotation for particular residue in separate file

FFT-ing map coefficients

mtz2map.html

If you already have map coefficients in an MTZ file and just want to run an FFT to produce X-plor or CCP4 map files, use `phenix.fft` or `phenix.mtz2map` (these are synonyms for the same program). In most cases this can be run automatically by passing the name of the MTZ file. MTZ files generated by `phenix.maps`, `phenix.refine`, `SOLVE/RESOLVE`, and `REFMAC` are supported, and additional combinations of map coefficients are possible if defined by the user. By default, output files will take the base name of the MTZ file, with the appropriate map type appended if it can be guessed from the column labels, otherwise a unique integer, followed by the file type extension (`.ccp4` or `.xplor`).

Other map types

For isomorphous difference maps (Fobs-Fobs), use `phenix.fobs_minus_fobs_map` (documentation to be added). For omit maps, including composite omit maps and iterative build omit maps, use the AutoBuild wizard. You can also generate a simple simulated-annealing omit map by deleting the relevant part of the molecule and running `phenix.refine` with `simulated_annealing=True`. (This is essentially the same as what is done by AutoBuild, however.)

- For isomorphous difference maps (Fobs-Fobs), use `phenix.fobs_minus_fobs_map` (documentation to be added).
- For omit maps, including composite omit maps and iterative build omit maps, use the AutoBuild wizard.
- You can also generate a simple simulated-annealing omit map by deleting the relevant part of the molecule and running `phenix.refine` with `simulated_annealing=True`. (This is essentially the same as what is done by AutoBuild, however.)

List of all available keywords

`mtz_file` = None MTZ file containing map coefficients
`pdb_file` = None Model file around which to draw the map. If not supplied, the map will fill the unit cell.
`labels` = None Map column labels. Common examples are 2FOFCWT,PH2FOFCWT and FP,SIGFP PHIM FOMM . If left blank, all maps present in the input file will be used.
`buffer` = 5.0 Extra padding (in Angstroms) around the selected region (or unit cell)
`selection` = None Atom selection around which to draw map (plus buffer). If left blank, the entire model file will be used.
`d_min` = None High-resolution cutoff
`d_max` = None Low-resolution cutoff
`grid_resolution_factor` = 0.25 Grid spacing (multiplied by the high-resolution limit)
`gridding` = None Gridding
`scale` = *sigma volume Scaling method for map values
`include_fmodel` = False
`show_maps` = False
`output_directory` = None Output directory (defaults to current)
`prefix` = None Output file prefix (defaults to the MTZ file name base)
`job_title` = None Job title in PHENIX GUI, not used on command line
`format` = xplor *ccp4 dsn6 extension = *Auto ccp4 xplor map dsn6
`r_free_flags_remove` = False
`file_name` = None
`label` = None
`test_flag_value` = None

- `mtz_file` = None MTZ file containing map coefficients
- `pdb_file` = None Model file around which to draw the map. If not supplied, the map will fill the unit cell.
- `labels` = None Map column labels. Common examples are 2FOFCWT,PH2FOFCWT and FP,SIGFP PHIM FOMM . If left blank, all maps present in the input file will be used.
- `buffer` = 5.0 Extra padding (in Angstroms) around the selected region (or unit cell)
- `selection` = None Atom selection around which to draw map (plus buffer). If left blank, the entire model file will be used.

- d_min = None High-resolution cutoff
- d_max = None Low-resolution cutoff
- grid_resolution_factor = 0.25 Grid spacing (multiplied by the high-resolution limit)
- gridding = None Gridding
- scale = *sigma volume Scaling method for map values
- include_fmodel = False
- show_maps = False
- outputdirectory = None Output directory (defaults to current)prefix = None Output file prefix (defaults to the MTZ file name base)job_title = None Job title in PHENIX GUI, not used on command lineformat = xplor *ccp4 dsn6 extension = *Auto ccp4 xplor map dsn6
- directory = None Output directory (defaults to current)
- prefix = None Output file prefix (defaults to the MTZ file name base)
- job_title = None Job title in PHENIX GUI, not used on command line
- format = xplor *ccp4 dsn6
- extension = *Auto ccp4 xplor map dsn6
- r_free_flagsremove = False file_name = None label = None test_flag_value = None
- remove = False
- file_name = None
- label = None
- test_flag_value = None

Density modification with multi-crystal averaging with `phenix.multi_crystal_average`

`multi_crystal_average.html`

Contents

- Author(s)
- Purpose
- Usage
- How `phenix.multi_crystal_average` works:
- Output files from `phenix.multi_crystal_average`
- Graphical interface
- Examples
- Standard run of `phenix.multi_crystal_average`:
- Run of `phenix.multi_crystal_average` with multiple domains:
- Run of `phenix.multi_crystal_average` using PDB files to define the NCS asymmetric unit:
- Possible Problems
- Specific limitations and problems:
- Literature
- Additional information
- List of all available keywords

Author(s)

- `phenix.multi_crystal_average`: Tom Terwilliger

Purpose

`phenix.multi_crystal_average` is a tool for carrying out density modification, including NCS symmetry within a crystal and electron density from multiple crystals.

Usage

How `phenix.multi_crystal_average` works:

The inputs to `phenix.multi_crystal_average` are a set of PDB files that define the NCS within each crystal and the relationships of density between crystals, structure factor amplitudes (and optional phases, FOM and HL coefficients) for each crystal, and starting electron density maps for one or more crystals.

The PDB files should each be composed of the exact same chains, placed in a different position and orientation for each NCS asymmetric unit of each crystal. You might create these PDB files by molecular replacement starting with the same search model for each crystal. You should not refine these MR solutions; they are only used to get the NCS relationships and the NCS will be more reliably found if the models for all NCS asymmetric units are identical. You can break the NCS asymmetric unit into domains and place them

independently. You can specify the domains by giving them unique chain IDs, (or you can use the routine `edit_chains.py` to do this for you, see below). A separate NCS group will be created for each domain. Additionally if your NCS asymmetric unit consists of more than one chain (A+B for example) then each chain will always be treated as a separate NCS group.

`phenix.multi_crystal_average` first uses the supplied PDB files to calculate NCS operators relating the NCS asymmetric unit in each crystal to all other NCS asymmetric units in that crystal and in other crystals. This is done by adding the unique chains in one crystal to each PDB file in turn, finding all the NCS relationships from all chains in that composite PDB file, and removing duplicate identity transformations. For example, suppose the NCS asymmetric unit is one chain (A,B,C,...). Then to relate all NCS asymmetric units to the NCS asymmetric unit of crystal 0, `phenix.multi_crystal_average` will compare all chains in the PDB file for each crystal to the unique chain in the PDB file for crystal 0, generating one NCS operator for each chain in each crystal. In this process the unique chain (in this case the NCS asymmetric unit of crystal 0) is renamed to a unique name (usually "***") and a composite PDB file is created with this chain along with all the chains in the PDB file for the crystal being considered, and `phenix.simple_ncs_from_pdb` is used to find the NCS operators. The centroids of the chains defining NCS are used as centers of the regions where the NCS operator is to be applied.

If the supplied PDB files have more than one domain or chain in each NCS asymmetric unit, then the domains or chains are grouped into separate NCS groups.

Once NCS operators have been identified, density modification is carried out sequentially on data from each crystal. During density modification for one crystal, the current electron density maps from all other crystals are used in generating target density for density modification in exactly the same way as NCS-related density is normally used when only a single crystal is available.

First the asymmetric unit of NCS is defined, in this case including the density in all NCS copies within the crystal being density modified as well as the density in all NCS copies in all other crystals. The asymmetric unit of NCS is the region over which the NCS operators apply. It is assumed to be identical for all NCS copies for all crystals, with orientation and position identified by the NCS operators. It is identified as the region over which all NCS copies have correlated density. If a mask for the protein/solvent boundary is supplied (by specifying "`use_model_mask`"), then the asymmetric unit of NCS is constrained to be within the non-solvent region of the map. Alternatively, if you request that the domains provided in your PDB files be used to define the NCS asymmetric unit (by specifying "`write_ncs_domain_pdb`") then the NCS asymmetric unit (for each NCS group) is limited to the region occupied by the corresponding chains in your PDB files.

Then a target density map is created for the crystal being density modified. For each NCS copy in this crystal, the average density for all other NCS copies in this and other crystals is used as a target.

Finally, statistical density modification is carried out using histograms of expected density, solvent flattening, and the NCS-based target density for this crystal. The process is then repeated for all other crystals. For those crystals for which no starting phases were available, one additional step is carried out in which the target density map is used by itself to calculate a starting electron density map (using RESOLVE map-based phasing).

This entire process is carried out several times, leading to electron density maps for all crystals that typically have a high level of correlation of density within all NCS copies in each crystal and between the corresponding NCS regions in different crystals.

Output files from `phenix.multi_crystal_average`

denmod_cycle_1_xl_0.mtz: Density-modified map coefficients for crystal 0, cycle 1. Crystal 0 is the first crystal specified in your pdb_list, map_coeff_list, etc.

denmod_cycle_5_xl_1.mtz: Density-modified map coefficients for crystal 1, cycle 5. These map coefficients are suitable for model-building. They also contain HL coefficients that can optionally be used in refinement. As the HL coefficients contain information from all crystals they may in some cases be useful in refinement (normally you would only use experimental HL phase information in refinement as the NCS-based information would come from your NCS restraints in refinement).

Graphical interface

The GUI is listed in the "Maps" category of the main interface. The configuration tab contains a set of tabs for the crystal forms: The label strings are automatically filled in when an MTZ file is loaded; these normally do not need to be further edited. At least one crystal form must have map coefficients, and all are required to have PDB and data files, as well as the estimated solvent content. All other options are on the second tab of the configuration panel: At the end of the run, the final density-modified and averaged map coefficients will be listed in a new tab. Selecting one of these and clicking the Coot or PyMOL buttons below will load the map coefficients and corresponding model for that crystal form.

Examples

Standard run of phenix.multi_crystal_average:

Running phenix.multi_crystal_average from the command line is easy. Usually you will want to edit a small parameter file (run_multi.eff) to contain your commands like this:

```
# run_multi.eff commands for running phenix.multi_crystal_average
# use: "phenix.multi_crystal_average run_multi.eff"
multi {
  crystal {
    pdb_file= "gene-5.pdb"
    map_coeffs = "resolve_1_offset.mtz"
    datafile = "phaser_1_offset.mtz"
    datafile_labin = "FP=F SIGFP=SIGF PHIB=PHI FOM=FOM HLA=HLA HLB=HLB HLC=HLC HLD=HLD"
    solvent_content = 0.43
  }
  crystal {
    pdb_file= "multi.pdb"
    map_coeffs = "None"
    datafile = "multi.pdb.mtz"
    datafile_labin = "FP=FP"
    solvent_content = 0.50
  }
}
```

Then you can run this with the command:

```
phenix.multi_crystal_average run_multi.eff
```

In this example we have 2 crystals. Crystal 1 has starting map coefficients in crystal_1_map_coeffs.mtz and data for FP in crystal_1_data.mtz. The contents of this crystal are represented by crystal_1.pdb. The second crystal has no starting map, has data for FP as well as PHI and HL coefficients in crystal_2_data.mtz, and the contents of this crystal are represented by crystal_2.pdb. The solvent contents of the 2 crystals are 0.43 and 0.50, and 5 overall cycles are to be done.

NOTE: Usually you will want to supply coefficients for a density-modified map (if available, and perhaps with NCS as well) for your starting map coefficients, and you will want to supply experimental phases and HL

coefficients for your datfiles.

The column label strings like "FP=FP" are optional...if you say instead "None" then phenix.multi_crystal_average will guess them for you. NOTE: You need to put quotes around the None. Also note: the order of entering pdb_list, map_coeff_list, data_file_list, solvent_content_list etc matters. They must all match.

Run of phenix.multi_crystal_average with multiple domains:

If your PDB files have more than one NCS domain within a chain, then you may want to split the chains up into sub-chains representing the individual NCS domains. This will provide a better definition of the NCS operators when the PDB files are analyzed.

You can use the jiffy "edit_chains.py" to do this. This jiffy splits your chains up into sub-chains based on the domains that you specify in "edit_chains.dat".

NOTE: edit_chains.py only works if your chains have single-letter ID's. (It simply adds another character to your chain ID's to make new ones.) If you have two-letter chain ID's, then you'll have to do this another way. To use it, type:

```
phenix.python $PHENIX/phenix/phenix/autosol/edit_chains.py file.pdb edited_file.pdb
```

The file edit_chains.dat is required and should look like:

```
A 1 321
A 322 597
A 598 750
A 751 902
A 903 1082
B 1 58
B 424 425
B 59 101
B 343 423
B 102 342
```

where the letter and residue range is your chain ID and residue range for a particular domain. You should specify these for ALL chains in your PDB files (not just the unique ones).

Run of phenix.multi_crystal_average using PDB files to define the NCS asymmetric unit:

If you specify the parameter write_ncs_domain_pdb=True, then phenix.multi_crystal_average will write out domain-specific PDB files for each domain in your model (based on its analysis of NCS, one for each NCS group). Then it will use those domain-specific PDB files to define the region over which the corresponding set of NCS operators apply. This is generally a good idea if you have multiple domains in your structure.

Possible Problems

Specific limitations and problems:

- The format for the parameters file for phenix.multi_crystal_average has changed (twice) (from a list of inputs for pdb_list etc, to several groups of inputs, one for each crystal.)

The format for the parameters file for phenix.multi_crystal_average has changed (twice) (from a list of inputs for pdb_list etc, to several groups of inputs, one for each crystal.)

- If the NCS asymmetric unit of your crystal contains more than one type of chain, `phenix.multi_crystal_average` will consider it to have more than one domain. This limitation comes from `phenix.simple_ncs_from_pdb`, which assigns one NCS group to each unique chain in the NCS asymmetric unit. If you would like `phenix.multi_crystal_average` to consider several chains as a single NCS group, then you would need to rename your chains and residues so that all the residues in a single NCS group have the same chain name and so that residue numbers are not duplicated. Normally you not need to do this, but if you want to use `phenix.multi_crystal_average` to generate phases for one crystal from another and you have more than one chain in the NCS asymmetric unit you would have to do this.

If the NCS asymmetric unit of your crystal contains more than one type of chain, `phenix.multi_crystal_average` will consider it to have more than one domain. This limitation comes from `phenix.simple_ncs_from_pdb`, which assigns one NCS group to each unique chain in the NCS asymmetric unit. If you would like `phenix.multi_crystal_average` to consider several chains as a single NCS group, then you would need to rename your chains and residues so that all the residues in a single NCS group have the same chain name and so that residue numbers are not duplicated. Normally you not need to do this, but if you want to use `phenix.multi_crystal_average` to generate phases for one crystal from another and you have more than one chain in the NCS asymmetric unit you would have to do this.

- If your NCS asymmetric unit has more than one domain (more than one type of chain chain, or else multiple domains within a chain that have different arrangements in different NCS asymmetric units) then `phenix.multi_crystal_average` requires that you provide map coefficients for all crystals. This is because `phenix.multi_crystal_average` cannot use the PDB files you provide to generate the NCS asymmetric unit directly at this point (i.e., it cannot use `pdb_domain` in `RESOLVE`.) Therefore if you don't provide map coefficients for one crystal then it does not have a way to individually identify the region occupied by each domain in the NCS asymmetric unit for that crystal. This isn't a problem if there are not multiple domains or types of chains in the NCS asymmetric unit because the automatic method for generation of the NCS asymmetric unit can be used.

If your NCS asymmetric unit has more than one domain (more than one type of chain chain, or else multiple domains within a chain that have different arrangements in different NCS asymmetric units) then `phenix.multi_crystal_average` requires that you provide map coefficients for all crystals. This is because `phenix.multi_crystal_average` cannot use the PDB files you provide to generate the NCS asymmetric unit directly at this point (i.e., it cannot use `pdb_domain` in `RESOLVE`.) Therefore if you don't provide map coefficients for one crystal then it does not have a way to individually identify the region occupied by each domain in the NCS asymmetric unit for that crystal. This isn't a problem if there are not multiple domains or types of chains in the NCS asymmetric unit because the automatic method for generation of the NCS asymmetric unit can be used.

- Normally you should supply PDB files defining the NCS in your crystals in which all the chains have identical sequences and conformations within each NCS copy. This is not absolutely required, however. If your PDB file contains chains that are not identical then NCS will be estimated from the chains you provide. It may be necessary to set the parameter:
`simple_ncs_from_pdb.maximize_size_of_groups=True` to get this to work if the chains have insertions, deletions, or sequence differences.

Normally you should supply PDB files defining the NCS in your crystals in which all the chains have identical sequences and conformations within each NCS copy. This is not absolutely required, however. If your PDB file contains chains that are not identical then NCS will be estimated from the chains you provide. It may be necessary to set the parameter:

```
simple_ncs_from_pdb.maximize_size_of_groups=True
```

to get this to work if the chains have insertions, deletions, or sequence differences.

- The size of the asymmetric unit in the SOLVE/RESOLVE portion of phenix.multi_crystal_average is limited by the memory in your computer and the binaries used. The Wizard is supplied with regular-size ("", size=6), giant ("_giant", size=12), huge ("_huge", size=18) and extra_huge ("_extra_huge", size=36). Larger-size versions can be obtained on request.

The size of the asymmetric unit in the SOLVE/RESOLVE portion of phenix.multi_crystal_average is limited by the memory in your computer and the binaries used. The Wizard is supplied with regular-size ("", size=6), giant ("_giant", size=12), huge ("_huge", size=18) and extra_huge ("_extra_huge", size=36). Larger-size versions can be obtained on request.

Literature

Rapid automatic NCS identification using heavy-atom substructures. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 58, 2213-5 (2002).

- Rapid automatic NCS identification using heavy-atom substructures. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 58, 2213-5 (2002).

Statistical density modification with non-crystallographic symmetry. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 58, 2082-6 (2002).

- Statistical density modification with non-crystallographic symmetry. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 58, 2082-6 (2002).

Maximum-likelihood density modification. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 56, 965-72 (2000).

- Maximum-likelihood density modification. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 56, 965-72 (2000).

Map-likelihood phasing. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 57, 1763-75 (2001).

- Map-likelihood phasing. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 57, 1763-75 (2001).

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
 crystalncs_domain_file = None PDB file for identifying region occupied by ncs copies. If specified, region defined by this file is used for ncs domains.
 pdb_file = None PDB files, one for each crystal (One pdb_file=xxx.pdb per crystal) These should be in the same order as datafiles and map files. They are used to identify the NCS within each crystal and between crystals. You should create these by placing the unique set of atoms (the NCS asymmetric unit) in each NCS asymmetric unit of each unit cell. Normally you would do this by carrying out molecular replacement on each crystal with the same search model.
 map_coeffs = None Mtz files with map coefficients. (One map_coeffs=xxx.mtz per crystal). At least one crystal must have map coefficients. Use "None" with quotes for any crystals that do not have starting maps. NOTE: If you have multiple NCS groups then you need map coefficients for all crystals.
 map_coeffs_labin = None Optional labin lines for mtz files with map coefficients. (One map_coeffs_labin=my_labin per crystal, or none at all) They look like map_coeffs_labin="FP=FP PHIB=PHIM FOM=FOMM" Put each set of labin values inside single quotes, and the whole list inside double quotes. You can leave out a labin statement for a file by putting in "None" and the routine will guess the column labels.
 datafile = None Mtz files with structure factors and optional phases and FOM and optional HL coefficients. One datafile for each crystal to be included (One

datafile=xxx.mtz for each crystal). datafile_labin = None Optional labin line for mtz file (In same order as mtz file). It can contain FP SIGFP [PHIB FOM] [HLA HLB HLC HLD]. It looks like this: datafile_labin='FP=FP SIGFP=SIGFP PHIB=PHIM FOM=FOMM' You can leave out a labin statement for a file by putting in "None" and the routine will guess the column labels NOTE: If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination. solvent_content = None Solvent content (0 to 1, typically 0.5) for each crystal (one solvent_content=xxx for each crystal). ha_file = None Optional file, normally containing heavy atom sites or other coordinates that are present in this structure, but not in the others. If supplied, the density near (within 2 Å) of atoms in this file will not be transferred to other crystals. perfect_map_coeffs = None Optional mtz files with perfect map coefficients for comparison. perfect_map_coeffs_labin = None Optional labin lines for mtz files with perfect map coefficients. For comparison and checking only. Not normally used. update_map = True Update the map for this crystal on each cycle. Set to False if this crystal represents a high-resolution model or is otherwise expected to be already as good as it can be. averaging_cycles = 5 Number of cycles of density modification resolution = None high-resolution limit for map calculation Default is use all data fill = True Fill in all missing reflections to resolution res_fill.res_fill = None Resolution for filling in missing data (default = highest resolution of any datafile) Default is use all data temp_dir = "temp_dir" Optional temporary work directory output_dir = "" Output directory where files are to be written write_target_map_and_sigma = False Write target map and sigma use_model_mask = False You can use the ncs_domain_pdb PDB files you input to define the solvent boundary if you wish. These will partially define the NCS asymmetric unit (by limiting it to the non-solvent region) but the exact NCS asymmetric unit will always be defined automatically (by the overlap of NCS-related density). Note that this is different than the command write_ncs_domain_pdb which defines individual regi...

- job_title = None Job title in PHENIX GUI, not used on command line

- crystal_ncs_domain_file = None PDB file for identifying region occupied by ncs copies. If specified, region defined by this file is used for ncs domains. pdb_file = None PDB files, one for each crystal (One pdb_file=xxx.pdb per crystal) These should be in the same order as datafiles and map files. They are used to identify the NCS within each crystal and between crystals. You should create these by placing the unique set of atoms (the NCS asymmetric unit) in each NCS asymmetric unit of each unit cell. Normally you would do this by carrying out molecular replacement on each crystal with the same search model. map_coeffs = None Mtz files with map coefficients. (One map_coeffs=xxx.mtz per crystal). At least one crystal must have map coefficients. Use "None" with quotes for any crystals that do not have starting maps. NOTE: If you have multiple NCS groups then you need map coefficients for all crystals. map_coeffs_labin = None Optional labin lines for mtz files with map coefficients. (One map_coeffs_labin=my_labin per crystal, or none at all) They look like map_coeffs_labin=" 'FP=FP PHIB=PHIM FOM=FOMM'" Put each set of labin values inside single quotes, and the whole list inside double quotes. You can leave out a labin statement for a file by putting in "None" and the routine will guess the column labels datafile = None Mtz files with structure factors and optional phases and FOM and optional HL coefficients. One datafile for each crystal to be included (One datafile=xxx.mtz for each crystal). datafile_labin = None Optional labin line for mtz file (In same order as mtz file). It can contain FP SIGFP [PHIB FOM] [HLA HLB HLC HLD]. It looks like this: datafile_labin='FP=FP SIGFP=SIGFP PHIB=PHIM FOM=FOMM' You can leave out a labin statement for a file by putting in "None" and the routine will guess the column labels NOTE: If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination. solvent_content = None Solvent content (0 to 1, typically 0.5) for each crystal (one solvent_content=xxx for each crystal). ha_file = None Optional file, normally containing heavy atom sites or other coordinates that are present in this structure, but not in the others. If supplied, the density near (within 2 Å) of atoms in

this file will not be transferred to other crystals. perfect_map_coeffs = None Optional mtz files with perfect map coefficients for comparison. perfect_map_coeffs_labin = None Optional labin lines for mtz files with perfect map coefficients. For comparison and checking only. Not normally used. update_map = True Update the map for this crystal on each cycle. Set to False if this crystal represents a high-resolution model or is otherwise expected to be already as good as it can be.

- ncs_domain_file = None PDB file for identifying region occupied by ncs copies. If specified, region defined by this file is used for ncs domains.

- pdb_file = None PDB files, one for each crystal (One pdb_file=xxx.pdb per crystal) These should be in the same order as datafiles and map files. They are used to identify the NCS within each crystal and between crystals. You should create these by placing the unique set of atoms (the NCS asymmetric unit) in each NCS asymmetric unit of each unit cell. Normally you would do this by carrying out molecular replacement on each crystal with the same search model.

- map_coeffs = None Mtz files with map coefficients. (One map_coeffs=xxx.mtz per crystal). At least one crystal must have map coefficients. Use "None" with quotes for any crystals that do not have starting maps. NOTE: If you have multiple NCS groups then you need map coefficients for all crystals.

- map_coeffs_labin = None Optional labin lines for mtz files with map coefficients. (One map_coeffs_labin=my_labin per crystal, or none at all) They look like map_coeffs_labin="FP=FP PHIB=PHIM FOM=FOMM" Put each set of labin values inside single quotes, and the whole list inside double quotes. You can leave out a labin statement for a file by putting in "None" and the routine will guess the column labels

- datafile = None Mtz files with structure factors and optional phases and FOM and optional HL coefficients. One datafile for each crystal to be included (One datafile=xxx.mtz for each crystal).

- datafile_labin = None Optional labin line for mtz file (In same order as mtz file). It can contain FP SIGFP [PHIB FOM] [HLA HLB HLC HLD]. It looks like this: datafile_labin='FP=FP SIGFP=SIGFP PHIB=PHIM FOM=FOMM' You can leave out a labin statement for a file by putting in "None" and the routine will guess the column labels NOTE: If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination.

- solvent_content = None Solvent content (0 to 1, typically 0.5) for each crystal (one solvent_content=xxx for each crystal).

- ha_file = None Optional file, normally containing heavy atom sites or other coordinates that are present in this structure, but not in the others. If supplied, the density near (within 2 Å) of atoms in this file will not be transferred to other crystals.

- perfect_map_coeffs = None Optional mtz files with perfect map coefficients for comparison.

- perfect_map_coeffs_labin = None Optional labin lines for mtz files with perfect map coefficients. For comparison and checking only. Not normally used.

- update_map = True Update the map for this crystal on each cycle. Set to False if this crystal represents a high-resolution model or is otherwise expected to be already as good as it can be.

- averagingcycles = 5 Number of cycles of density modification resolution = None high-resolution limit for map calculation Default is use all datafill = True Fill in all missing reflections to resolution res_fill.res_fill = None Resolution for filling in missing data (default = highest resolution of any datafile) Default is use all datatemp_dir = "temp_dir" Optional temporary work directoryoutput_dir = "" Output directory where files are to be writtenwrite_target_map_and_sigma = False Write target map and sigmause_model_mask = False You can use the ncs_domain_pdb PDB files you input to define the solvent boundary if you wish. These will partially define the NCS asymmetric unit (by limiting it to the non-solvent region) but the exact

NCS asymmetric unit will always be defined automatically (by the overlap of NCS-related density). Note that this is different than the command `write_ncs_domain_pdb` which defines individual regions where NCS applies for each domain. `sharpen = False` You can sharpen the maps or not in the density-modification process. (They are unsharpened at the end of the process if so). Not normally used, as an anisotropy correction with sharpening is normally applied to all the data. `equal_ncs_weight = False` You can fix the NCS weighting to equally weight all copies. `weight_ncs = None` You can set the weighting on NCS symmetry (and cross-crystal averaging) `write_ncs_domain_pdb = False` You can use the input PDB files to define NCS boundaries. The atoms in the PDB files will be grouped into domains during the analysis of NCS and written out to domain-specific PDB files. (If there is only one domain or NCS group then there will be only one domain-specific PDB file and it will be the same as the starting PDB file.) Then the domain-specific PDB files will be used to define the regions over which the corresponding NCS operators apply. Note that this is different than the command `use_model_mask` which only defines the overall solvent boundary with your model. `mask_cycles = 1` Number of mask cycles in each cycle of density modification `map_phasing_ab = False` Use map phasing (A,B instead of r,phi) `use_FP_PHIM = False` Use FP and PHIM as labels for all intermediate map coeffs. Used to get around problem with labels when they vary. `no_recombination = False` Do not recombine with original data. Use just map phases and amplitudes. `output_filetarget_output_format = *None` `pdb` `mmCIF` Desired output format (if possible). Choices are `None` (try to use input format), `pdb`, `mmCIF`. If output model does not fit in `pdb` format, `mmCIF` will be used. Default is `pdb`.

- `cycles = 5` Number of cycles of density modification
- `resolution = None` high-resolution limit for map calculation Default is use all data
- `fill = True` Fill in all missing reflections to resolution `res_fill`.
- `res_fill = None` Resolution for filling in missing data (default = highest resolution of any datafile) Default is use all data
- `temp_dir = "temp_dir"` Optional temporary work directory
- `output_dir = ""` Output directory where files are to be written
- `write_target_map_and_sigma = False` Write target map and sigma
- `use_model_mask = False` You can use the `ncs_domain_pdb` PDB files you input to define the solvent boundary if you wish. These will partially define the NCS asymmetric unit (by limiting it to the non-solvent region) but the exact NCS asymmetric unit will always be defined automatically (by the overlap of NCS-related density). Note that this is different than the command `write_ncs_domain_pdb` which defines individual regions where NCS applies for each domain.
- `sharpen = False` You can sharpen the maps or not in the density-modification process. (They are unsharpened at the end of the process if so). Not normally used, as an anisotropy correction with sharpening is normally applied to all the data.
- `equal_ncs_weight = False` You can fix the NCS weighting to equally weight all copies.
- `weight_ncs = None` You can set the weighting on NCS symmetry (and cross-crystal averaging)
- `write_ncs_domain_pdb = False` You can use the input PDB files to define NCS boundaries. The atoms in the PDB files will be grouped into domains during the analysis of NCS and written out to domain-specific PDB files. (If there is only one domain or NCS group then there will be only one domain-specific PDB file and it will be the same as the starting PDB file.) Then the domain-specific PDB files will be used to define the regions over which the corresponding NCS operators apply. Note that this is different than the command `use_model_mask` which only defines the overall solvent boundary with your model.

- `mask_cycles = 1` Number of mask cycles in each cycle of density modification
- `map_phasing_ab = False` Use map phasing (A,B instead of r,phi)
- `use_FP_PHIM = False` Use FP and PHIM as labels for all intermediate map coeffs. Used to get around problem with labels when they vary.
- `no_recombination = False` Do not recombine with original data. Use just map phases and amplitudes.
- `output_filetarget_output_format = *None` pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- `target_output_format = *None` pdb mmCIF Desired output format (if possible). Choices are None (try to use input format), pdb, mmCIF. If output model does not fit in pdb format, mmCIF will be used. Default is pdb.
- `anisoremove_aniso = True` Remove anisotropy from data files before use Note: map files are assumed to be already corrected
`b_iso = None` Target overall B value for anisotropy correction. Ignored if `remove_aniso = False`. If None, default is minimum of (`max_b_iso`, lowest B of datasets, `target_b_ratio*resolution`)
`max_b_iso = 40`. Default maximum overall B value for anisotropy correction. Ignored if `remove_aniso = False`. Ignored if `b_iso` is set. If used, default is minimum of (`max_b_iso`, lowest B of datasets, `target_b_ratio*resolution`)
`target_b_ratio = 10`. Default ratio of target B value to resolution for anisotropy correction. Ignored if `remove_aniso = False`. Ignored if `b_iso` is set. If used, default is minimum of (`max_b_iso`, lowest B of datasets, `target_b_ratio*resolution`)
- `remove_aniso = True` Remove anisotropy from data files before use Note: map files are assumed to be already corrected
- `b_iso = None` Target overall B value for anisotropy correction. Ignored if `remove_aniso = False`. If None, default is minimum of (`max_b_iso`, lowest B of datasets, `target_b_ratio*resolution`)
- `max_b_iso = 40`. Default maximum overall B value for anisotropy correction. Ignored if `remove_aniso = False`. Ignored if `b_iso` is set. If used, default is minimum of (`max_b_iso`, lowest B of datasets, `target_b_ratio*resolution`)
- `target_b_ratio = 10`. Default ratio of target B value to resolution for anisotropy correction. Ignored if `remove_aniso = False`. Ignored if `b_iso` is set. If used, default is minimum of (`max_b_iso`, lowest B of datasets, `target_b_ratio*resolution`)
- `controlverbose = True` verbose output
`debug = False` debugging output
`raise_sorry = False` Raise sorry if problems
`sn_xyz = None` gridding for maps
`coarse_grid = False` You can set `coarse_grid` in `resolve`
`resolve_size = 12` Size for solve/resolve ('giant', 'huge', 'extra_huge', or a number where 12=giant, 18=huge)
`resolve_command_list = None` Commands for resolve. One per line in the form: keyword value value can be optional Examples: `coarse_grid resolution 200 2.0 hklIn test.mtz` NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: `resolve_command_list="no_build 'b_overall 23' "`
`dry_run = False` Just read in and check parameter names
`base_gui_dir = None` GUI parameter only
- `verbose = True` verbose output
- `debug = False` debugging output
- `raise_sorry = False` Raise sorry if problems
- `sn_xyz = None` gridding for maps
- `coarse_grid = False` You can set `coarse_grid` in `resolve`

- `resolve_size = 12` Size for solve/resolve ("", "_giant", "_huge", "_extra_huge" or a number where 12=giant 18=huge
- `resolve_command_list = None` Commands for resolve. One per line in the form: keyword value value can be optional Examples: `coarse_grid resolution 200 2.0 hkl in test.mtz` NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (",) and each individual command in single quotes (') like this: `resolve_command_list="'no_build' 'b_overall 23' "`;
- `dry_run = False` Just read in and check parameter names
- `base_gui_dir = None` GUI parameter only
- `simple_ncs_from_pdbpdb_in = None` Input PDB file to be used to identify `ncstemp_dir = ""` temporary directory (`ncs_domain_pdb` will be written there)
`ncs_domain_pdb_stem = None` NCS domains will be written to `ncs_domain_pdb_stem+"group_"+nn`
`write_ncs_domain_pdb = False` You can write out PDB files representing NCS domains for density modification if you want
`write_ncs_phil = False` Write out phil parameters for the NCS groups to file in working directory
`show_summary = False` Show NCS information summary
`write_spec_files = False` Write .ncs_spec and .resolve representation of NCS groups
`ncs_search` Set of parameters for NCS search procedure. Some of them also used for filtering user-supplied `ncs_group`.
`enabled = False` Enable NCS restraints or constraints in refinement (in some cases may be switched on inside refinement program).
`exclude_selection = "element H or element D or water"` Atoms selected by this selection will be excluded from the model before any NCS search and/or filtering procedures. There is no way atoms defined by this selection will be in
`ncs.chain_similarity_threshold = 0.85` Threshold for sequence similarity between matching chains. A smaller value may cause more chains to be grouped together and can lower the number of common residues
`chain_max_rmsd = 2.` limit of rms difference between chains to be considered as copies
`residue_match_radius = 4.0` Maximum allowed distance difference between pairs of matching atoms of two residues
`try_shortcuts = False` Try very quick check to speed up the search when chains are identical. If failed, regular search will be performed automatically.
`minimum_number_of_atoms_in_copy = 3` Do not create ncs groups where master and copies would contain less than specified amount of atoms
`validate_user_supplied_groups = True` Enable validation of user-supplied `ncs_group`. Need to exercise a lot of caution turning this off. This option is for developers only.
`ncs_group` The definition of one NCS group. Note, that almost always in refinement programs they will be checked and filtered if needed.
`reference = None` Residue selection string for the complete master NCS copy
`selection = None` Residue selection string for each NCS copy location in ASU
- `pdb_in = None` Input PDB file to be used to identify ncs
- `temp_dir = ""` temporary directory (`ncs_domain_pdb` will be written there)
- `ncs_domain_pdb_stem = None` NCS domains will be written to `ncs_domain_pdb_stem+"group_"+nn`
- `write_ncs_domain_pdb = False` You can write out PDB files representing NCS domains for density modification if you want
- `write_ncs_phil = False` Write out phil parameters for the NCS groups to file in working directory
- `show_summary = False` Show NCS information summary
- `write_spec_files = False` Write .ncs_spec and .resolve representation of NCS groups
- `ncs_search` Set of parameters for NCS search procedure. Some of them also used for filtering user-supplied `ncs_group`.
`enabled = False` Enable NCS restraints or constraints in refinement (in some cases may be switched on inside refinement program).
`exclude_selection = "element H or element D or water"` Atoms selected by this selection will be excluded from the model before any NCS search and/or filtering procedures. There is no way atoms defined by this selection will be in
`ncs.chain_similarity_threshold = 0.85` Threshold for sequence similarity between matching chains. A

smaller value may cause more chains to be grouped together and can lower the number of common residues
`chain_max_rmsd = 2`. limit of rms difference between chains to be considered as copies
`residue_match_radius = 4.0` Maximum allowed distance difference between pairs of matching atoms of two residues
`try_shortcuts = False` Try very quick check to speed up the search when chains are identical. If failed, regular search will be performed automatically.
`minimum_number_of_atoms_in_copy = 3` Do not create ncs groups where master and copies would contain less than specified amount of atoms
`validate_user_supplied_groups = True` Enable validation of user-supplied ncs_group. Need to exercise a lot of caution turning this off. This option is for developers only.

- `enabled = False` Enable NCS restraints or constraints in refinement (in some cases may be switched on inside refinement program).

- `exclude_selection = "element H or element D or water"` Atoms selected by this selection will be excluded from the model before any NCS search and/or filtering procedures. There is no way atoms defined by this selection will be in NCS.

- `chain_similarity_threshold = 0.85` Threshold for sequence similarity between matching chains. A smaller value may cause more chains to be grouped together and can lower the number of common residues

- `chain_max_rmsd = 2`. limit of rms difference between chains to be considered as copies

- `residue_match_radius = 4.0` Maximum allowed distance difference between pairs of matching atoms of two residues

- `try_shortcuts = False` Try very quick check to speed up the search when chains are identical. If failed, regular search will be performed automatically.

- `minimum_number_of_atoms_in_copy = 3` Do not create ncs groups where master and copies would contain less than specified amount of atoms

- `validate_user_supplied_groups = True` Enable validation of user-supplied ncs_group. Need to exercise a lot of caution turning this off. This option is for developers only.

- `ncs_group` The definition of one NCS group. Note, that almost always in refinement programs they will be checked and filtered if needed.
`reference = None` Residue selection string for the complete master NCS copy
`selection = None` Residue selection string for each NCS copy location in ASU

- `reference = None` Residue selection string for the complete master NCS copy

- `selection = None` Residue selection string for each NCS copy location in ASU

Building biological assembly and crystallographic asymmetric unit from a PDB file

multimer_reconstruction.html

Introduction

In PDB files, not all the atoms locations of a biological assembly and the crystallographic asymmetric unit are given. The transformations information, rotational and translational (for both crystallographic and non-crystallographic), needed to reconstruct the multimer, is given in REMARK 350 and MTRIX line of the pdb file.

Phenix provide a commands to create a pdb file of the reconstructed a multimer for both the biological assembly and the crystallographic asymmetric unit.

The bioassembly_reconstruction command function applies all the transformations in the the pdb file REMARK 350 BIOMT, to each of the chains, produces new chains with new coordinates and adds them to the current model. The crystall_asym_unit_reconstruction does the same, using the MTRIX data.

Command-line use

Biological Assembly

Usage: phenix.pdb.biomt_reconstruction [pdf_file] [pdf_file] [options]

Options:
--help, -h

Example:

```
>>> phenix.pdb.biomt_reconstruction 'input_file_name.pdb'
bio-assembly reconstructed pdb file was added to your current directory
/path/biological_assembly_input_input_file_name.pdb
```

```
>>> phenix.pdb.biomt_reconstruction 'input_file_name.pdb' 'output_file_name.pdb'
bio-assembly reconstructed pdb file was added to your current directory
/path/output_file_name.pdb
```

Crystallographic Asymmetric Unit

Usage: phenix.pdb.mtrix_reconstruction [pdf_file] [pdf_file] [options]

Options:
--help, -h

Example:

```
>>> phenix.pdb.mtrix_reconstruction 'input_file_name.pdb'
bio-assembly reconstructed pdb file was added to your current directory
/path/crystall_asymmetric_unit_input_file_name.pdb
```

```
>>> phenix.pdb.mtrix_reconstruction 'input_file_name.pdb' 'output_file_name.pdb'
bio-assembly reconstructed pdb file was added to your current directory
/path/output_file_name.pdb
```

Output

For both commands the output is a pdb file in the current working directory

Calculating a simple NCS averaged map

[ncs_average.html](#)

If you already have map coefficients in an MTZ file and have identified your NCS with `phenix.find_ncs` and just want to calculate the average density in all your NCS copies, you can use `phenix.ncs_average`. In most cases this can be run automatically by passing the name of the MTZ file and the `ncs_spec` file with NCS information.

The output of `phenix.ncs_average` is a CCP4-style map file.

These simple NCS averaged maps can be accurate maps for cases with high NCS symmetry and they are useful for evaluating what an NCS density modified map should be expected to look like.

If you want to see the region that is being averaged, you can run `phenix.ncs_average` like this (notice the double and single quotes, both are necessary):

```
phenix.ncs_average ncs_map_coeffs.mtz ncs.ncs_spec resolve_command_list=" 'ncs_mask_file  
ncs_mask.ccp4' "
```

If you want to specify the region to be averaged, you can do this by supplying a PDB file and the mask will be around atoms in that file. Note: the mask you supply is an overall mask around ALL the areas to be averaged, not just a single asymmetric unit of NCS.

```
phenix.ncs_average ncs_map_coeffs.mtz ncs.ncs_spec resolve_command_list=" 'ncs_mask_file  
ncs_mask.ccp4' 'model model_defining_mask.pdb' 'use_model_mask' 'rad_mask 5' "
```

List of all available keywords

`input_filesmtz_in` = None Input MTZ file with map coefficients
`map_coeff_labels` = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: `map_coeff_labels` (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM)
`map_file` = None Optional input CCP4-style map file. No `mtz_in` is needed if a map file is supplied
`ncs_in` = None File with NCS operators to be applied to `ncs_in`. This can be a `.ncs_spec` file from `find_ncs`, `simple_ncs_from_pdb` or `find_ncs_from_density`. It can also be NCS formatted for `resolve`, including rotations, translations, and centers. Note: you can alternatively supply a PDB file with `pdb_in`
`pdb_in` = None Optional PDB file containing NCS to apply
`output_filesccp4_map_out` = `ncs_average.ccp4` Output CCP4-style map file with NCS-averaged density
`ncs_mask_file` = `ncs_mask.ccp4` Output CCP4-style map file with NCS-mask
`map_coeffs_out` = `ncs_average.mtz` Output MTZ file with map coefficients representing NCS-averaged density. Note: only applicable if input map is represented as map coefficients information with `mtz_in`. not applicable if `map_in` is used.
`log` = `ncs_average.log` Output log file
`params_out` = `ncs_average_params.eff` Parameters file to rerun `ncs_averaging`
`directories`
`temp_dir` = "temp_dir" Optional temporary work directory
`output_dir` = "" Output directory where files are to be written
`crystal_info`
`high_resolution` = None high-resolution limit for map calculations
`solvent_content` = 0.5 solvent fraction
`wang_radius` = 5 Smoothing radius for identification of solvent boundary
`use_pdb_in_as_mask` = False Use `pdb_in` file as mask for NCS (if supplied)
`rad_mask` = 4 mask radius around atoms in `pdb_in`
`high_resolution_scale` = 0.95 Resolution of output map coeffs will be `high_resolution_scale`* resolution of input map coeffs if `high_resolution` is None
`fraction_ncs_min` = 0.05 If the fraction of the asymmetric unit occupied by the NCS related molecules is less than `fraction_ncs_min` it is ignored
`refine_ncs_operators` = False Refine NCS operators
`control`
`verbose` = False Verbose output
`raise_sorry` = False Raise sorry if problems
`debug` = False Debugging output
`dry_run` = False Just read in and check parameter

namescoarse_grid = False Use coarse grid (saves on memory)resolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this:
 resolve_command_list="'no_build' 'b_overall 23' ";

- input_filesmtz_in = None Input MTZ file with map coefficientsmap_coeff_labels = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: map_coeff_labels (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM)map_file = None Optional input CCP4-style map file. No mtz_in is needed if a map file is suppliedncs_in = None File with NCS operators to be applied to ncs_in This can be a .ncs_spec file from find_ncs, simple_ncs_from_pdb or find_ncs_from_density It can also be NCS formatted for resolve, including rotations, translations, and centers Note: you can alternatively supply a PDB file with pdb_inpdb_in = None Optional PDB file containing NCS to apply
- mtz_in = None Input MTZ file with map coefficients
- map_coeff_labels = None If map coefficients cannot be identified automatically from your MTZ file, you can specify the label or labels for them. (Please separate labels with blank space, MTZ columns grouped together separated by commas with no blanks.) You can specify: map_coeff_labels (e.g., FWT,PHIFWT) amplitudes and phases (e.g., FP,SIGFP PHIB) or amplitudes, phases, weights (e.g., FP,SIGFP PHIB FOM)
- map_file = None Optional input CCP4-style map file. No mtz_in is needed if a map file is supplied
- ncs_in = None File with NCS operators to be applied to ncs_in This can be a .ncs_spec file from find_ncs, simple_ncs_from_pdb or find_ncs_from_density It can also be NCS formatted for resolve, including rotations, translations, and centers Note: you can alternatively supply a PDB file with pdb_in
- pdb_in = None Optional PDB file containing NCS to apply
- output_filesccp4_map_out = ncs_average.ccp4 Output CCP4-style map file with NCS-averaged densityncs_mask_file = ncs_mask.ccp4 Output CCP4-style map file with NCS-maskmap_coeffs_out = ncs_average.mtz Output MTZ file with map coefficients representing NCS-averaged density. Note: only applicable if input map is represented as map coefficients information with mtz_in. not applicable if map_in is used.log = ncs_average.log Output log fileparams_out = ncs_average_params.eff Parameters file to rerun ncs averaging
- ccp4_map_out = ncs_average.ccp4 Output CCP4-style map file with NCS-averaged density
- ncs_mask_file = ncs_mask.ccp4 Output CCP4-style map file with NCS-mask
- map_coeffs_out = ncs_average.mtz Output MTZ file with map coefficients representing NCS-averaged density. Note: only applicable if input map is represented as map coefficients information with mtz_in. not applicable if map_in is used.
- log = ncs_average.log Output log file
- params_out = ncs_average_params.eff Parameters file to rerun ncs averaging
- directoriestemp_dir = "temp_dir" Optional temporary work directory output_dir = "" Output directory where files are to be written
- temp_dir = "temp_dir" Optional temporary work directory
- output_dir = "" Output directory where files are to be written

- `crystal_info` `high_resolution = None` high-resolution limit for map calculation `solvent_content = 0.5` solvent fraction `wang_radius = 5` Smoothing radius for identification of solvent boundary `use_pdb_in_as_mask = False` Use `pdb_in` file as mask for NCS (if supplied) `rad_mask = 4` mask radius around atoms in `pdb_in`. `high_resolution_scale = 0.95` Resolution of output map coeffs will be `high_resolution_scale`* resolution of input map coeffs if `high_resolution` is `None` `fraction_ncs_min = 0.05` If the fraction of the asymmetric unit occupied by the NCS related molecules is less than `fraction_ncs_min` it is ignored `refine_ncs_operators = False` Refine NCS operators
- `high_resolution = None` high-resolution limit for map calculation
- `solvent_content = 0.5` solvent fraction
- `wang_radius = 5` Smoothing radius for identification of solvent boundary
- `use_pdb_in_as_mask = False` Use `pdb_in` file as mask for NCS (if supplied)
- `rad_mask = 4` mask radius around atoms in `pdb_in`.
- `high_resolution_scale = 0.95` Resolution of output map coeffs will be `high_resolution_scale`* resolution of input map coeffs if `high_resolution` is `None`
- `fraction_ncs_min = 0.05` If the fraction of the asymmetric unit occupied by the NCS related molecules is less than `fraction_ncs_min` it is ignored
- `refine_ncs_operators = False` Refine NCS operators
- `control` `verbose = False` Verbose output `raise_sorry = False` Raise sorry if problems `debug = False` Debugging output `dry_run = False` Just read in and check parameter names `coarse_grid = False` Use coarse grid (saves on memory) `resolve_command_list = None` You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this:
`resolve_command_list="'no_build' 'b_overall 23' "`
- `verbose = False` Verbose output
- `raise_sorry = False` Raise sorry if problems
- `debug = False` Debugging output
- `dry_run = False` Just read in and check parameter names
- `coarse_grid = False` Use coarse grid (saves on memory)
- `resolve_command_list = None` You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: `resolve_command_list="'no_build' 'b_overall 23' "`

Model building with Pathwalker

pathwalker.html

Author(s)

- pathwalker: Matthew Baker

Purpose

Build a model with path walking

How pathwalker works:

Examples

Standard run of pathwalker:

Running pathwalker is easy. From the command-line you can type:

```
phenix.pathwalker my_map.mrc resolution=3 threshold=0.2 nres=150
```

This will build a model based on the map my_map.mrc at a resolution of 3 Å using a threshold of 0.2 and looking for 150 CA atoms

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
input_filesmap_file = None File with CCP4-style map. May have origin in any location.
default_paramsthreshold = None Threshold used for populating pseudatoms.
nres = None Number of pseudoatoms to populate the density with
strategyseq_file = None Sequence file
pa_file = None Optional input pseudoatoms to be used as starting point for pathwalker to be run. This model should be in the same reference frame as the input map unless starting_model_is_from_box_map_au is set.
start_pt = 0 If a pa_file is used, does the user want to specify a start PA. Must be used in conjunction with end_pt.
end_pt = 0 If a pa_file is used, does the user want to specify a end PA. Must be used in conjunction with start_pt.
verbose = False Verbose turns on path evaluation.
tsp = *ortools LKH TSP solver to be used.
pa_type = *kmeans sc ac ms gmm birch PA type used for populating pseudatoms. (kmeans is default)
noise = 0 Add noise to atom positions (float)
map_weight = False Weighting distances by map value
filter = False Apply a filter to the map. Default is 4.5Å, if turned on. This may help with PA placement
filter_resolution = 4.5 Resolution to apply a filter. The default is 4.5Å if turned on.
or_time = 30 Time (in second) to allow OR tools to refine pathwalker trace Default is 30

secondslocal_search_strategy = *GUIDED_LOCAL_SEARCH AUTOMATIC GREEDY_DESCENT
SIMULATED_ANNEALING TABU_SEARCH OBJECTIVE_TABU_SEARCH Local search strategies (also
called metaheuristics) for the OR-Tools TSP solver.prob_model = False compute a probabilistic model based
on multiple pathwalker paths.all_atom = False Convert the final pathwalker model into an all-atom model
using Pulchra.reverse = False Generate the all-atom reverse path with Pulchra.refine_resolution = 0 Add a
map resolution value to run real-space refinementrefine_cycles = 5 The number of real space refinement
cycles that are to be run.bracket = [] This handles the options for iterative PA generation and tsp solver. Three
values are required: Threshold minimum Threshold maximum Threshold step They should be entered like
bracket=0.22,0.24,0.01tsp_runs = [0] Number of TSP runs and the amount of noise to apply to each run. for
example: tsp_runs=0,0,0,1,0 will actually create 5 models, where runs 1-3 and 5 have no additive noise,
while run 4 has noise equivalent to 1 sigma added to the original PAaverage_model = False if multiple TSP
runs are set, an average model can be generated with this flagoutput_files pw_out_pdb = pw-threshold.pdb
Search PDB file, transformed to superpose on the target map file.pts_out_pdb = pts-threshold.pdb Search
PDB file, transformed to superpose on the target map file.pts_out = pts.txt Pseudoatom points generated
(coordinates)guiGUI-specific parameter required for output directoryoutput_dir = None

- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesmap_file = None File with CCP4-style map. May have origin in any location.
- map_file = None File with CCP4-style map. May have origin in any location.
- default_paramsthreshold = None Threshold used for populating pseudatoms.nres = None Number of
pseudoatoms to populate the density with
- threshold = None Threshold used for populating pseudatoms.
- nres = None Number of pseudoatoms to populate the density with
- strategyseq_file = None Sequence filepa_file = None Optional input pseudoatoms to be used as starting
point for pathwalker to be run. This model should be in the same reference frame as the input map unless
starting_model_is_from_box_map_au is set.start_pt = 0 If a pa_file is used, does the user want to specify
a start PA. Must be used in conjunction with end_pt.end_pt = 0 If a pa_file is used, does the user want to
specify a end PA. Must be used in conjunction with start_pt.verbose = False Verbose turns on path
evaluation.tsp = *ortools LKH TSP solver to be used.pa_type = *kmeans sc ac ms gmm birch PA type
used for populating pseudatoms. (kmeans is default)noise = 0 Add noise to atom positions
(float)map_weight = False Weighting distances by map valuefilter = False Apply a filter to the map.
Default is 4.5A, if turned on. This may help with PA placementfilter_resolution = 4.5 Resolution to apply a
filter. The default is 4.5A if turned on.or_time = 30 Time (in second) to allow OR tools to refine pathwalker
trace Default is 30 secondslocal_search_strategy = *GUIDED_LOCAL_SEARCH AUTOMATIC
GREEDY_DESCENT SIMULATED_ANNEALING TABU_SEARCH OBJECTIVE_TABU_SEARCH Local
search strategies (also called metaheuristics) for the OR-Tools TSP solver.prob_model = False compute
a probabilistic model based on multiple pathwalker paths.all_atom = False Convert the final pathwalker
model into an all-atom model using Pulchra.reverse = False Generate the all-atom reverse path with
Pulchra.refine_resolution = 0 Add a map resolution value to run real-space refinementrefine_cycles = 5
The number of real space refinement cycles that are to be run.bracket = [] This handles the options for
iterative PA generation and tsp solver. Three values are required: Threshold minimum Threshold
maximum Threshold step They should be entered like bracket=0.22,0.24,0.01tsp_runs = [0] Number of
TSP runs and the amount of noise to apply to each run. for example: tsp_runs=0,0,0,1,0 will actually
create 5 models, where runs 1-3 and 5 have no additive noise, while run 4 has noise equivalent to 1
sigma added to the original PAaverage_model = False if multiple TSP runs are set, an average model
can be generated with this flag
- seq_file = None Sequence file

- `pa_file = None` Optional input pseudoatoms to be used as starting point for pathwalker to be run. This model should be in the same reference frame as the input map unless `starting_model_is_from_box_map_au` is set.
- `start_pt = 0` If a `pa_file` is used, does the user want to specify a start PA. Must be used in conjunction with `end_pt`.
- `end_pt = 0` If a `pa_file` is used, does the user want to specify a end PA. Must be used in conjunction with `start_pt`.
- `verbose = False` Verbose turns on path evaluation.
- `tsp = *ortools` LKH TSP solver to be used.
- `pa_type = *kmeans` sc ac ms gmm birch PA type used for populating pseudatoms. (kmeans is default)
- `noise = 0` Add noise to atom positions (float)
- `map_weight = False` Weighting distances by map value
- `filter = False` Apply a filter to the map. Default is 4.5Å, if turned on. This may help with PA placement
- `filter_resolution = 4.5` Resolution to apply a filter. The default is 4.5Å if turned on.
- `or_time = 30` Time (in second) to allow OR tools to refine pathwalker trace Default is 30 seconds
- `local_search_strategy = *GUIDED_LOCAL_SEARCH` AUTOMATIC GREEDY_DESCENT SIMULATED_ANNEALING TABU_SEARCH OBJECTIVE_TABU_SEARCH Local search strategies (also called metaheuristics) for the OR-Tools TSP solver.
- `prob_model = False` compute a probabilistic model based on multiple pathwalker paths.
- `all_atom = False` Convert the final pathwalker model into an all-atom model using Pulchra.
- `reverse = False` Generate the all-atom reverse path with Pulchra
- `refine_resolution = 0` Add a map resolution value to run real-space refinement
- `refine_cycles = 5` The number of real space refinement cycles that are to be run.
- `bracket = []` This handles the options for iterative PA generation and tsp solver. Three values are required: Threshold minimum Threshold maximum Threshold step They should be entered like `bracket=0.22,0.24,0.01`
- `tsp_runs = [0]` Number of TSP runs and the amount of noise to apply to each run. for example: `tsp_runs=0,0,0,1,0` will actually create 5 models, where runs 1-3 and 5 have no additive noise, while run 4 has noise equivalent to 1 sigma added to the original PAs
- `average_model = False` if multiple TSP runs are set, an average model can be generated with this flag
- `output_files` `pw_out_pdb = pw-threshold.pdb` Search PDB file, transformed to superpose on the target map file. `pts_out_pdb = pts-threshold.pdb` Search PDB file, transformed to superpose on the target map file. `pts_out = pts.txt` Pseudoatom points generated (coordinates)
- `pw_out_pdb = pw-threshold.pdb` Search PDB file, transformed to superpose on the target map file.
- `pts_out_pdb = pts-threshold.pdb` Search PDB file, transformed to superpose on the target map file.
- `pts_out = pts.txt` Pseudoatom points generated (coordinates)
- `gui` GUI-specific parameter required for output directory `output_dir = None`
- `output_dir = None`

Rapid phase improvement and model-building using phase_and_build

phase_and_build.html

Contents

- Author(s)
- Purpose
- Usage
- How phase_and_build works:
- Output files from phase_and_build
- Parameters files in phase_and_build
- Examples
- Standard run of phase_and_build:
- Possible Problems
- Specific limitations and problems:
- Literature
- Additional information

Author(s)

- phase_and_build: Tom Terwilliger

Purpose

phase_and_build is a new and rapid method for improving the quality of your map and building a model. The approach is to carry out an iterative process of building a model as rapidly as possible and using this model in density modification to improve the map. This approach is related to the older phenix.autobuild approach. The difference is that in phenix.autobuild much effort was spent on building the best possible model at each stage before carrying out density modification, while in phenix.phase_and_build speed of model-building is optimized. The result is that phenix.phase_and_build is 10 times faster than phenix.autobuild, yet it produces nearly as good a model in the end. The phenix.phase_and_build approach will also find NCS from your starting map and apply it during density modification.

Usage

How phase_and_build works:

phase_and_build first identifies a free set of reflections if they are not supplied. A map without the test set reflections is created for real-space refinement. A set of data for density modification is created which contains anisotropy-corrected data (if supplied).

- Next phase_and_build estimates the solvent fraction (if not supplied) from the sequence file and cell parameters and the maximum and most likely number of NCS copies.

- A starting map is obtained by density modification of the input data (unless a map coefficients file is supplied).
- NCS is identified from the starting map (if the maximum number of NCS copies is greater than 1). If a heavy-atom file is supplied, NCS will first be identified from the heavy-atom sites, and the map will be used if the expected number of sites is not obtained.
- Next one or more cycles of phase improvement are carried out. In each cycle a model is built using `phenix.build_one_model`, the model is refined in real-space and/or in reciprocal space, and then the model is included in density modification to create a new map.
- Finally, a full model-building cycle is carried out using the most recent map. One or more models are built with `phenix.build_one_model` and are refined, then are combined, again with `phenix.build_one_model`. The resulting model is refined and sequence assignment and fitting of short loops is carried out with `phenix.assign_sequence`. Then longer loops are fit with `phenix.fit_loops` and the final model is refined and written out.

Output files from `phase_and_build`

`phase_and_build.pdb`: A PDB file with the resulting model
`phase_and_build_map_coeffs.mtz`: An MTZ file with optimized phases
`phase_and_build_refine_data.mtz`: An MTZ file with data and test set used in refinement

Parameters files in `phase_and_build`

When you run `phenix.phase_and_build` it will write out a `phase_and_build_params.eff` parameter file that can be used to re-run `phenix.phase_and_build` (just as for essentially all PHENIX methods). In addition, `phenix.phase_and_build` will write out the parameters files for the intermediate methods used as part of `phenix.phase_and_build` to the temporary directory used in building. You can run these with:

```
phenix.find_ncs temp_dir/find_ncs_params.eff # runs NCS identification
phenix.autobuild temp_dir/AutoBuild_run_1/autobuild.eff # runs first cycle of density modification
phenix.build_one_model temp_dir/build_one_model_params.eff # runs most recent model-building
phenix.assign_sequence temp_dir/assign_sequence_params.eff # runs sequence assignment and filling short gaps
phenix.fit_loops temp_dir/fit_loops_params.eff # runs loop fitting
```

This gives you control of all the steps in map improvement and model-building in addition to letting you run them all together with `phenix.phase_and_build`

Examples

Standard run of `phase_and_build`:

Running `phase_and_build` is easy. From the command-line you can type:

```
phenix.phase_and_build exptl_fobs_phases_freeR_flags.mtz sequence.dat
```

If you want to supply a file with anisotropy-corrected data to use in density modification you can do so:

```
phenix.phase_and_build data=exptl_fobs_phases_freeR_flags.mtz \
seq_file=sequence.dat \
aniso_corrected_data=solve_1.mtz
```

where `solve_1.mtz` is anisotropy-corrected (the amplitudes are not measured amplitudes, but rather are corrected with an anisotropic B-factor), and `exptl_fobs_phases_freeR_flags.mtz` contains experimental amplitudes. These two files normally will contain the same phase information. (Usually these files will come

from phenix.autosol.)

You can also add a starting model or a starting map to phenix.phase_and_build. This means that you can run it once, get a new model and map, then run it again to further improve your model and map.

Possible Problems

Specific limitations and problems:

phenix.phase_and_build does not have the full flexibility of phenix.autobuild, so you may want to get a nearly-complete model with phenix.phase_and_build and then use phenix.autobuild to increase the completeness and quality.

Literature

Additional information

<string>

phenix_maps.html

phenix.maps: a command line tool to compute various maps and save them in most of known formats.

How to run the command line version:

Run phenix.maps without any arguments: just type phenix.maps in the command line and hit Enter. This will create a parameter file called maps.params, which can be renamed if desired. Edit maps.params file to specify input/output file names, data labels and the desired maps. It is possible to request as many maps as desired. By default, the file maps.params specifies 5 maps to be created: 2mFo-DFc, 2mFo-DFc with missing Fobs filled with DFcalc, mFo-DFc and anomalous difference maps will be output in MTZ format, and one 2mFo-DFc map will be output in CCP4 format. NOTE: the anomalous difference map will only be created if the input reflection data file contains Bijvoet maps (F+/F- or I+/I-). Run this command to compute requested maps: phenix.maps maps.params

- Run phenix.maps without any arguments: just type phenix.maps in the command line and hit Enter. This will create a parameter file called maps.params, which can be renamed if desired.
- Edit maps.params file to specify input/output file names, data labels and the desired maps. It is possible to request as many maps as desired. By default, the file maps.params specifies 5 maps to be created: 2mFo-DFc, 2mFo-DFc with missing Fobs filled with DFcalc, mFo-DFc and anomalous difference maps will be output in MTZ format, and one 2mFo-DFc map will be output in CCP4 format. NOTE: the anomalous difference map will only be created if the input reflection data file contains Bijvoet maps (F+/F- or I+/I-).
- Run this command to compute requested maps: phenix.maps maps.params

Alternately, you may specify input files (and additional parameters) directly on the command line:

```
% phenix.maps model.pdb data.mtz
```

and it will automatically generate the default maps as described above.

Important Facts:

phenix.maps is available in PHENIX GUI. The scope of parameters 'map_coefficients' defines the map that will be output as Fourier map coefficients. The scope of parameters 'map' defines the maps that will be output as CCP4 or X-plor format. To create several maps: duplicate either 'map_coefficients' or 'map' or both scopes of parameters as many times as many maps is desired. Then edit each of them to define the maps. A map is defined by specifying a map type using 'map_type' keyword available within each scope of parameters: 'map_coefficients' or 'map'. The general supported format for 'map_type' is: [p][m]Fo+[q][D]Fc[_filled]. For example: 2Fo-Fc, 2mFobs-DFcalc, 3Fobs-2Fmodel, Fo-Fc, mFobs-DFcalc, anom, llg. The 'map_type' parser will automatically recognize which map is requested. The program creates as many files with CCP4 or X-plor formatted maps as is requested, and it creates only one MTZ formatted file with all Fourier map coefficients in it. The CCP4 or X-plor formatted maps can be computed in the entire unit cell or around selected atoms only. Twinning (if detected) will be accounted for automatically. This can be disabled by using "skip_twin_detection=True" keyword. All arrays used in map calculation, for example: Fobs, Fmodel, Fcalc, Fmask, m, D, etc., can be output into a CNS or MTZ formatted reflection file. For those who likes to experiment: bulk solvent correction and anisotropic scaling can be turned off, the data can be filtered by sigma and resolution. For some map types certain 'map_coefficients' or 'map' scope parameters may not be

applicable. For example, for "map_type=anomalous" the keywords "fill_missing_f_obs" and some other are not applicable. For LLG map calculation, if you specify the wavelength any existing heavy atoms (P or heavier) will be modeled as anomalous scatterers using the theoretical values of f' and f'' .

- phenix.maps is available in PHENIX GUI.
- The scope of parameters 'map_coefficients' defines the map that will be output as Fourier map coefficients. The scope of parameters 'map' defines the maps that will be output as CCP4 or X-plor format.
- To create several maps: duplicate either 'map_coefficients' or 'map' or both scopes of parameters as many times as many maps is desired. Then edit each of them to define the maps.
- A map is defined by specifying a map type using 'map_type' keyword available within each scope of parameters: 'map_coefficients' or 'map'. The general supported format for 'map_type' is: [p][m]Fo+[q][D]Fc[_filled]. For example: 2Fo-Fc, 2mFobs-DFcalc, 3Fobs-2Fmodel, Fo-Fc, mfobs-Dfcalc, anom, llg. The 'map_type' parser will automatically recognize which map is requested.
- The program creates as many files with CCP4 or X-plor formatted maps as is requested, and it creates only one MTZ formatted file with all Fourier map coefficients in it.
- The CCP4 or X-plor formatted maps can be computed in the entire unit cell or around selected atoms only.
- Twinning (if detected) will be accounted for automatically. This can be disabled by using "skip_twin_detection=True" keyword.
- All arrays used in map calculation, for example: Fobs, Fmodel, Fcalc, Fmask, m, D, etc., can be output into a CNS or MTZ formatted reflection file.
- For those who likes to experiment: bulk solvent correction and anisotropic scaling can be turned off, the data can be filtered by sigma and resolution.
- For some map types certain 'map_coefficients' or 'map' scope parameters may not be applicable. For example, for "map_type=anomalous" the keywords "fill_missing_f_obs" and some other are not applicable.
- For LLG map calculation, if you specify the wavelength any existing heavy atoms (P or heavier) will be modeled as anomalous scatterers using the theoretical values of f' and f'' .

All phenix.maps parameters:

```
maps {
  input {
    pdb_file_name = None
    reflection_data {
      file_name = None
      labels = None
      high_resolution = None
      low_resolution = None
      twin_law = None
      outliers_rejection = True
      french_wilson_scale = True
      french_wilson {
        max_bins = 60
        min_bin_size = 40
      }
      sigma_fobs_rejection_criterion = None
      sigma_iobs_rejection_criterion = None
      r_free_flags {
        file_name = None
        label = None
        required = True
      }
    }
  }
}
```

```

test_flag_value = None
ignore_r_free_flags = False
}
}
}
output {
directory = None
prefix = None
job_title = None
fmodel_data_file_format = mtz
include_r_free_flags = False
}
scattering_table = wk1995 it1992 *n_gaussian neutron electron
wavelength = None
bulk_solvent_correction = True
anisotropic_scaling = True
skip_twin_detection = False
omit {
method = *simple
selection = None
}
map_coefficients {
map_type = 2mFo-DFc
format = *mtz phs
mtz_label_amplitudes = 2FOFCWT
mtz_label_phases = PH2FOFCWT
fill_missing_f_obs = False
sharpening = False
sharpening_b_factor = None
exclude_free_r_reflections = False
isotropize = True
}
map_coefficients {
map_type = 2mFo-DFc
format = *mtz phs
mtz_label_amplitudes = 2FOFCWT_fill
mtz_label_phases = PH2FOFCWT_fill
fill_missing_f_obs = True
sharpening = False
sharpening_b_factor = None
exclude_free_r_reflections = False
isotropize = True
}
map_coefficients {
map_type = mFo-DFc
format = *mtz phs
mtz_label_amplitudes = FOFCWT
mtz_label_phases = PHFOFCWT
fill_missing_f_obs = False
sharpening = False
sharpening_b_factor = None
exclude_free_r_reflections = False
isotropize = True
}
map_coefficients {
map_type = anomalous
format = *mtz phs
mtz_label_amplitudes = ANOM
mtz_label_phases = PHANOM
fill_missing_f_obs = False
sharpening = False
sharpening_b_factor = None
exclude_free_r_reflections = False
isotropize = True
}
map {
map_type = 2mFo-DFc

```

```
format = xplor *ccp4
file_name = None
fill_missing_f_obs = False
grid_resolution_factor = 1/4.
region = *selection cell
atom_selection = None
atom_selection_buffer = 3
sharpening = False
sharpening_b_factor = None
exclude_free_r_reflections = False
isotropize = True
}
}
```

Planning a SAD experiment with plan_sad_experiment

plan_sad_experiment.html

Author(s)

- plan_sad_experiment: Tom Terwilliger

Purpose

plan_sad_experiment is a tool for estimating the anomalous signal that you might get from your SAD experiment and for predicting whether this signal would be sufficient to solve the structure.

plan_sad_experiment is normally used along with scale_and_merge and anomalous_signal to plan a SAD experiment, scale the data, and analyze the anomalous signal before solving the structure.

Usage

How plan_sad_experiment works:

- You supply plan_sad_experiment with a sequence file, the anomalously-scattering atom you plan to use for the experiment, and the wavelength for data collection.
- plan_sad_experiment will estimate the necessary I/σ_I of your dataset to provide enough anomalous signal to solve the structure.
- plan_sad_experiment will try various values of I/σ_I for your dataset at each of several resolutions. For each I/σ_I it will estimate the half-dataset anomalous correlation that would result along with the likely true correlation between your anomalous differences and those that would be calculated from a final model of your structure (cc^*_ano). From this anomalous correlation (cc^*_ano), plan_sad_experiment will estimate the anomalous signal (related to cc^*_ano by the square root of the number of reflections divided by the square root of the number of sites). Then plan_sad_experiment will choose a value of I/σ_I that gives an anomalous signal of about 30 (if achievable with the maximum I/σ_I you specify).
- The way that plan_sad_experiment and anomalous_signal estimate the probability that you can solve your dataset is to compare the anomalous signal in this dataset with the anomalous signal in other datasets at the same resolution. Then the fraction of similar datasets that can be solved by HySS is used as the probability that the anomalous substructure for your dataset will also be found.
- Similarly, the mean figure of merit for datasets with an estimated anomalous correlation (cc^*_ano) similar to that for your data is used as an estimate of the figure of merit that you would obtain if the substructure is found for your crystal.

Output from plan_sad_experiment

plan_sad_experiment provides a summary of the scattering expected from your crystal and a summary of the anomalous signal expected if you are able to measure your data with the suggested overall I/σ_I . You can set the maximum I/σ_I to look for. Here is an example setting `max_i_over_sigma=30`:

```
-----Dataset overall I/sigma required to solve a structure-----  
  
Dataset characteristics:  
  Target anomalous signal: 30.0  
  Residues: 325
```

Chain-type: PROTEIN
 Solvent_fraction: 0.50
 Atoms: 2642
 Anomalous-scattering atom: se
 Wavelength: 0.9792 Å
 Sites: 7
 f-double-prime: 3.84

Target anomalous scatterer:
 Atom: se f": 3.84 n: 7 rmsF: 10.2

Other anomalous scatterers in the structure:

Atom: C f": 0.00 n: 1674 rmsF: 0.1
 Atom: N f": 0.01 n: 445 rmsF: 0.1
 Atom: O f": 0.01 n: 514 rmsF: 0.3
 Atom: S f": 0.23 n: 10 rmsF: 0.7

Normalized anomalous scattering:

From target anomalous atoms $\text{rms}(x^2)/\text{rms}(F^2)$: 2.97
 From other anomalous atoms $\text{rms}(e^2)/\text{rms}(F^2)$: 0.24
 Correlation of useful to total anomalous scattering: 1.00

-----Dataset <I>/<sigma>; needed for anomalous signal of 15-30-----

-----Targets for entire dataset----- -----Likely outcome-----

Anomalous Useful Useful
 Half-dataset Anom CC Anomalous
 Dmin N I/sig I sigF/F CC (cc*_anom) Signal P(Substr) FOM
 (%) (%)

6.00	852	29	3.0	0.58	0.64	7	51	0.22
5.00	1473	29	3.0	0.62	0.66	9	79	0.15
3.00	6821	29	3.0	0.64	0.66	19	89	0.22
2.50	11787	29	3.0	0.70	0.68	25	96	0.19
2.00	23021	28	3.2	0.62	0.66	29	97	0.17
1.50	54569	13	6.7	0.18	0.42	29	97	0.15

Note: Target anomalous signal not achievable with tested I/sigma (up to 30)
 for resolutions of 2.50 Å and lower. I/sigma shown is value
 of $\max_i \frac{I_i}{\sigma_i}$.

This table says that if you collect your data to a resolution of 2.0 Å with
 an overall <I>/<sigma> of about 28 then the half-dataset anomalous
 correlation should be about 0.62 (typically within a factor of 2). This
 should lead to a correlation of your anomalous data to true anomalous
 differences (CC*_ano) of about 0.66, and a useful anomalous signal around
 29 (again within a factor of about two). With this value of estimated
 anomalous signal the probability of finding the anomalous substructure is
 about 96% (based on estimated anomalous signal and actual outcomes for
 real structures.), and the estimated figure of merit of phasing is 0.17.

The value of sigF/F (actually $\text{rms}(\text{sigF})/\text{rms}(F)$) is approximately the inverse
 of I/sigma. The calculations are based on $\text{rms}(\text{sigF})/\text{rms}(F)$.

Note that these values assume data measured with little radiation damage or at
 least with anomalous pairs measured close in time. The values also assume that
 the anomalously-scattering atoms are nearly as well-ordered as other atoms.
 If your crystal does not fit these assumptions it may be necessary to collect
 data with even higher I/sigma than indicated here.

Note also that anomalous signal is roughly proportional to the anomalous
 structure factors at a given resolution. That means that if you have 50%
 occupancy of your anomalous atoms, the signal will be 50% of what it otherwise
 would be. Also it means that if your anomalously scattering atoms only
 contribute to 5 Å, you should only consider data to 5 Å in this analysis.

What to do next:

1. Collect your data, trying to obtain a value of I/σ for the whole dataset at least as high as your target.
2. Scale and analyze your unmerged data with `phenix.scale\_and\_merge` to get accurate scaled and merged data as well as two half-dataset data files that can be used to estimate the quality of your data.
3. Analyze your anomalous data (the scaled merged data and the two half-dataset data files) with `phenix.anomalous\_signal` to estimate the anomalous signal in your data. This tool will again guess the fraction of the substructure that can be obtained with your data, this time with knowledge of the actual anomalous signal. It will also estimate the figure of merit of phasing that you can obtain once you solve the substructure.
4. Compare the anomalous signal in your measured data with the estimated values in the table above. If they are lower than expected you may need to collect more data to obtain the target anomalous signal.

Literature

List of all available keywords

`job_title` = None Job title in PHENIX GUI, not used on command line
`i_over_sigma` = None Optional I/σ . If supplied, the expected values of half-dataset correlation and `cc*_ano` based on this I/σ be calculated.
`max_i_over_sigma` = 100 Limit search of necessary I/σ to less than this value. You might increase this if you plan to do a very careful or very high-multiplicity experiment.
`i_over_sigma_range_low` = None If you set `i_over_sigma_range_low` and `i_over_sigma_range_high` then the value of `i_over_sigma` will be varied between these limits
`i_over_sigma_range_high` = None If you set `i_over_sigma_range_low` and `i_over_sigma_range_high` then the value of `i_over_sigma` will be varied between these limits
`steps` = 20 Number of steps for sampling ranges (i.e., `i_over_sigma_low` to `i_over_sigma_high`)
`min_in_bin` = 50 Minimum data points per bin in Bayesian estimation. Higher values smooth the predictor.
`target_signal` = 30. The anomalous signal that you would like to obtain. The value of I/σ will be adjusted to obtain this signal. Typically you will need a signal of 15-30 so solve the substructure.
`min_cc_ano` = 0.15 You can set the target minimum (true) anomalous correlation (`CC*_ano`). This value affects the phasing accuracy after the substructure is determined.
`ideal_cc_anom` = 0.75 The `ideal_cc_anom` is the expected anomalous correlation between an accurate model with isotropic anomalous scatterers and perfectly-measured data. The `ideal_cc_anom` is determined empirically. It is typically not unity because anomalous scatterers may have multiple locations with low occupancy or may be non-isotropic. A value of about 0.75 is a reasonable guess.
`include_weak_anomalous_scattering` = Auto At longer wavelengths the scattering of C, N, and O become significant relative to S. Default is to consider the scattering from C, N, O as noise. Additionally, (see `intrinsic_scatterers_as_noise`) if intrinsic anomalous scatterers (P and S) are weak, they will be counted as noise. This weak anomalous scattering is effectively noise and has the same effect as the `ideal_cc_anom` but it can be calculated from the composition. Its effects are added to those modeled by the `ideal_cc_anom` parameter. Default is to include weak anomalous scattering if a sequence file or the number of sulfurs is provided.
`intrinsic_scatterers_as_noise` = None Applies if `include_weak_anomalous_scattering`=True. You can choose to treat any intrinsic scatterers (S for protein, P for nucleic acid) as noise, just like any contributions from C, N, or O atoms. This is default if anomalous scattering (f-double-prime) from these atoms is less than half that of your specified anomalous scatterer. Alternatively these atoms are excluded from the noise calculation and are assumed to be included in the number of sites you specify.
`bayesian_updates` = False Use Bayesian updates of half-dataset CC and signal. First predict these values using standard approach, then use empirical half-dataset CC and signal for a training set of datasets to re-estimate these values. This helps correct for typical errors in measurement and typical resolution-dependent effects. Note that if you use `bayesian_updates`=True then the predictions may not vary smoothly with resolution or changes in parameters.
`input_filesdata` = None Data file (I or I+ and I- or F or F+ and F-). Any standard format is

fine.data_labels = None Optional label specifying which columns of anomalous data to use. Not necessary if your input file has only one set of anomalous data.
 crystal_inforesolution = None High-resolution limit. Either a high-resolution limit or a (Wilson) b_value or both is required
 b_value = None Estimated Wilson B-value for the dataset. Either a high-resolution limit or a (Wilson) b_value or both is required
 b_value_anomalous = None Estimated Wilson B-value for the anomalously-scattering atoms. Normally leave as None and it will be estimated from b_value.
 seq_file = None Optional sequence file (1-letter code). Separate chains with a blank line or line ...

- job_title = None Job title in PHENIX GUI, not used on command line
- i_over_sigma = None Optional I/σ . If supplied, the expected values of half-dataset correlation and CC^*_{ano} based on this I/σ be calculated.
- max_i_over_sigma = 100 Limit search of necessary I/σ to less than this value. You might increase this if you plan to do a very careful or very high-multiplicity experiment.
- i_over_sigma_range_low = None If you set i_over_sigma_range_low and i_over_sigma_range_high then the value of i_over_sigma will be varied between these limits
- i_over_sigma_range_high = None If you set i_over_sigma_range_low and i_over_sigma_range_high then the value of i_over_sigma will be varied between these limits
- steps = 20 Number of steps for sampling ranges (i.e., i_over_sigma_low to i_over_sigma_high)
- min_in_bin = 50 Minimum data points per bin in Bayesian estimation. Higher values smooth the predictor.
- target_signal = 30. The anomalous signal that you would like to obtain. The value of I/σ will be adjusted to obtain this signal. Typically you will need a signal of 15-30 so solve the substructure.
- min_cc_ano = 0.15 You can set the target minimum (true) anomalous correlation (CC^*_{ano}). This value affects the phasing accuracy after the substructure is determined.
- ideal_cc_anom = 0.75 The ideal_cc_anom is the expected anomalous correlation between an accurate model with isotropic anomalous scatterers and perfectly-measured data. The ideal_cc_anom is determined empirically. It is typically not unity because anomalous scatterers may have multiple locations with low occupancy or may be non-isotropic. A value of about 0.75 is a reasonable guess.
- include_weak_anomalous_scattering = Auto At longer wavelengths the scattering of C, N, and O become significant relative to S. Default is to consider the scattering from C, N, O as noise. Additionally, (see intrinsic_scatterers_as_noise) if intrinsic anomalous scatterers (P and S) are weak, they will be counted as noise. This weak anomalous scattering is effectively noise and has the same effect as the ideal_cc_anom but it can be calculated from the composition. Its effects are added to those modeled by the ideal_cc_anom parameter. Default is to include weak anomalous scattering if a sequence file or the number of sulfurs is provided
- intrinsic_scatterers_as_noise = None Applies if include_weak_anomalous_scattering=True. You can choose to treat any intrinsic scatterers (S for protein, P for nucleic acid) as noise, just like any contributions from C, N, or O atoms. This is default if anomalous scattering (f'') from these atoms is less than half that of your specified anomalous scatterer. Alternatively these atoms are excluded from the noise calculation and are assumed to be included in the number of sites you specify.
- bayesian_updates = False Use Bayesian updates of half-dataset CC and signal. First predict these values using standard approach, then use empirical half-dataset CC and signal for a training set of datasets to re-estimate these values. This helps correct for typical errors in measurement and typical resolution-dependent effects. Note that if you use bayesian_updates=True then the predictions may not vary smoothly with resolution or changes in parameters.

- `input_filesdata` = None Data file (I or I+ and I- or F or F+ and F-). Any standard format is fine.`data_labels` = None Optional label specifying which columns of anomalous data to use. Not necessary if your input file has only one set of anomalous data.

- `data` = None Data file (I or I+ and I- or F or F+ and F-). Any standard format is fine.

- `data_labels` = None Optional label specifying which columns of anomalous data to use. Not necessary if your input file has only one set of anomalous data.

- `crystal_inforesolution` = None High-resolution limit. Either a high-resolution limit or a (Wilson) `b_value` or both is required.`b_value` = None Estimated Wilson B-value for the dataset. Either a high-resolution limit or a (Wilson) `b_value` or both is required.`b_value_anomalous` = None Estimated Wilson B-value for the anomalously-scattering atoms. Normally leave as None and it will be estimated from `b_value`.`seq_file` = None Optional sequence file (1-letter code). Separate chains with a blank line or line starting with `>`.`chain_type` = *PROTEIN RNA DNA Chain type (PROTEIN RNA DNA). This is used to estimate the number of atoms from the number of residues.`ncs_copies` = None Optional estimate of NCS copies in your crystals (only used if a data file is supplied).`solvent_fraction` = None Optional estimate of solvent fraction in your crystals (0 to 1).`residues` = None The number of residues in the molecule or asymmetric unit. Note that it is the ratio of residues to anomalously-scattering atoms that matters.`atom_type` = None Optional name of anomalously-scattering atom. If supplied, you also need to supply the wavelength for X-ray data collection. If not supplied, then you need to supply a value for `f_double_prime`.`number_of_s` = None You can specify the number of S atoms in the asymmetric unit. Only used if `include_weak_anomalous_scattering`=True. If not set, the number is guessed from the sequence file if present.`f_double_prime` = None F-double-prime value for the anomalously-scattering atom. Alternatively you can specify the atom type and wavelength.`wavelength` = None Wavelength for X-ray data collection. If supplied, also specify the `atom_type`. Alternatively you can specify the value of `f_double_prime`.`sites` = None The number of anomalously-scattering atoms in the molecule or asymmetric unit. Note that it is the ratio of residues to anomalously-scattering atoms that matters.`sites_min` = None If you set `sites_min` and `sites_max` and not `sites` the sites will be varied from `sites_min` to `sites_max`.`sites_max` = None If you set `sites_min` and `sites_max` and not `sites` the sites will be varied from `sites_min` to `sites_max`.`occupancy` = 1 Estimate of occupancy of anomalously-scattering atoms

- `resolution` = None High-resolution limit. Either a high-resolution limit or a (Wilson) `b_value` or both is required

- `b_value` = None Estimated Wilson B-value for the dataset. Either a high-resolution limit or a (Wilson) `b_value` or both is required

- `b_value_anomalous` = None Estimated Wilson B-value for the anomalously-scattering atoms. Normally leave as None and it will be estimated from `b_value`.

- `seq_file` = None Optional sequence file (1-letter code). Separate chains with a blank line or line starting with `>`.

- `chain_type` = *PROTEIN RNA DNA Chain type (PROTEIN RNA DNA). This is used to estimate the number of atoms from the number of residues

- `ncs_copies` = None Optional estimate of NCS copies in your crystals (only used if a data file is supplied).

- `solvent_fraction` = None Optional estimate of solvent fraction in your crystals (0 to 1)

- `residues` = None The number of residues in the molecule or asymmetric unit. Note that it is the ratio of residues to anomalously-scattering atoms that matters.

- `atom_type` = None Optional name of anomalously-scattering atom. If supplied, you also need to supply the wavelength for X-ray data collection. If not supplied, then you need to supply a value for `f_double_prime`.

- `number_of_s = None` You can specify the number of S atoms in the asymmetric unit. Only used if `include_weak_anomalous_scattering=True`. If not set, the number is guessed from the sequence file if present.
- `f_double_prime = None` F-double-prime value for the anomalously-scattering atom. Alternatively you can specify the atom type and wavelength.
- `wavelength = None` Wavelength for X-ray data collection. If supplied, also specify the `atom_type`. Alternatively you can specify the value of `f_double_prime`
- `sites = None` The number of anomalously-scattering atoms in the molecule or asymmetric unit. Note that it is the ratio of residues to anomalously-scattering atoms that matters.
- `sites_min = None` If you set `sites_min` and `sites_max` and not `sites` the sites will be varied from `sites_min` to `sites_max`
- `sites_max = None` If you set `sites_min` and `sites_max` and not `sites` the sites will be varied from `sites_min` to `sites_max`
- `occupancy = 1` Estimate of occupancy of anomalously-scattering atoms
- `controlfixed_resolution = False` Only run calculation at high_resolution limit
`show_summary = False` Show summary only
`verbose = False` Verbose output
- `fixed_resolution = False` Only run calculation at high_resolution limit
- `show_summary = False` Show summary only
- `verbose = False` Verbose output